Part VI

Visual SLAM and the Vision-Navigated Robot

Simultaneous Localization and Mapping (SLAM)

How to Build a Vision-Based Navigation and Localization Drone



Editor: Lee Shihyuan (李世淵)

LINE OA @prof

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Part VI Visual SLAM and the Vision-Navigated Robot

Chapter 20

An Introduction to Visual SLAM

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Part VI changes the sensor but not the line of thought. Part V used a laser to measure the three-dimensional world, and in Chapter 19 it carried that technology into an inspection site that must run every day for a year; this part uses a camera to look instead. Students will quickly find, however, that not one of LOAM's four steps—extract features, estimate quickly at the front end, optimize finely at the back end, correct by loop closure—is missing from visual SLAM. The difference lies only in the nature of the features: a LiDAR's features are geometric and come into the world already carrying three-dimensional coordinates, while a camera's features are features of appearance, with only a two-dimensional pixel position, and their depth must be obtained some other way.

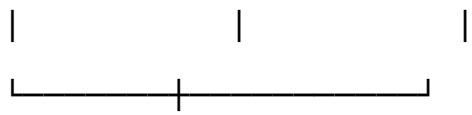
That difference gives this part its two themes. The first is how to choose features and how to match them, which is the content of Sections 20.3 and 20.4 of this chapter; the second is what to do without depth, which is the scale ambiguity of Section 20.5 and the question epipolar geometry sets out to answer in Chapter 21. One more thing is worth keeping in mind from the start: what Section 19.5 measured—a long corridor whose geometric degeneracy leaves a LiDAR unable to say how far it has traveled—is exactly where a camera is at its best, because the posters on the wall, the room numbers, and the texture of the floor are all distinct visual features. Before any of that, this chapter puts the whole framework in place so that students can see which piece each of the next four chapters supplies.

Figure 20-0 The Link Map of Chapter 20

The Foundations Already Laid

Chapter 9 The camera model Chapter 12 Optimization Chapters 18 and 19 LOAM and field

experience



▼

【Chapter 20 Visual SLAM: the framework and the fundamentals of features】

Feature-based / direct methods ─ ORB ─ matching and rejection ─ scale ambiguity

▼

Chapter 21 Epipolar geometry (where depth comes from)

Chapter 22 ORB-SLAM2 Chapter 23 ORB-SLAM3 (visual-inertial)

▼

Part VII Miniature robots and drones Part VIII ROS implementation Part IX AI and semantic navigation

Learning Objectives

In Part V we used a laser to measure the three-dimensional world; from this part on we use a camera to look at it instead. The sensor has changed, but students will find the central ideas entirely common to both—features, registration, optimization, loop closure—only with image pixels in place of a geometric point cloud. This chapter is the doorway to Part VI, and it establishes the overall framework of visual SLAM together with the fundamentals of feature technique. After studying this chapter, students should be able to:

1. Describe the basic pipeline of visual SLAM and indicate which chapters of this book correspond to each of its stages.
2. Compare the principles, the strengths and weaknesses, and the proper domains of the two great schools, the feature-based method and the direct method.

3. Explain the role and the characteristics of ORB, SIFT, SURF, FAST, and BRIEF, and understand why ORB became the mainstream choice for SLAM.
4. Distinguish feature tracking from feature matching and describe the common strategies for rejecting false matches.
5. Explain the scale ambiguity of monocular SLAM and the three ways of resolving it.
6. Compare stereo SLAM with RGB-D SLAM in their differences and their proper domains.
7. Give examples of applications of visual SLAM in AR / VR and in robotics.

Core Concepts

Chinese Term	English Term	Explanation
視覺 SLAM	Visual SLAM	Simultaneous localization and mapping with a camera as the principal sensor.
視覺里程計	Visual Odometry	Estimating camera motion from consecutive images; it accumulates drift.
特徵法	Feature-based / Indirect	Extracts and matches feature points, then solves for motion from the geometric relation.
直接法	Direct Method	Extracts no features and estimates motion directly from differences of pixel brightness.
關鍵點	Keypoint	A salient and repeatably detectable position in an image, such as a corner.
描述子	Descriptor	A vector describing the appearance around a key point, used for comparison.
尺度不變性	Scale Invariance	The ability to detect the same feature as

Chinese Term	English Term	Explanation
		an object's distance changes.
旋轉不變性	Rotation Invariance	The ability to match correctly after the image has been rotated.
漢明距離	Hamming Distance	The number of differing bits between two binary descriptors; extremely fast to compare.
特徵匹配	Feature Matching	Finding the same feature point in correspondence between two images.
特徵追蹤	Feature Tracking	Following the position of the same feature through consecutive images.
尺度不確定性	Scale Ambiguity	The inherent limitation that a monocular camera cannot determine the true size of the map.
詞袋模型	Bag of Words	Representing an image as a statistic over visual words, used for place recognition.

20.1 The Basic Pipeline of Visual SLAM

In Section 9.6 we drew a ladder of knowledge: image processing → feature extraction → multiple view geometry → visual odometry → visual SLAM. This section unfolds that ladder into a complete system pipeline.

Five Stages

1. Sensor data acquisition and preprocessing: read the image and undistort it (the camera calibration of Section 9.2). For a stereo or RGB-D camera, stereo matching or depth alignment is also required.
2. Visual odometry (the front end): estimate the camera motion between consecutive images and triangulate local three-dimensional points. This stage is fast but accumulates drift (the LiDAR odometry of Chapter 18 is its laser counterpart).
3. Back-end optimization: perform a nonlinear optimization over a batch of poses and map points (the bundle adjustment of Chapter 12) to remove the accumulated error.
4. Loop closure detection: recognize that this is a place I have been before, and generate a loop closure constraint to correct the global drift (Chapters 12 and 15). Visual SLAM commonly uses a bag-of-words model for this.
5. Mapping: output a map according to the needs of the application—a sparse map of feature points, a dense point cloud, or a semantic map (Chapters 1 and 3).

Figure 20-1 The Standard Pipeline of Visual SLAM

Image input (monocular / stereo / RGB-D)

↓

① Preprocessing: undistortion, stereo matching (Chapter 9)

↓

② Visual odometry (the front end): feature extraction and matching → estimating camera motion (Chapter 21)

↓

③ Back-end optimization: bundle adjustment refining poses and map points (Chapter 12)

↓

④ Loop closure detection: recognizing a revisited place by bag of words → global correction (Chapter 22)

↓

⑤ Map output: sparse point cloud / dense map / semantic map (Chapter 32)

A Comparison with LiDAR SLAM

Set this pipeline alongside the LOAM of Chapter 18 and the architectures prove strikingly alike: both run feature extraction → fast front-end estimation → fine back-end optimization → loop closure correction. The difference lies only in the nature of the features:

- A LiDAR's features are geometric (edges, planes) and carry three-dimensional coordinates and ranges directly.

- A camera's features are features of appearance (a corner and the texture around it), with only two-dimensional pixel coordinates; their depth must be obtained separately.

That difference gives rise to the central difficulty of visual SLAM: how to recover three-dimensional camera motion and scene structure from two-dimensional observations that carry no depth. This is precisely the question epipolar geometry sets out to answer in Chapter 21.

20.2 Feature-Based Methods and Direct Methods

Visual SLAM has two principal technical routes, and to understand how they diverge is to understand the lay of the land in this field.

The Feature-Based Method (also called the indirect method)

The feature-based method proceeds in three steps:

6. Detection: find key points such as corners in the image.
7. Description: compute a descriptor for each key point, recording what its surroundings look like.
8. Matching: find corresponding points between different images according to the similarity of their descriptors.

The feature-based method is widely adopted in visual odometry because it is robust to camera motion, to changes of illumination, and to moving objects. It is called indirect because it does not use pixel values directly but first abstracts the image into the intermediate representation of a set of feature points, and then solves for motion by geometric means.

- Advantages: robust to changes of illumination (descriptors generally have some invariance to brightness); able to handle fairly large changes of viewpoint; easy to use for loop closure detection (bag-of-words comparison of descriptors); mature and stable.
- Disadvantages: computing features and descriptors takes time; most of the pixel information is thrown away; where texture is scarce (a white wall) no features are found at all and the method fails (the curse of the white wall of Chapter 9).

The Direct Method

The direct method skips feature extraction entirely and uses pixel brightness directly. Its central assumption is brightness constancy: the same point in space should have the same brightness in different images. Estimating motion then becomes this: find a camera motion such that projecting the pixels of the previous frame into the current frame under that motion makes the difference in brightness as small as possible.

- Advantages: no features or descriptors need be computed, so it can be very fast; it can use every pixel in the image that has a gradient (not only corners), and so it still works in scenes where texture is scarce but a faint gradient remains; and it produces a semi-dense or dense map naturally.
- Disadvantages: the assumption of brightness constancy is fragile—changes of illumination, automatic exposure, and non-Lambertian surfaces all break it; convergence is difficult under large motion (a good initial value is needed); and loop closure detection is harder to do.

Representative systems include LSD-SLAM and DSO (Direct Sparse Odometry).

Item of Comparison	Feature-based (indirect)	Direct
Information used	Feature points and descriptors	Pixel brightness
Central assumption	Features can be detected and matched repeatably	Brightness constancy
Against changes of illumination	Fairly robust	Sensitive
Scenes of weak texture	Fails easily	Can still use a faint gradient
Large motion	Handles it fairly well	Diverges easily; needs a good initial value
Loop closure detection	Easy (bag of words)	Harder
Map density	Sparse	Semi-dense or dense
Representative systems	The ORB-SLAM family	LSD-SLAM, DSO

Table 20-1 A Comparison of Feature-Based and Direct Methods

Part VI of this book takes the feature-based method as its main line (the ORB-SLAM family of Chapters 21 through 23), because it is mature and stable, richly resourced, and best suited to teaching. But students should know that the direct method exists—and recent years have also brought hybrid methods that combine the strengths of both.

20.3 ORB, SIFT, SURF, FAST, and BRIEF

These five names often appear together, but their roles are in fact different: some are responsible only for detection, some only for description, and some do both.

Getting this straight is the first point of this section.

Detectors and Descriptors: a Division of Labor

- FAST: a detector only. It finds corners extremely quickly but produces no descriptor.
- BRIEF: a descriptor only. It produces a binary descriptor vector for a given key point but is not responsible for finding points.
- SIFT, SURF, ORB: both at once, detecting as well as describing.

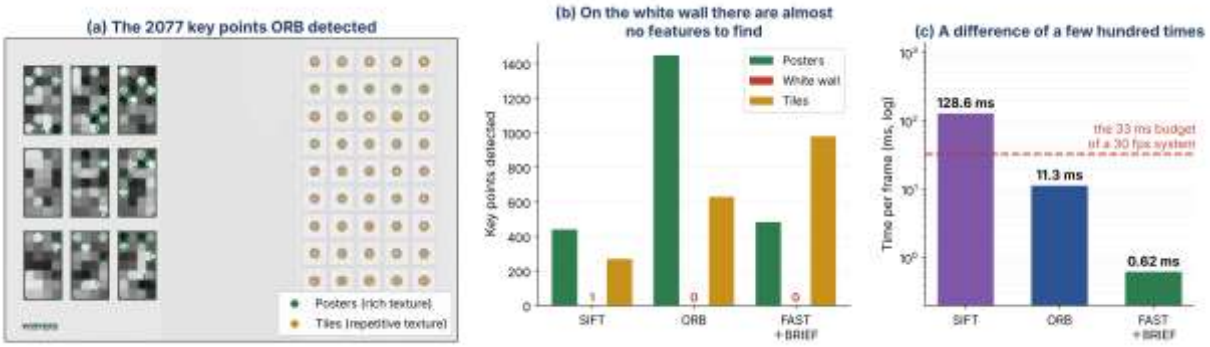
In practice these methods can be used independently and separately as detectors or as descriptors, and different combinations give different performance. This explains why combined forms such as FAST-BRIEF and SIFT-SURF appear in the literature.

An Introduction to the Five Principal Algorithms

- SIFT (Scale-Invariant Feature Transform): the classic of classics. It finds extrema across different scales by a difference of Gaussians and produces a 128-dimensional descriptor with excellent invariance to scale and rotation. SIFT essentially merges feature detection and the generation of the description vector into a single key derivation step. Its drawback is its computational cost.
- SURF (Speeded-Up Robust Features): an accelerated version of SIFT that approximates the Gaussian operations with an integral image and box filters. On the whole SURF demands less computation than SIFT.
- FAST (Features from Accelerated Segment Test): simple and fast, it compares a pixel with the pixels on a Bresenham circle around it, first checking whether the pixel is brighter or darker than N consecutive pixels on that circle; if it is, it may be a corner. The algorithm then applies non-maximum suppression to identify and retain the most discriminative key points.

- BRIEF (Binary Robust Independent Elementary Features): produces a binary string descriptor by comparing the brightness of randomly chosen pairs of points. Its advantages are that it is extremely fast to produce and that comparison requires only a Hamming distance (a bitwise operation), which is highly efficient. Its drawback is that the original version has neither rotation nor scale invariance.
- ORB (Oriented FAST and Rotated BRIEF): combines FAST and BRIEF while improving each—it adds orientation information to the FAST corner (oriented FAST) and lets the BRIEF descriptor rotate with that orientation (rotated BRIEF), thereby gaining rotation invariance.

Figure 20-2 One Wall, Three Feature Algorithms: Number, Distribution, and Cost



To give the comparison a control, this book synthesized a classroom wall deliberately divided into three regions: on the left are nine different posters (rich and distinctive texture), in the middle a stretch of nearly textureless white wall, and on the right a row of identical tiles (repetitive texture). These three regions correspond exactly to the three situations visual SLAM will meet.

Panel (a) draws the 2,077 key points ORB detected onto the image, and the result is plain at a glance: the poster region is thick with them, the tile region has a good many as well, and on that stretch of white wall in the middle there is not a single point. Panel

(b) sets the per-region counts of the three algorithms side by side; the numbers for the white wall region are 1 for SIFT, 0 for ORB, and 0 for FAST—the most direct evidence there is for the curse of the white wall of Chapter 9. Note that this is not a matter of one algorithm being inferior: a place with no variation in brightness simply has no information to extract, and all three methods fail there impartially.

Panel (c) is the reason for choosing ORB. On the same 960×640 image, SIFT takes 128.6 milliseconds per frame, ORB 11.3 milliseconds, and plain FAST detection only 0.62 milliseconds. SIFT is more than eleven times slower than ORB and more than two hundred times slower than FAST. The red dashed line marks the total budget of 33 milliseconds per frame for a 30 fps system—and into those 33 milliseconds must also be fitted matching, pose solving, and the map update. SIFT overruns the budget nearly fourfold on feature extraction alone, while ORB uses up only a third of it. What Case 20-1 calls the cost and the reward of speed is exactly this red line.

Panel (b) also hides a trap worth noting now: the number of features in the tile region is in fact considerable, with ORB finding 628 points there and FAST as many as 979. A large number does not mean a useful one—Figure 20-3 in the next section will reveal that these points are almost all a source of disaster.

The Experimental Conditions of This Figure

The image is a 960×640 grayscale synthetic image; ORB is capped at 2000 points, SIFT and FAST use OpenCV's default parameters, and the times are averages of five repetitions on the same image.

The absolute numbers will vary with image size, parameters, and hardware; what matters is the difference in order of magnitude: more than an order of

magnitude between SIFT and ORB, a conclusion that is quite robust.

The original ORB paper reports it to be about 14 times faster than SURF and SURF about 24 times faster than SIFT (Section 20.3), and the SIFT / ORB ratio in this experiment agrees with that magnitude.

Students may rerun this in Part One of Activity 20-1 with classroom photographs of their own, and should look in particular for a white wall and a tiled floor to reproduce the three situations of panel (b).

Why ORB Became the Mainstream in SLAM

The answer is speed. ORB's developers report it to be 14 times faster than SURF, and SURF in turn 24 times faster than SIFT; this newer algorithm also performs roughly as well, and in some situations considerably better. Other literature puts ORB's extraction speed at about 15 times that of SURF while retaining robustness to changes of rotation and scale, an efficiency that makes it particularly suited to real-time applications.

Among the various feature extraction algorithms of visual SLAM, the advantage of the ORB feature lies in the speed of extraction and matching, while little accuracy is lost against SIFT and SURF. ORB has therefore become the mainstream method of feature point detection in SLAM systems—it trades a portion of accuracy and robustness for a gain in computational speed, a bargain that is extremely favorable for a real-time system.

The Ingenuity of the ORB Descriptor

A detail of the design of the ORB descriptor is worth mentioning. In ORB the BRIEF descriptor is a 256-bit vector, each bit of which records the result of comparing the

mean intensities of two 5×5 pixel windows within the same 31×31 pixel patch; there are 676 such windows in a patch, and therefore more than two hundred thousand possible pairwise comparisons. To maximize the discriminative power of the descriptor, ORB selects from those two hundred thousand combinations the 256 comparisons that are farthest from the mean and least correlated with one another.

This design displays an important kind of engineering thinking: not choosing at random, but picking the most informative comparisons in a data-driven way. Should students one day design features or metrics of their own, this principle of maximizing discriminative power while minimizing redundancy is worth taking as a model.

Algorithm	Role	Form of descriptor	Speed	Invariance
SIFT	Detection + description	128-dimensional floating point	Slowest	Scale, rotation
SURF	Detection + description	64 / 128-dimensional floating point	Medium	Scale, rotation
FAST	Detection only	None	Extremely fast	None (original version)
BRIEF	Description only	Binary string	Extremely fast	None (original version)
ORB	Detection + description	256-bit binary	Fast	Rotation, and scale in part

Table 20-2 A Comparison of the Five Feature Algorithms

20.4 Feature Tracking and Feature Matching

Once features have been found, the next step is to establish correspondences

between different images. There are two lines of thought here, which students often confuse; this section sets them apart.

Feature Matching: Comparing Descriptors

Matching works like this: extract features and descriptors independently in each of two images, then, for every feature of the first, find the one in the second whose descriptor is most similar.

- Similarity measure: floating-point descriptors (SIFT, SURF) use the Euclidean distance; binary descriptors (ORB, BRIEF) use the Hamming distance—one need only count how many bits differ between two bit strings, which an XOR and a bit count accomplish extremely quickly. This is the greatest advantage of the binary descriptor.
- Where it applies: when the two images differ considerably (a changed viewpoint, a long interval of time, loop closure detection), matching is the only workable method.

Feature Tracking: Searching Nearby

Tracking works like this: after extracting features in the first image, do not extract them again in the second, but assume that the features have moved only a short distance and search directly near their original positions for the most similar image patch. The typical method is optical flow (Section 9.4).

- Advantages: no descriptor need be computed, so it is faster; and it works well on consecutive images.
- Limitations: it applies only where the image changes little (a high frame rate, slow motion); and after tracking for a while drift accumulates, so a feature may be lost or wander off onto something else.

False Matches: a Reality That Must Be Faced

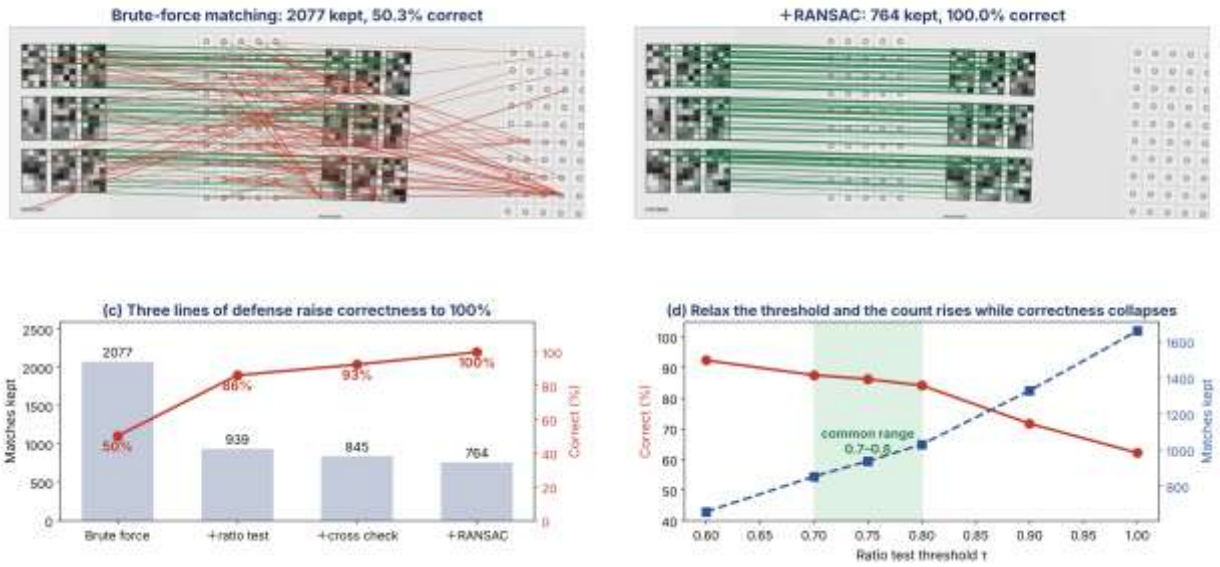
Whether one matches or tracks, wrong correspondences are inevitable. The causes include repetitive texture (tiles, railings, leaves, all looking the same everywhere), too great a change of viewpoint, changed illumination, and moving objects. And in a geometric solution a few severe false matches may be enough to pull the result off course—one of the most common causes of failure in visual SLAM.

Three Strategies for Rejecting False Matches

9. The ratio test: for each feature, find the two most similar candidates, the best and the second best. If their distances are close together, the match is hard to distinguish and therefore unreliable, and it is discarded. This is the classic approach proposed in the SIFT paper, simple and effective.
10. The cross check: the feature a of image A matches the feature b of image B, and b in turn matches back to a; only then is the match accepted. A one-way match is taken as unreliable.
11. Geometric verification (RANSAC): the most important move of all. Using the geometric constraint that all correct matches must conform to one and the same camera motion, it samples at random and solves repeatedly, counts those that conform, and rejects as outliers those that do not. In practice, estimating the essential matrix and the inlier set with RANSAC is the standard way of estimating the relative pose of a camera from feature correspondences that contain outliers. This method will be unfolded in detail in Chapter 21.

Figure 20-3 Three Lines of Defense Against False Matches: from 50 Percent

Correct to 100 Percent



The effect of rejecting false matches can usually be judged only by eye, from whether the lines look orderly; but if the second image has been produced from the first by a known homography, then every match has a right answer—project the feature point of the first image through that matrix, and if the result falls within four pixels of the matched point, the match is correct. This figure establishes its ground truth in exactly that way.

Panel (a) is brute-force matching with no rejection at all: 2,077 matches, the green lines correct and the red ones wrong, for a correctness of 50.3 percent—the same as tossing a coin. Note that the red lines are not evenly distributed: a great many of them run from the tile region on the right across to the poster region on the left, a plainly systematic confusion. Panel (b) is the result after all three lines of defense: 764 matches at 100 percent correct, the lines neatly displaying a single direction of translation.

Panel (c) takes the process apart. The ratio test cuts 2,077 matches down to 939 at a stroke, and correctness leaps from 50.3 percent to 86.3 percent—this is the decisive cut. The cross check trims further to 845 at 92.7 percent; and finally RANSAC clears away

the remaining outliers by geometric consistency, leaving 764 at 100 percent. The three lines of defense discard 63 percent of the matches in all, and what they buy is 764 correspondences that can be trusted completely. For back-end optimization this is an excellent bargain: a few correct constraints are far better than a great many constraints adulterated with wrong ones.

Panel (d) answers the second discussion question. As the ratio threshold τ is relaxed from 0.6 to 1.0, the number of matches retained rises from 659 to 1,661, but correctness collapses all the way from 92.4 percent to 62.2 percent. The curve turns sharply after 0.8—which explains why the values recommended in the literature almost all fall between 0.7 and 0.8 (the green band in the figure). The meaning of τ is in fact quite intuitive: it asks whether the best candidate is appreciably better than the second best, and $\tau = 1.0$ amounts to not asking at all, degenerating naturally into brute-force matching.

What Happened in the Tile Region: Case 20-2 Quantified

Region by region, brute-force matching is 69.8 percent correct in the poster region and only 1.0 percent in the tile region—of 598 matches, just 6 are right. Repetitive texture is not merely harder; it is all but bound to go wrong.

The ratio test behaves quite differently in the two regions as well: in the poster region it keeps 587 of 1,479 matches (about four tenths), while in the tile region it keeps only 10 of 598 (under 2 percent).

This is precisely the design intent of the ratio test: where everything looks alike, the distances of the best and second-best candidates are bound to be close, and the algorithm honestly says it cannot tell them apart and gives up, rather

than forcing a choice.

After RANSAC at the end, only 2 matches remain in the tile region. Few, but right—and far safer than keeping 598 of which 99 percent are wrong.

20.5 The Scale Ambiguity of Monocular SLAM

This is the most fundamental and the most inescapable limitation of monocular visual SLAM. Section 9.1 pointed out its geometric root in the pinhole projection formula; this section explains its consequences and its remedies.

The Nature of the Problem

Recall the projection formula of Section 9.1: $x = f \cdot X/Z$. Note that in this expression, if the three-dimensional coordinates of every point in the scene are scaled up by a factor of k while every translation of the camera is scaled up by k as well, the result projected onto the image is exactly the same.

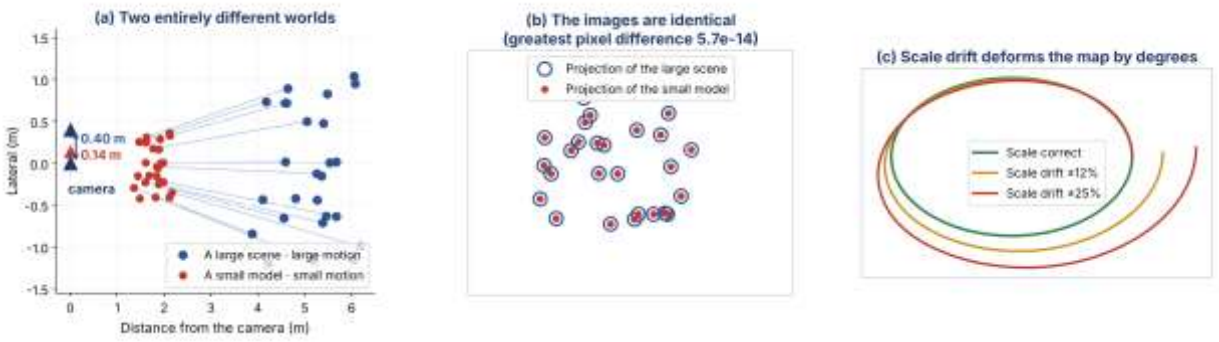
In other words, a sequence of images taken by a monocular camera may correspond to real motion in a real room, or equally to a minute motion in a model house—and the two cannot be told apart in the image at all. Monocular SLAM can therefore recover a trajectory and a map only up to an unknown scale factor.

Consequences

- The map has no true size: the map built can say that this wall is 3.7 units from that one, but not how many meters a unit is.
- It cannot be used for navigation directly: the robot does not know whether the obstacle ahead is 0.5 meters or 5 meters away, and so of course cannot avoid it safely.

- Scale drift: more troublesome still, this unknown scale factor also changes slowly over a long run (called scale drift), so that the map gradually deforms.

Figure 20-4 Scale Ambiguity: Two Worlds a Monocular Camera Is Inherently Unable to Tell Apart



Section 20.5 says that scaling the scene by k and the camera translation by k as well leaves the image exactly the same, and that statement can be verified by direct calculation. Panel (a) is a side view: in blue is a real room, with points distributed between 3.6 and 6.2 meters from the camera and the camera translating 0.40 meters; in red the whole scene and the translation have been scaled down to 0.35 times—a model house set in front of one's eyes, with the camera moving only 0.14 meters. Physically the two are entirely different worlds.

Panel (b) projects the two sets of three-dimensional points into images with the same camera intrinsics. The hollow blue circles are the projection of the large scene and the solid red dots that of the small model—they are not merely close together but exactly coincident. This book actually computed the greatest pixel difference between the two projections: 1.1×10^{-13}, which is to say the rounding error of floating-point arithmetic, and mathematically zero. This is not a matter of the algorithm not being clever enough; it is decided by the projection equation itself: a monocular image simply carries no information about scale.

Panel (c) then displays what this ambiguity actually looks like in practice—scale drift. The green line is the true trajectory at the correct scale, a closed ellipse. If the scale factor of each displacement drifts slowly as a random walk by ±12 percent (the amber line) or ±25 percent (the red line), the trajectory gradually deforms and its two ends no longer meet. Note that the direction of every step here is correct; the only thing wrong is how far each step went, yet the accumulation is enough to distort the whole map.

This is why monocular SLAM is almost never used alone. The three remedies of Section 20.5—add an IMU, change to a stereo or RGB-D camera, or introduce a reference object of known size—all amount to supplying a definition of the meter from outside. It is worth mentioning that this limitation has not been broken to this day: even the newest monocular SLAM systems built around a foundation model, to be introduced in Section 20.8, still optimize on Sim(3) and still perform a scale alignment before reporting their evaluation—because what they face is the same geometric fact.

The Difference Between Scale and Depth Estimation

A deep learning model (of the Depth Anything kind) can guess depth from a single image, but that is a prior learned from training data—seeing a door frame, it infers a height of about 2 meters, rather than measuring it.

Such a prior works well in ordinary scenes and errs systematically in scenes of unusual size: a model house, an enlarged prop, an aerial top-down view are all typical failures.

By contrast, an accelerometer measures acceleration in physical units and a stereo baseline is a fixed distance measured with a ruler—what these two supply is a measurement, not a guess.

Practical advice: where safety must be assured (obstacle avoidance, grasping, navigation), use a measurement; where a visual effect is wanted (AR overlays, dense reconstruction), a prior is acceptable.

Three Ways of Resolving It

12. Introduce an IMU (visual-inertial): an accelerometer measures acceleration in physical units, and integrating it supplies true scale. This is the most general remedy, and it is the central theme of Chapter 7 and Chapter 23. The price is that sufficient excitation of motion is required before scale becomes observable (Section 7.7).
13. Use a stereo or RGB-D camera: the baseline of a stereo pair is a known fixed distance, and an RGB-D camera outputs depth in meters directly. Solving the problem in hardware is the least troublesome approach (Section 20.6).
14. A reference object of known size: if the scene contains an object of known size (an AprilTag or a checkerboard, as in Chapter 4), or if the height of the camera above the ground is known, the scale can be recovered. This is quite practical in particular applications.

It should be added that recent years have also brought the practice of estimating depth from a single image by deep learning (models of the Depth Anything kind in Section 9.4) to supply a prior on scale, though its accuracy and reliability still fall short of the three physical methods above.

20.6 Stereo SLAM and RGB-D SLAM

Section 9.3 introduced three ways for a camera to obtain depth; this section focuses on how they differ within a SLAM system.

Stereo SLAM

A stereo camera photographs with two lenses of known baseline at the same moment and computes depth directly from the disparity between the left and right images (the formula Z = f·b/d of Section 9.6).

- Advantages: depth from a single frame; absolute scale; operation indoors and out alike (it does not depend on active projection); and a measuring range that can be extended by enlarging the baseline.
- Limitations: the relative position of the two cameras (the extrinsics) must be calibrated precisely; on a textureless surface no correspondence is found and so there is no depth; the depth error grows with the square of the distance (Section 9.6); and the computational load is fairly heavy (stereo matching must be performed).
- Where it applies: outdoor robots, drones, self-driving cars—anywhere that must work in sunlight and over a fairly large range of distances.

RGB-D SLAM

An RGB-D camera actively projects an infrared pattern to obtain dense depth (the structured light or active infrared stereo of Section 9.3).

- Advantages: it outputs the depth of every pixel directly, with no stereo matching to do oneself; it has depth even on a textureless white wall (because the projection is active); it has absolute scale; and a dense three-dimensional map is easily built.
- Limitations: it mostly relies on infrared and fails in strong sunlight; its effective range is usually between 0.2 and 10 meters; it is equally helpless before glass

and mirrors (the same weakness as the LiDAR of Chapter 8); and its power consumption is fairly high.

- Where it applies: indoor service robots, desktop three-dimensional reconstruction, indoor surveying—which is also why the implementation of Chapter 30 chooses RGB-D.

Item of Comparison	Monocular SLAM	Stereo SLAM	RGB-D SLAM
Absolute scale	None (must be supplied externally)	Yes	Yes
Source of depth	Triangulation from motion	Left-right disparity	Active projection
Textureless surfaces	Fails	No depth	Still has depth
Strong sunlight outdoors	Usable	Usable	Mostly fails
Effective range	Unlimited (but without scale)	Fairly far	About 0.2–10 m
Map density	Sparse	Can be semi-dense	Dense
Hardware cost	Lowest	Medium	Medium

Table 20-3 A Comparison of the Three Visual SLAM Configurations

One great merit of ORB-SLAM2 and ORB-SLAM3 (Chapters 22 and 23) is that they support all three of these modes at once—one and the same system, with only the input mode changed. This lets students compare the three within a single framework,

which makes excellent teaching material.

20.7 Applications of Visual SLAM in AR / VR and in Robotics

Section 1.4 of Chapter 1 mentioned AR / VR as one of the four great applications of SLAM; this section goes deeper into the role of visual SLAM within it and extends to robotic applications.

AR / VR: Visual SLAM in Your Pocket

For AR glasses or AR on a phone to pin a virtual object stably to a real tabletop, the device must know its own six-degree-of-freedom pose in space in real time. The industry commonly calls this capability spatial localization or inside-out tracking, and technically it is nothing other than visual-inertial SLAM.

- Why vision and not LiDAR: an AR device is extremely sensitive to weight, power consumption, and cost, and a camera plus an IMU is the only workable combination (the advantages of vision in Section 9.5).
- Special requirements: the latency must be extremely low (otherwise the virtual object drifts and causes discomfort), and tracking must be maintained even as the user turns their head quickly—a severe test for visual SLAM, and the reason an IMU is indispensable (Section 7.6).
- The evolution of the VR headset: early VR required external base stations for localization, while a modern headset uses cameras on the body of the device to perform visual SLAM, so that the user can walk about freely—an example of SLAM changing the form of a product directly.

Robotic Applications

- Indoor service robots: what visual SLAM supplies is not only geometry but also semantics (Section 9.7)—the robot can recognize that this is a door and that is an elevator, which a LiDAR cannot give (the theme of Chapter 32).
- Drones: payload and electrical power are extremely constrained, and a camera plus an IMU is the best choice (the theme of Chapter 26).
- Low-speed autonomous driving and campus vehicles: the application of visual SLAM in low-speed autonomous driving is an active direction of research, and the feature-based method is widely adopted for its robustness to camera motion, to changes of illumination, and to moving objects.
- Place recognition and relocalization: visual features are naturally suited to judging whether I have been here before. The bag-of-words technique is among the most widespread approaches, using local feature descriptors as visual words; and DBoW2 brings in the efficient FAST detector together with the BRIEF descriptor, in the form of a binary bag of words, cutting the time needed for feature extraction sharply against the traditional practice of using SURF and SIFT, so that the system is both more efficient and more robust. This is precisely the technical basis of loop closure detection in ORB-SLAM (Chapter 22).

The Trend: the Arrival of Deep Learning

Traditional hand-crafted feature methods have one problem: they cannot adapt effectively to every complicated and unforeseeable situation; many researchers have therefore adopted convolutional neural networks to extract deep features from images, applying them at every stage—visual odometry, pose estimation, loop closure detection, and the rest.

This is one of the most active directions in visual SLAM at present: replacing hand-

crafted features with learned ones, performing place recognition with a neural network, and even estimating pose end to end. In working systems, however, the traditional geometric methods remain the mainstream for their interpretability and their stability; learned methods mostly serve as an auxiliary or are used for particular difficult scenes. Chapter 32 returns to this theme and discusses how AI and SLAM are brought together.

At this point students have established the overall framework of visual SLAM. In the next chapter we go deeper into the mathematical core of the front end—epipolar geometry: what geometric relation is in fact hidden between two images? What are the essential matrix and the fundamental matrix? How is the PnP problem solved? How does triangulation recover three-dimensional points? These are the bridge that turns two-dimensional pixel correspondences into three-dimensional camera motion.

20.8 After Features: Three Turns in the Visual Front End, 2024–2026

The SIFT and ORB introduced in this chapter are hand-crafted features, and Case 20-3 has already heralded the change deep learning brings. That change has accelerated markedly in the past two years, and it runs deeper than merely changing to another kind of feature. This section sets out three turns.

One: Learned Features Can Finally Run Fast Enough

Learned local features (SuperPoint, for instance) long ago surpassed ORB in accuracy, but for years they were stuck on the same problem: too slow, and in need of a GPU. That is exactly the barrier set by the 33-millisecond red line of Figure 20-2(c). XFeat, of 2024, was designed against precisely this point—it is built to run in real time on the CPU of an inexpensive laptop, with no specialized hardware acceleration, and it is up to

five times faster than existing lightweight deep features while matching the accuracy of far larger models such as SuperPoint.

There has been a similar evolution on the matching side. Methods of the LightGlue kind let a deep matcher adaptively stop computing early according to the difficulty of the images, so that an easy image pair runs through fewer layers. Taken together the meaning is this: the three lines of defense of Figure 20-3 (the ratio test, the cross check, and RANSAC) are being replaced by a network that has learned to judge whether a correspondence can be trusted. It must be stressed, however, that the geometric consistency verification RANSAC stands for has not disappeared; it has merely retreated from being the principal line of defense to being the final confirmation.

Two: From Between Two Images to Putting Out Three Dimensions Directly

The second turn is more fundamental. The front end of traditional visual SLAM is a long chain: extract features → match → reject → solve the epipolar geometry → triangulate. The line of work of DUSt3R and MASt3R proposes instead that the network simply put out the three-dimensional coordinates of every pixel from two images, skipping that whole string of intermediate steps at once.

MASt3R-SLAM (CVPR 2025), built upon it, is a complete monocular dense SLAM system: it replaces the KD tree or brute-force search with dense point map matching, and it does not even need to know the camera intrinsics beforehand. On the TUM RGB-D dataset its mean absolute trajectory error is 0.030 meters when calibrated and 0.060 meters when uncalibrated; on an ordinary desktop machine with a high-end graphics card it reaches about 15 fps.

But there is a detail here that students must see clearly, and it answers directly to Figure 20-4: MASt3R-SLAM has not solved the scale ambiguity. That paper states plainly that scale is the principal source of inconsistency among its predictions, which is why it optimizes pose on Sim(3), and its monocular evaluation results are reported only after a scale alignment has been performed. In other words, even with the entire front end replaced by a foundation model, it remains true that a monocular camera cannot see absolute scale—because that is a property of the geometry and not a defect of the algorithm.

Three: Map Representation, from a Sparse Point Cloud to a Renderable Scene

The third turn belongs to the same current as the three-dimensional Gaussian splatting mentioned in Section 17.8, only it is still more natural in the visual domain. What traditional feature-based SLAM builds is a sparse map of feature points—adequate for localization, but no more than a cloud of scattered dots to look at, useless for rendering and unhelpful for any later semantic understanding. The dense visual SLAM of the past two years represents the scene directly in a renderable form, making localization and the reconstruction of a lifelike scene two sides of one thing.

This matters especially for the AR / VR applications discussed in Section 20.7: what AR needs has never been only the camera pose but also a knowledge of which real surfaces will occlude the virtual object. A sparse point cloud cannot give that; a dense renderable representation has it inherently. The price is equally plain, however—both memory and computation are far greater than for a sparse map, which is why robot navigation still works mainly with sparse or semi-dense maps.

The Thread Common to the Three Turns

Front-end components (XFeat / LightGlue): learned features have at last crossed the threshold of real-time operation and are beginning to replace ORB.

Front-end structure (MASt3R-SLAM): the whole chain of feature—match—geometry is crossed at a stride by an end-to-end network.

Map representation (dense and renderable): the map turns from sparse points for localizing into a scene that can be seen, can occlude, and can be understood.

What has not changed: the scale ambiguity is still there (Figure 20-4), geometric consistency verification is still needed, and there is still no information to extract from a white wall.

The common thread: what is being replaced is how it is computed, not what is computed. The four basic questions of this chapter—where are the features, which correspondences can be trusted, where does depth come from, and who supplies the scale—remain the standard by which any new system is judged.

Carrying Forward

This chapter has established the overall framework of visual SLAM together with the fundamentals of features. Figure 20-2 showed where features are to be found and why ORB is the choice; Figure 20-3 showed that matching must go wrong and how three lines of defense rescue correctness from 50 percent back to 100 percent; and Figure 20-4 showed the inherent limit of the monocular camera. These three things are also the background to every design decision of the ORB-SLAM family in Chapters 22 and 23.

But we have not yet answered the most central question: given a set of trustworthy two-dimensional correspondences, how exactly does one compute how far the camera has moved, and where those feature points lie in three-dimensional space? The next chapter enters epipolar geometry—the essential matrix, the fundamental matrix, the homography, and triangulation. This is the mathematical heart of the whole of visual SLAM, and the true face of that geometric constraint RANSAC verifies in Figure 20-3.

Formulas and Procedures

One: the Hamming Distance, for Comparing Binary Descriptors

The similarity of two binary descriptors d_i and d_j is measured by the Hamming distance—the number of bits at which they differ:

$$Hamming(d_i, d_j) = popcount(d_i \ XOR \ d_j)$$

Count the ones after an XOR (popcount). For a 256-bit ORB descriptor this takes only a few bitwise instructions, far faster than computing the Euclidean distance of a floating-point descriptor. This is one of the keys to ORB's real-time performance. The smaller the distance, the greater the similarity.

Two: the Ratio Test

For a given feature, let d_1 be its distance to the most similar candidate and d_2 its distance to the second most similar; the match is accepted on the condition that

$$d_1 \ / \ d_2 < \tau \quad (\tau \text{ is commonly taken between 0.7 and 0.8})$$

Intuitively: if the gap between the best and the second best is not pronounced enough (a ratio near 1), this feature has several similar candidates in the image and is hard to distinguish, so it should be discarded. This simple criterion rejects a great many

false matches.

Three: the Photometric Error of the Direct Method

The direct method does not compare descriptors but minimizes the photometric error—the difference in brightness after projecting the pixels of the previous frame into the current frame under the camera motion T:

$$T^* = \arg\min \Sigma_i \, \| I_2(\pi(T \cdot P_i)) - I_1(u_i) \|^2$$

where I_1 and I_2 are the brightnesses of the two frames, u_i a pixel in the first frame, P_i the three-dimensional point corresponding to it, and $\pi(\cdot)$ the projection function of Chapter 9. Compared with the feature-based method: the feature-based method minimizes a reprojection error (a geometric distance, in pixels), while the direct method minimizes a photometric error (a difference in brightness). Both are least squares problems (Chapter 12); only the definition of the error differs.

Four: the Front-End Processing Pipeline of Visual SLAM

Figure 20-5 The Per-Frame Processing Flow of a Feature-Based Front End

① Read the image → undistort (Section 9.2)

↓

② Detect key points: find corners with FAST and apply non-maximum suppression

↓

③ Compute orientation and descriptor: oriented FAST + rotated BRIEF → a 256-bit descriptor

↓

④ Match against the previous frame (or the map): compare by Hamming distance

↓

⑤ Reject false matches: the ratio test → the cross check → geometric verification by RANSAC

↓

⑥ Solve for pose: estimate the camera motion from the verified matches (Chapter 21)

↓

⑦ Triangulate new three-dimensional points → hand them to the back-end optimization (Chapters 12 and 22)

Cases

Case 20-1 The Cost and the Reward of Speed: Why ORB Won the SLAM Battlefield

Judged on the quality of the features alone, SIFT is superior to ORB on nearly every measure—its scale invariance is more complete, its matching more accurate, its tolerance of changes in viewpoint greater. Why then do the great majority of real-time SLAM systems choose ORB?

The answer is the harsh demand of running in real time. A SLAM system running at 30 frames per second has a budget of about 33 milliseconds for each frame, and within that time it must also complete feature extraction, matching, pose solving, the map update, and everything else. ORB's developers report it to be 14 times faster than SURF, and SURF in turn 24 times faster than SIFT—which works out to ORB being roughly two orders of magnitude faster than SIFT. That gap is not a matter of being somewhat quicker; it is the dividing line between running in real time and not.

More important still, the accuracy ORB sacrifices can be made good within a SLAM system. SLAM has back-end optimization and loop closure correction (Chapter 12), and even if a single match is slightly inferior, a great many redundant observations and a global optimization can press the error down. In other words: design at the level of the system compensates for a shortfall at the level of the component.

This case gives students an important engineering outlook: in choosing a technical component one must look not at that component's isolated figures of merit but at its contribution and its cost within the whole system. This is of a piece with sensor selection is an overall judgment in Chapter 8 and the design orientation is decided by the needs of the application in Chapter 15.

Case 20-2 The Trap of Repetitive Texture: Tiled Floors and Railings

Here is a failure experiment easily reproduced in a university classroom: have a robot run visual SLAM while moving over a floor paved with identical tiles, or along a row of identical railings. The result is often that the pose estimate suddenly jumps to something wrong and the map shows a severe misalignment.

The cause is systematic false matching produced by repetitive texture. The corner of every tile looks exactly the same and the descriptors are nearly identical. In comparing them the algorithm may well match the corner of this tile to the corner of the one next door—and it looks like a perfect match, with an extremely small descriptor distance.

This is precisely the point of the ratio test: when the distances of the best and second-best candidates are very close together (as they must be in a tiled scene), the ratio test judges the match indistinguishable and discards it, preferring fewer matches to wrong ones. Set the ratio threshold too loosely and scenes of this kind will go wrong.

Set beside the long corridor of Section 13.6, the structural degeneracy of Section 17.3, and the curse of the white wall of Chapter 9, students should see a pattern running through the whole book: repetitiveness of the environment and a poverty of features are the common enemies of SLAM, whether the sensor is a laser or a camera. Understand this and you will be able to foresee, at the planning stage of a project, which sites will give trouble.

Case 20-3 From Hand-Crafted Features to Deep Learning: a Change Now Under Way

SIFT, SURF, and ORB, introduced in this chapter, are all hand-crafted features—

human researchers designed rules for detection and description out of their own understanding of images. These methods dominated computer vision and SLAM for more than twenty years.

But they share one weakness: they cannot adapt effectively to every complicated and unforeseeable situation; many researchers have therefore turned to convolutional neural networks to extract deep features from images, applying them at the stages of visual odometry, pose estimation, and loop closure detection. Learned features can adapt to situations that hand-crafted rules describe only with difficulty—changes of season, the difference between day and night, the effects of weather.

Students should note, however, that this change is not yet complete, and that it will not necessarily supplant the old methods entirely. The traditional geometric methods have several advantages that learned methods can hardly match at present: they are interpretable (when they go wrong one knows why), they need no training data, their behavior is predictable in scenes they have not seen before, and their demands on computational resources are definite. Mainstream working systems still take geometric methods as their skeleton, with learned methods as a reinforcement at particular points.

The advice for readers of this book is to learn the traditional methods thoroughly first. Understanding why ORB is designed as it is, why RANSAC works, and what bundle adjustment is solving—these principles are a framework of thought that will not go out of date. With that foundation, when you come to look at the deep learning methods you will know what they are replacing and what they are keeping. Chapter 32 returns to this theme.

Hands-On Activities

Activity 20-1 Feature Extraction, Matching, and the Rejection of False Matches

Purpose: to implement the complete front-end pipeline of visual SLAM with your own hands, to compare the performance of different feature algorithms, and to witness the power of rejecting false matches. This activity uses Python with OpenCV and needs only an ordinary camera or images already to hand, and it is a direct warm-up for Chapters 21 through 23.

Part One: a Comparison of Feature Algorithms (everyone)

1. Prepare two photographs of the same scene from slightly different viewpoints (photograph the classroom with a phone, stepping a short distance forward or back between shots).
2. Detect key points with SIFT, ORB, and FAST in turn, draw the results on the images and compare them side by side: how many points does each detect? Where are they distributed?
3. A timing comparison: record the extraction time of the three methods on the same image and verify the differences in speed mentioned in this chapter (ORB far faster than SIFT).
4. Repeat the experiment on a photograph of a large expanse of white wall and observe that all three methods detect almost no features at all—verifying once again the curse of the white wall of Chapter 9.
5. A rotation test: rotate the second image by 45 degrees and compare the matching success rate of ORB against the original BRIEF, verifying the value of rotation invariance.

Part Two: Matching and the Rejection of False Matches (everyone)

6. Use ORB to perform brute-force matching between the two images (by Hamming distance), draw all the match lines, and observe by eye how many are plainly wrong.
7. Apply the ratio test (threshold 0.75), redraw the matching result, and count how many matches were rejected and how good the remainder is.
8. Add the cross check and observe the degree of improvement once again.
9. Use RANSAC to estimate the geometric relation between the two images (a homography will do to begin with), display inliers and outliers in different colors, and observe which matches RANSAC rejects.
10. A ratio threshold experiment: set the threshold to 0.6, 0.75, and 0.9, compare the number of matches retained against the correctness, and find the value best suited to your own scene.

Part Three: Repetitive Texture and Tracking (university level)

11. A repetitive texture experiment: photograph a tiled floor, a railing, or a latticed window, run the matching, and observe the false matches, reproducing Case 20-2. Compare how important the ratio test is in scenes of this kind.
12. Optical flow tracking: record a video of slow motion, track feature points with OpenCV's Lucas-Kanade optical flow, draw out the tracks, and observe which points are tracked stably and which are lost.
13. Tracking against matching: on the same video, run both extract and match afresh every frame and optical flow tracking, and compare the time taken and the stability of the two against the discussion of Section 20.4.

Discussion Questions

1. By what factor do the extraction times of ORB and SIFT differ? Does this agree with the figures quoted in this chapter? If your SLAM system must run at 30 fps, which of them is workable?
2. What percentage of the matches did the ratio test reject? Is the quality of the remainder appreciably better? What happens if the threshold is relaxed to 0.9?
3. In a scene of repetitive texture, what pattern do the false matches display (are they random, or systematically biased in some direction)? Why is a systematic false match more dangerous than a random one?
4. Under what conditions does optical flow tracking lose a point? What does this suggest about the relation between the speed of camera motion and the frame rate?
5. (University level) Following Table 20-3, if you had to choose a visual configuration for an indoor tour-guide robot at a university, would you choose monocular plus IMU, stereo, or RGB-D? Give your reasons.

Extended Challenge (university level)

First: implement the simplest possible visual odometry—read the frames of a video in sequence, match consecutive frames with ORB, reject false matches with RANSAC, estimate the relative motion and accumulate it, and draw out the trajectory. Observe how drift accumulates with time (precisely the problem Chapters 21 and 22 set out to solve). Second: implement a simple place recognition—extract ORB descriptors from all the frames of a video, build a simple bag-of-words representation, and test whether it can correctly find images of the same place taken at different moments. This is the embryo of the loop closure detection of Chapter 22.

Further Reading

1. E. Rublee et al., "ORB: An Efficient Alternative to SIFT or SURF," ICCV, 2011. (The original ORB paper, the principal source for Section 20.3 of this chapter.)
2. D. G. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," IJCV, 2004. (The classic SIFT paper, including the proposal of the ratio test.)
3. E. Rosten and T. Drummond, "Machine Learning for High-Speed Corner Detection," ECCV, 2006. (The original paper on the FAST detector.)
4. M. Calonder et al., "BRIEF: Binary Robust Independent Elementary Features," ECCV, 2010. (The pioneering work on binary descriptors.)
5. D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," IEEE T-RO, 2012. (DBoW2, the basis of loop closure detection in ORB-SLAM.)
6. Gao Xiang and Zhang Tao, Fourteen Lectures on Visual SLAM, Lecture 7 (Visual odometry: the feature point method) and Lecture 8 (The direct method). (The clearest teaching material on feature-based and direct methods in the Chinese-speaking world, with complete source code.)
7. The official OpenCV tutorial on Feature Detection and Description (docs.opencv.org). (The principal tool for the implementation in this chapter, with complete examples.)

Summary

1. The standard pipeline of visual SLAM is preprocessing and undistortion → visual odometry (the front end) → back-end optimization → loop closure detection → mapping; its architecture agrees closely with the LiDAR SLAM of Chapter 18, the difference being that a visual feature has only two-dimensional pixel coordinates and its depth must be obtained separately.

2. The feature-based method first extracts key points and descriptors and then solves geometrically; it is robust to changes of illumination and to moving objects but fails where texture is weak. The direct method minimizes a photometric error over pixel brightness; it can use faint gradients and produce a dense map, but it rests on the fragile assumption of brightness constancy.
3. FAST only detects, BRIEF only describes, and SIFT / SURF / ORB do both; ORB combines oriented FAST with rotated BRIEF and achieves extremely high efficiency with a 256-bit binary descriptor and the Hamming distance—about 14 times faster than SURF, and SURF in turn about 24 times faster than SIFT—and has thereby become the mainstream choice for SLAM.
4. Matching compares descriptors and suits a large difference of viewpoint and loop closure detection; tracking searches near the original position, suits consecutive images, and is faster. False matches are inevitable, and the three strategies for rejecting them are the ratio test, the cross check, and geometric verification by RANSAC.
5. Monocular SLAM has an inherent scale ambiguity: scale the scene and the motion by k together and the image is exactly the same. The three remedies are to introduce an IMU, to use a stereo or RGB-D camera, or to use a reference object of known size.
6. A stereo camera measures depth from the disparity of a known baseline, has absolute scale, and works indoors and out, but it fears a lack of texture and its error grows with the square of the distance; an RGB-D camera obtains dense depth by active projection and still works on a white wall, but it fears strong light and glass and its range is limited.

7. Visual SLAM is, technically, what inside-out tracking for AR / VR really is, and it is also widely used in service robots, drones, and low-speed autonomous driving; the bag-of-words model (DBoW2's binary bag of words, for instance) is the key technique for place recognition and loop closure detection, while deep-learned features are steadily entering this field.

This chapter has established the overall framework of visual SLAM together with the fundamentals of features. But we have not yet answered the most central question: given feature correspondences between two images, how exactly does one compute how far the camera has moved? The next chapter, Epipolar Geometry and the Estimation of Camera Motion, supplies that piece of the puzzle—the essential matrix and the fundamental matrix describe the geometric constraint between two views, the PnP problem handles the correspondence of three-dimensional points to a two-dimensional image, triangulation recovers three-dimensional coordinates, and RANSAC makes all of it workable in a reality that contains false matches. It is the most mathematical and the most elegant chapter of visual SLAM.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 21

Epipolar Geometry and the Estimation of Camera Motion

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 20 taught the robot to find the same thing in two images. This chapter answers the next question: having found it, how does one compute how far the camera has moved?

This is the chapter of highest mathematical density in the book, but its reward is also the most direct—the three experimental figures of this chapter will prove one thing: epipolar geometry is not an approximation and not a rule of thumb but a set of exact relations whose error is zero in the absence of noise. Figure 21-2 will demonstrate that, even with 0.3 pixels of noise on every corresponding point, the rotation recovered by the eight-point algorithm is in error by only 0.16 degrees and the direction of translation by 0.70; and that among the four mathematical solutions the cheirality check picks out the one correct solution by 235 votes to 0.

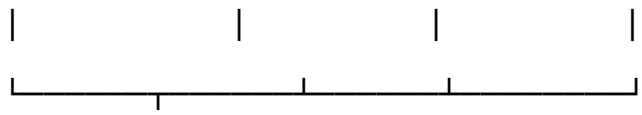
Equally important, though, is its fragility. Figure 21-3 will measure how noise is magnified into pose error, and why depth cannot be trusted at all when the baseline is short; and Figure 21-4 will show that a mere one match in ten being wrong brings this elegant mathematics down entirely—which is exactly the reason RANSAC exists, and the motive behind every robustness measure in ORB-SLAM2 in Chapter 22.

Figure 21-0 The Link Map of Chapter 21

The Foundations Already Laid

Chapter 9 The camera model Chapter 10 Coordinate transformation Chapter 12

Optimization Chapter 20 Feature matching



▼

【Chapter 21 Computing three-dimensional motion from two-dimensional correspondences】

The epipolar constraint ─ the essential matrix ─ PnP ─ triangulation ─ RANSAC

▼

Chapter 22 ORB-SLAM2 Chapter 23 ORB-SLAM3 (visual-inertial)

▼

Chapter 24 The vision-navigated robot Part VIII ROS implementation

Learning Objectives

In the last chapter we learned how to find corresponding feature points between two images, but we have not yet answered the most central question: given these correspondences, how exactly does one compute how far the camera has moved? This chapter supplies that piece of the puzzle. It is the most mathematical and the most elegant chapter of visual SLAM—once you understand epipolar geometry you will find that two seemingly ordinary photographs conceal a three-dimensional constraint of remarkable rigor. After studying this chapter, students should be able to:

1. Describe the overall line of thought in estimating camera motion from two images, and the conditions under which it applies.
2. Describe the central elements of epipolar geometry: the baseline, the epipole, the epipolar line, and the epipolar constraint.

3. Distinguish the essential matrix from the fundamental matrix and explain how camera motion is recovered from them.
4. Describe the definition of the PnP problem, the methods of solving it, and its role in SLAM.
5. Describe the principle of triangulation and the factors that affect its accuracy.
6. Describe how RANSAC operates and understand why it can stand up to a great many false matches.
7. Connect the knowledge of an image processing course to the geometric solutions of visual SLAM.

Core Concepts

Chinese Term	English Term	Explanation
對極幾何	Epipolar Geometry	The theory describing the intrinsic geometric relation between two views of the same scene.
基線	Baseline	The line joining the optical centers of two cameras.
對極點	Epipole	The intersection of the baseline with each image plane.
對極線	Epipolar Line	The line in one image corresponding to a point in the other.
對極約束	Epipolar Constraint	The geometric restriction that a corresponding point must fall on the epipolar line.
本質矩陣	Essential Matrix, E	The 3×3 matrix describing the relation between two views when the intrinsics are known.

Chinese Term	English Term	Explanation
基礎矩陣	Fundamental Matrix, F	The 3×3 matrix describing the relation between two views when the intrinsics are unknown.
單應矩陣	Homography, H	The matrix describing the relation between two views of a planar scene or under pure rotation.
PnP 問題	Perspective-n-Point	Solving for camera pose from known three-dimensional points and their image projections.
三角測量	Triangulation	Recovering three-dimensional coordinates from corresponding points in two views.
視差角	Parallax Angle	The angle a point subtends at the two optical centers, which affects the accuracy of triangulation.
隨機取樣一致	RANSAC	A robust estimation method that rejects outliers by random sampling and voting.
內點與外點	Inlier / Outlier	Data points that do and do not conform to the estimated model.

21.1 How Two Images Yield an Estimate of Camera Motion

Let us first form the overall picture of this chapter. Suppose a camera takes a photograph at position A, moves to position B, and takes another. Chapter 20 has already found for us the corresponding feature points in the two images. The question now is this: from these correspondences, can the rotation and translation of the camera from A to B be computed?

The Intuition: the Movement of Corresponding Points Gives the Motion Away

Intuitively the answer is yes. Imagine standing at the front of a classroom to take a photograph, then stepping two paces to the right and taking another: everything in the frame has moved to the left. If instead you step forward, things in the frame spread outward radially (near things moving a great deal, far things only a little). Different motions produce different patterns of displacement in the image—and that pattern is the clue we are to read.

Three Situations for Solving

In visual SLAM, three different methods of solution are used according to what information is in hand. This classification is the skeleton of the chapter and students should fix it firmly in mind:

Situation	What is known	Method of solution	Section of this chapter
2D–2D	Corresponding pixels in two images	Epipolar geometry (the essential / fundamental	20.2–20.3

Situation	What is known	Method of solution	Section of this chapter
		matrix)	
3D–2D	Known three-dimensional points + their image projections	The PnP problem	20.4
3D–3D	Two corresponding sets of three-dimensional points	ICP (Chapter 17)	Chapter 17

Table 21-1 Three Situations in Estimating Camera Motion

- 2D–2D: when the system has just started there are no three-dimensional map points at all, and initialization can rest only on the pixel correspondences of two images. This is the opening move of monocular SLAM.
- 3D–2D: once a map has been built, every subsequent frame can solve for pose using three-dimensional points already known in the map together with their two-dimensional projections seen in the current image. This is the method most used while SLAM is running, being both efficient and stable.
- 3D–3D: when both views carry depth information (stereo, RGB-D), two sets of three-dimensional points can be registered directly—which is precisely the ICP of Chapter 17, which students have already studied.

This chapter focuses on the first two. Understand this classification and students will be able to see, in the source code of ORB-SLAM (Chapter 22), when each kind of solver is called.

21.2 Epipolar Geometry

Epipolar geometry is the theoretical core of this chapter. It answers a question that seems simple but runs very deep: where in the second image can a given point of the first image appear?

The Search Space Falls from Two Dimensions to One

Intuitively the answer seems to be anywhere—after all, we do not know how far away the point is. But epipolar geometry tells us something remarkable: it can appear only on one particular straight line.

Why? Consider a pixel p_1 in the first image. The three-dimensional point P it corresponds to must lie on the ray that passes from the optical center of the first camera through p_1—only we do not know where on that ray (precisely the loss of depth of Chapter 9). Now project that entire ray into the second image and a straight line results. Wherever P actually lies on the ray, its projection in the second image must fall on this line.

That line is the epipolar line, and that constraint is the epipolar constraint. Its practical value is enormous: where the corresponding point had to be sought over the whole image (two dimensions), it need now be sought only along a line (one dimension). This both speeds up the search and greatly reduces the chance of a false match.

The Four Elements of Epipolar Geometry

- The baseline: the line joining the two optical centers O_1 and O_2. It is the backbone of the whole geometry.
- The epipolar plane: the plane determined by the two optical centers together with the point P in space. A different P gives a different epipolar plane, but all of

them contain the baseline—all epipolar planes form a pencil about the baseline.

- The epipole: the intersection of the baseline with each image plane, written e_1 and e_2. Geometrically it is the position at which the other camera's optical center projects into this image. Interestingly, every epipolar line in a given image passes through that image's epipole.
- The epipolar line: the intersection of the epipolar plane with the image plane. The epipolar line of p_1 in the second image is written l_2.

Figure 21-1 The Structure of Epipolar Geometry

A point P in space

├──── projects into image 1 → the pixel p_1

└──── projects into image 2 → the pixel p_2

O_1 ──────baseline──────→ O_2 (the two optical centers)

| |

└→ epipole e_1 └→ epipole e_2

The epipolar plane = the plane determined by O_1, O_2, and P

The epipolar line l_2 = the intersection of the epipolar plane with image 2

【The central constraint】p_2 must fall on the epipolar line l_2

→ the search space falls from two dimensions (the whole image) to one (a line)

A Verification from Everyday Life

Students can verify this phenomenon for themselves: close the left eye and look at a pen in front of you with the right, noting its position in the field of view; then close the right eye and look with the left, and the pen jumps to another position. Move the pen nearer and farther and repeat, and you will find that the track along which it jumps in the field of view always lies on one fixed straight line—that is the epipolar line. A pair of human eyes is a natural stereo system.

21.3 The Essential Matrix and the Fundamental Matrix

Described only in words, the epipolar constraint cannot be computed with. This section writes it in matrix form—and that is the essential matrix and the fundamental matrix.

The Matrix Form of the Epipolar Constraint

The mathematical form of the epipolar constraint is extremely concise and elegant. Let p_1 and p_2 be a pair of corresponding points in the two images (in homogeneous coordinates); then:

$$p_2^T \cdot F \cdot p_1 = 0 \qquad \textit{(the fundamental matrix form, in pixel coordinates)}$$

$$x_2^T \cdot E \cdot x_1 = 0 \qquad \textit{(the essential matrix form, in normalized coordinates)}$$

These two expressions are the most important formulas of the chapter. They say that the result of multiplying the corresponding points by the matrix must be zero—in other words, any correct pair of corresponding points must satisfy this constraint. Conversely, if we gather enough corresponding points, we can solve back for the matrix.

How the Two Differ

- The fundamental matrix F: acts on the raw pixel coordinates and does not require the camera intrinsics K (Chapter 9) to be known. It mixes together two kinds of information, the internal parameters of the cameras and the relative motion between them. It has 7 degrees of freedom.
- The essential matrix E: acts on normalized coordinates (pixel coordinates already transformed by the intrinsics K) and therefore contains only information about the relative motion, which makes it the purer of the two. It

has 5 degrees of freedom (three of rotation and three of translation, less one because the direction of the translation is known but not its magnitude).

The relation between them is

$$E = K_2^T \cdot F \cdot K_1$$

In practice the cameras of a SLAM system are calibrated (Section 9.2) and the intrinsics are known, so the essential matrix is generally used directly. The fundamental matrix is used where the intrinsics are unknown (handling photographs of uncertain origin from the internet, say).

Recovering Camera Motion from E

Once the essential matrix has been found, a singular value decomposition (SVD) can factor it into a rotation R and a translation t. But two important properties must be noted here:

- The translation is known in direction only, not in magnitude: in the definition of E, multiplying t by any nonzero constant leaves the constraint satisfied. This is exactly the mathematical embodiment of the monocular scale ambiguity of Section 20.5—epipolar geometry by itself cannot tell us how many meters the camera has moved, only in which direction.
- The decomposition has four solutions: mathematically, factoring E yields four possible combinations of (R, t). To pick the correct one from among them requires a physical criterion: after triangulating the corresponding points into three-dimensional coordinates, only the correct solution puts every point in front of both cameras (with positive depth). This is called the cheirality check.

A Special Case: the Homography

Epipolar geometry has one important exception. When every point in the scene lies on one and the same plane (the camera facing a wall, a tabletop, or the floor, for instance), or when the camera undergoes pure rotation with no translation, the epipolar constraint degenerates—solving for the essential matrix becomes unstable or even impossible.

In such a case one should use a homography H instead, which describes directly how the points of one image map into another:

$$p_2 = H \cdot p_1$$

One clever feature of ORB-SLAM's initialization is that it computes the fundamental matrix and the homography at the same time and judges automatically, from the fitting scores of the two, whether this scene is more planar or more three-dimensional, then chooses the appropriate model. This detail will be mentioned again in Chapter 22.

Computing the Whole Mathematical Chain Through Once

The three sections above have all been derivation. Derivations read elegantly, but a doubt inevitably forms: can this machinery really produce a number? This subsection takes a set of fully reproducible synthetic data and walks the whole chain through once: projection → the eight-point algorithm → decomposition → the cheirality check → triangulation.

The setup is this: the camera intrinsics are a focal length of 520 pixels, a principal point at the image center, and a resolution of 640×480; 240 three-dimensional points are scattered at random through the scene at depths between 4 and 12 meters; and the second camera is rotated about 9 degrees and translated 0.66 meters relative to the

first. These points are projected into the two images, Gaussian noise of 0.3 pixels is added, and then, pretending not to know the true motion, we recover it from these two-dimensional correspondences alone.

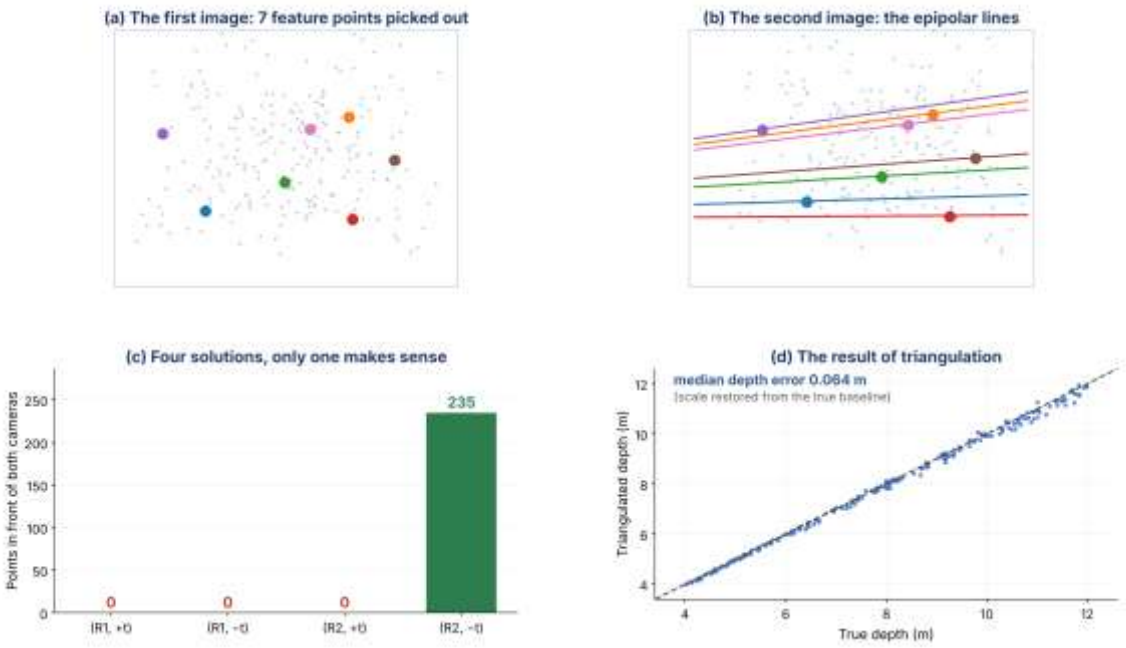


Figure 21-2 A Numerical Verification of the Epipolar Constraint: from Corresponding Points to Three-Dimensional Coordinates

Panel (a) is the first image, with 7 of the 240 points picked out and colored. Panel (b) is the second image, and each colored straight line is the epipolar line of the point of that color—computed from the fundamental matrix by $l_2 = F \cdot p_1$. Note that every colored dot falls exactly on the line of its own color: this is what the epipolar constraint looks like in an image. Where the corresponding point had to be sought over the whole 640×480 image, it need now be sought only along one line.

Panel (b) also reveals a detail: the seven epipolar lines are not parallel but converge slightly to one side. Extended, they meet at a single point—that is the epipole, the position at which the optical center of the first camera projects into the second image. In this example the camera mainly translates sideways with a small rotation, so the

epipole falls far outside the image and the lines look nearly parallel. Had the camera advanced straight ahead, the epipole would fall at the center of the image and all the epipolar lines would spread out from the center like rays of light—students can change the parameters and try it for themselves.

Panel (c) is the figure that best illustrates letting mathematics be mathematics and physics be physics. Take the singular value decomposition of the essential matrix and mathematics gives four combinations of (R, t), all of which satisfy the epipolar constraint, so that algebra alone cannot tell which is right. But take each of them, triangulate, and check whether those three-dimensional points lie in front of both cameras, and the result is this: the fourth combination has 235 points of positive depth and the other three have not one. 0 to 0 to 0 to 235—this is not a narrow win but an overwhelming one. The cheirality check works precisely because the three wrong solutions put the whole scene behind the camera, and that could not physically have been photographed at all.

Panel (d) is the last step: the selected (R, t) is used to triangulate the 235 points back into three dimensions and the result compared with the true depths. The points lie almost entirely on the diagonal, with a median depth error of 0.064 meters—about eight parts in a thousand in a scene running from 4 to 12 meters. It must be explained that the true depth on the horizontal axis can be matched at all only because we restored the scale with the known true baseline; in a genuine monocular system the whole figure would keep its shape but be scaled by an unknown factor, which is the scale ambiguity of Section 20.5.

With Zero Noise, the Error Really Is Zero

Set the noise of the experiment above to 0, and the rotation error recovered by the eight-point algorithm is 0.0000 degrees and the translation direction error 0.0000 degrees as well.

This is worth pausing over. It means that epipolar geometry is not an adequate approximation but a set of exact algebraic relations—the error comes entirely from measurement and not at all from the method. This is something quite different from the occupancy grid map of Chapter 13 or the particle filter of Chapter 14, which are approximations in their very nature.

So when students find in an implementation that the pose is estimated badly, they should not doubt the formulas but check three things: the accuracy of feature localization, the accuracy of the intrinsic calibration, and whether any false matches have crept in. The next two sections handle the first two and the third respectively.

21.4 The PnP Problem

Epipolar geometry handles the case of two images. But once SLAM is running we already have a three-dimensional map in hand—and then there is a better way to find pose.

The Definition of the Problem

The PnP (Perspective-n-Point) problem is defined thus: given the coordinates of n points in three-dimensional space together with the positions at which they project into the image in two dimensions, find the pose of the camera (the rotation R and the translation t).

Students should find this familiar—Section 4.6 used the same idea when it introduced AprilTag: the three-dimensional coordinates of the marker's four corners are known (the side of the square being known) and their pixel coordinates in the image can be detected, so the complete 6-DoF pose of the camera relative to the marker can be solved. That is a PnP problem with n = 4.

Why PnP Is More Useful Than Epipolar Geometry

- It has absolute scale: the coordinates of the three-dimensional points carry units (meters), so the translation solved for has a true magnitude as well—unlike epipolar geometry, which can give only a direction.
- No degeneracy to watch for: there is no problem of degeneracy in a planar scene.
- Few points required: a solution exists with as few as three points (P3P), far fewer than epipolar geometry requires.
- It is efficient: in SLAM each frame need only project and compare the points already known in the map, which makes it the mainstay method of the tracking stage.

Common Methods of Solution

- P3P: solves with three points and yields at most four solutions, requiring a fourth point to eliminate the rest. It is the basis of many methods.
- EPnP: an efficient linear solution able to handle any number of points, fast and of good accuracy, and the default choice of OpenCV and of many SLAM systems.

- The direct linear transform (DLT): converts the problem into a system of linear equations. Simple in concept but of poorer accuracy, it often serves to provide an initial value.
- Nonlinear optimization (BA): having obtained an initial value by one of the methods above, refine it by minimizing the reprojection error (Chapter 12). This is the final step, obligatory in practice.

The Reprojection Error: the Objective of PnP

An exact solution of PnP is in essence the minimization of the reprojection error—the central idea of bundle adjustment in Section 12.5:

1. Using the currently estimated pose, project each three-dimensional point into the image to obtain a predicted pixel position.
2. Compare it with the pixel position actually detected; the difference between the two is the reprojection error.
3. Adjust the pose so as to minimize the sum of the squared reprojection errors over all points.

Students can see that PnP is in fact a simplified bundle adjustment that optimizes only the camera pose and holds the three-dimensional points fixed, while a full BA optimizes poses and three-dimensional points together. The two are the same mathematics applied over different scopes—another appearance of the mathematical tools of Chapter 12.

21.5 Triangulation

The two preceding sections found camera motion from corresponding points. This section works the other way round: given the camera motion, find three-dimensional

coordinates from corresponding points. This is triangulation, and it is what allows monocular SLAM to build a map.

The Principle: the Intersection of Two Rays

The principle is extremely intuitive. Given the poses of the camera at two positions:

4. From the optical center of the first camera, draw a ray through the pixel p_1. The three-dimensional point P must lie on this ray.
5. From the optical center of the second camera, draw a ray through the pixel p_2. P must lie on this ray as well.
6. The intersection of the two rays is the three-dimensional position of P.

Perfect in theory, but there is a problem in practice: because the pixel coordinates carry measurement error, the two rays generally do not intersect exactly but pass each other by. The actual practice is therefore to find the point nearest to both rays, or to solve by minimizing the reprojection error.

The Key to Accuracy: the Parallax Angle

This is the most important idea of the section. The accuracy of triangulation depends on the angle between the two rays—called the parallax angle.

- A large parallax angle (the camera's displacement being large relative to the distance of the object): the two rays intersect at nearly a right angle, the position of the intersection is definite, and the depth estimate is accurate.
- A small parallax angle (the camera having hardly moved, or the object being very far away): the two rays are nearly parallel, and a minute error in the pixels lets the intersection slide a long way along the rays, so that the depth is extremely inaccurate.

This explains several important phenomena in practice:

7. Distant points cannot be measured accurately: for the same camera displacement the parallax angle of a distant object is far smaller, and so the depth of the sky or of distant hills can hardly be estimated at all. This is the same thing as the stereo depth error growing with the square of the distance in Section 9.6.
8. Pure rotation builds no map: if the camera only turns and does not move, the two optical centers coincide, the parallax angle is zero, and triangulation is impossible. This is a common cause of failed initialization in monocular SLAM—when students find in the implementation of Chapter 30 that initialization will not succeed, the reason is usually turning without moving.
9. A sufficient baseline is required: a SLAM system therefore waits until the camera has moved far enough (producing enough parallax) before triangulating new map points. This is one of the considerations behind the strategy for selecting key frames (Chapter 22).

The Tension, and How It Is Resolved

There is a subtle tension here: a large parallax angle requires the camera to move farther; but the farther it moves, the greater the difference in appearance between the two images, and the harder feature matching becomes (Chapter 20).

The resolution adopted in working systems is to track stably using neighboring frames with a short baseline, and to triangulate using key frames that are farther apart. This strategy of tracking on near frames and mapping on far frames is one reason ORB-SLAM distinguishes an ordinary frame from a key frame.

Measuring It: How Much Is Noise Worth, and How Much the Baseline

Section 21.5 says that depth is inaccurate when the parallax angle is small. That is true but not specific enough. How inaccurate? And how long must the baseline be? This subsection measures both questions with two sweep experiments.

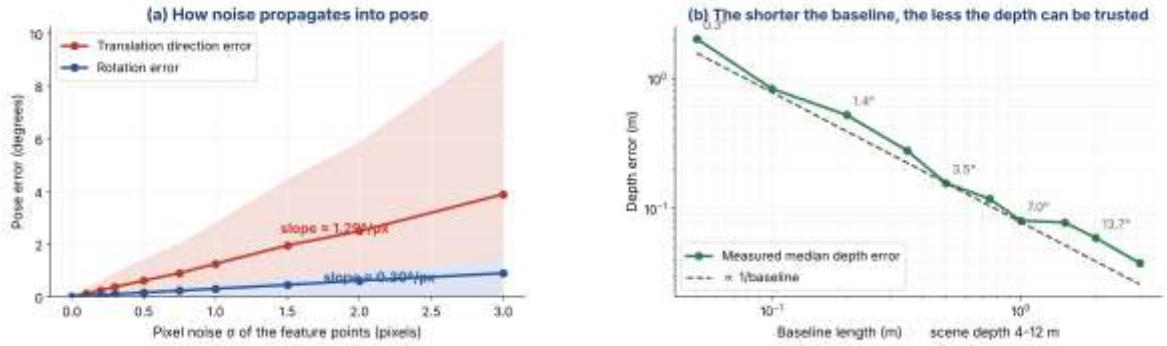


Figure 21-3 How Noise Propagates into Pose, and How the Baseline Decides Depth Accuracy

Panel (a) holds the scene and the motion fixed and varies only the pixel noise σ of the feature points, running 40 trials at each setting and taking the median, with the pale band showing the 90th percentile. Both lines are very nearly straight—pose error is proportional to pixel noise, with no threshold at which it suddenly collapses. That is good news: improve the accuracy of feature localization by a factor of two and the accuracy of the pose improves by a factor of two.

What is really worth remembering is the difference in the slopes of the two lines. The rotation error is about 0.30 degrees per pixel and the translation direction error about 1.29 degrees per pixel—the direction of translation is four times as sensitive to noise as the rotation. This asymmetry matters greatly in practice: the trajectory of visual odometry drifts off not mainly because the turns are estimated wrongly but because the direction of advance at every step is a little askew. It also explains why the visual-inertial SLAM of Chapter 23 brings in the IMU—an accelerometer supplies an

independent constraint on the direction of translation, exactly where vision is weakest.

Panel (b) holds the noise at 0.5 pixels, varies the length of the baseline, and measures the depth error of triangulation. Both axes are logarithmic, and the measured points lie almost on the dashed line of inverse proportionality to the baseline: at a baseline of 5 centimeters the depth error is 2.02 meters (of no value whatever for reference in a scene running from 4 to 12 meters), at 0.5 meters it falls to 0.16 meters, and at 3 meters to 0.038 meters. Marked beside each point is the corresponding parallax angle, running from 0.3 degrees all the way to 19 degrees.

Read backwards the figure is more useful still: if a student's project requires a depth error within 5 centimeters at a scene depth of about 10 meters, then the baseline must be at least 2.5 meters—which is to say that the camera must move a good two meters and more before it is fit to triangulate new map points. This is the concrete number behind the sentence a sufficient baseline is required in Section 21.5, and the arithmetic behind the key frame selection strategy of Chapter 22.

Why Practice Uses the Parallax Angle Rather Than the Baseline

The horizontal axis of Figure 21-3(b) is the length of the baseline, but what really decides the accuracy is the parallax angle—the ratio of the baseline to the depth of the scene. The same baseline of 0.5 meters gives a parallax of 5.7 degrees for an object 5 meters away and only 0.57 degrees for one 50 meters away.

So when a SLAM system judges whether it can triangulate yet, what it looks at is the parallax angle and not the distance moved. A common threshold is 1 to 2 degrees: below that, a new point is not created and the camera must travel

farther first.

This also explains why an open outdoor scene is especially unkind to monocular SLAM—distant buildings and the horizon never reach sufficient parallax, and all that can be used for mapping is the nearby road surface and the objects beside it.

21.6 Rejecting False Matches with RANSAC

The mathematics of the preceding sections is elegant, but it all shares one premise: that the correspondences fed in are correct. And Section 20.4 told us that false matches are bound to exist. This section introduces the key weapon against them.

Why a False Match Is So Deadly

Ordinary least squares (Chapter 12) minimizes the sum of the squared errors of all the data points. The trouble is that squaring magnifies large errors. One severe false match may produce an error of tens or even hundreds of pixels, which after squaring dominates the whole objective function and drags the estimate bodily off course.

By way of analogy: all ninety-nine students in a class are between 160 and 180 centimeters tall, but one entry in the data has been mistyped as 1800 centimeters. Estimate the height of the class by the mean and the result is severely distorted by that one absurd entry. Least squares facing a false match is in exactly this position.

The Central Idea of RANSAC

RANSAC (RANdom SAmple Consensus) adopts an entirely different strategy: rather than letting all the data take part, pick a few at random and see how many agree.

10. Random sampling: from all the correspondences, pick at random exactly the smallest number needed to solve the model (5 or 8 points for the essential matrix, 3 or 4 for PnP, say).
11. Solve the model: compute a candidate camera motion (or matrix) from those few points.
12. Count the votes: test all the correspondences against this candidate model and compute the error of each point. Those whose error is below a threshold are called inliers, and their number is recorded.
13. Iterate: return to step 1 and repeat tens to hundreds of times.
14. Choose the best: keep the model with the most inliers and solve once more, exactly and by least squares, using all of its inliers.

Why This Works

The key insight is this: if the few points picked at random happen to be all correct matches, then the model computed is correct, and the other correct matches will all agree with it (becoming inliers), so the inlier count is high. Conversely, if a false match is picked, the model computed is wrong, almost no other point agrees with it, and the inlier count is low.

The greatest number of inliers thus becomes an effective criterion for judging whether a model is correct. RANSAC does not try to correct false matches but simply excludes them—and this spirit of majority rule lets it tolerate a very high proportion of outliers (sometimes more than half).

Practical Points

- The number of iterations: how many are needed for a high probability of drawing at least one sample that is entirely of inliers? This can be estimated from a probability formula (see Formulas and Procedures). The lower the inlier ratio and the more sample points required, the more iterations are needed.
- Setting the threshold: the error threshold decides how close counts as an inlier. Too strict and correct matches are excluded too; too loose and false matches get in. It is usually set according to the level of pixel noise, at 1 to 3 pixels, say.
- Its relation to the robust kernel: RANSAC is the hard approach of rejecting first and optimizing after; the robust kernel mentioned in Chapters 12 and 15 is the soft approach of reducing the weight of outliers. The two are often used together—RANSAC first to reject the obvious false matches, then a robust kernel in the optimization to handle the outliers that remain.
- Its generality: RANSAC is not confined to vision. The point cloud registration of Chapter 17, the plane fitting of Chapter 13, and indeed any problem of model fitting in the presence of outliers can use it. It is among the most practical methods in an engineer's toolbox.

Measuring It: How Much Does RANSAC Actually Rescue

Section 21.6 calls a false match deadly. This section measures that word.

The method of the experiment is this: among correct matches at 0.5 pixels of noise, deliberately replace the second-image coordinates of a proportion of the correspondences with random positions (simulating false matches), then compare two approaches—throwing all the matches straight into the eight-point algorithm, and screening for inliers with RANSAC first and then estimating. Each proportion is run 25 times and the median taken.

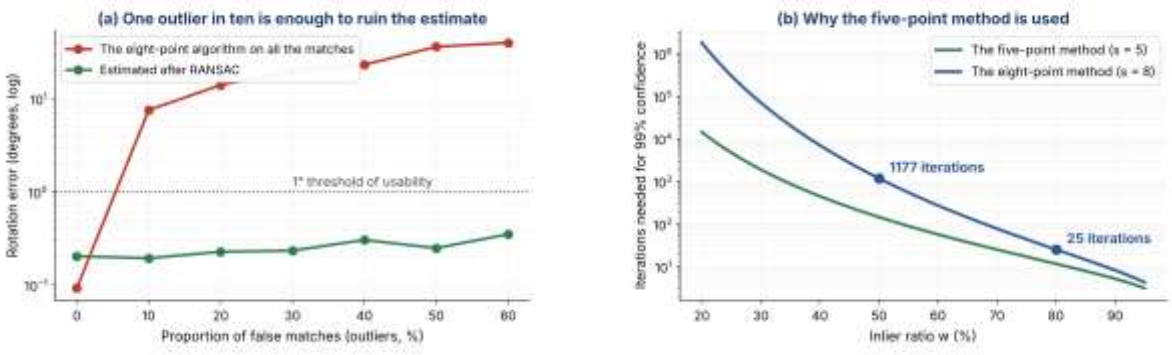


Figure 21-4 The Cost of False Matches, and Why the Number of Sample Points Is Decisive

The red line of panel (a) shows how serious the problem is. With no outliers, the rotation error of the direct eight-point algorithm is 0.09 degrees, which is quite accurate. But let in a mere 10 percent of false matches and the error explodes to 7.5 degrees—eightyfold. At 20 percent it is 14.0 degrees, and at 50 percent 36.1 degrees, which is to say wholly meaningless. Note that the vertical axis is logarithmic: this line does not degrade slowly but collapses outright from one outlier in ten.

The reason was given in Section 21.6: least squares sums the squares of the errors, and one false match may cause an error of hundreds of pixels, which after squaring dominates the whole objective function. In other words, this is not a slight loss of accuracy but the objective of the optimization itself being held hostage by a few points.

The green line is the result with RANSAC added. At 10 percent outliers the error is 0.19 degrees, scarcely worse than the direct estimate with no outliers at all; all the way to 40 percent outliers it stays within 0.31 degrees, far below the 1-degree threshold of usability; and even at 50 and 60 percent it is still 0.25 and 0.35 degrees. The theory nevertheless sets a limit on how far this can be pushed: when the inlier ratio falls to a half, the probability of drawing eight points that are all inliers is only $0.5^8 \approx 0.4$ percent, so the number of iterations must be raised accordingly—which is exactly what panel (b)

computes.

An honest addition: in this experiment the consensus set RANSAC found was a faithful one—at 50 and 60 percent outliers the proportion it judged to be inliers was 49 and 39 percent respectively, against true inlier ratios of 50 and 40 percent. That is the comfortable case, with the outliers scattered at random. Where false matches are systematic (a repeated texture, for instance, of the kind the tile region of Figure 20-3 produces) they may conform to one and the same wrong geometry, and RANSAC can then settle confidently on the wrong model. The remedy in practice is to increase the number of iterations, or to change to a variant more robust to outliers (MAGSAC or LO-RANSAC, say).

Panel (b) answers a question Section 21.6 raised but did not compute: how many iterations are enough? For an inlier ratio w and a minimum sample of s points, the number required for 99 percent confidence is $N = \log(1-0.99) \,/\, \log(1-w^s)$. At an inlier ratio of 50 percent the eight-point algorithm needs 1,177 iterations and the five-point algorithm only 145; at an inlier ratio of 80 percent the figures are 25 and 12. This is why the five-point algorithm for the essential matrix has in practice almost entirely displaced the eight-point algorithm—not because it is more accurate, but because it makes RANSAC an order of magnitude cheaper.

Why the Number of Sample Points Matters So Much

The key is the term w^s: s appears in the exponent. At an inlier ratio of 50 percent, the probability of drawing eight points that are all inliers is $0.5^8 = 0.39$ percent, while for five points it is $0.5^5 = 3.1$ percent—eight times greater.

This observation will appear again in Section 21.8: a new solution of recent

years that uses monocular depth estimation needs only three points, and at the same 50 percent inlier ratio it requires only 34 iterations—thirty-four times fewer than the eight-point method.

So when students design any algorithm of the RANSAC kind, the first question to ask is always: can I pin the model down with fewer points?

21.7 Connecting an Image Processing Course to Visual SLAM

The mathematical density of this chapter is fairly high, so this section steps back to connect the whole line of knowledge for students and to explain how one walks, step by step, from an image processing course already taken to visual SLAM.

How the Knowledge Follows On

Stage	The central question	Key techniques	Chapter of this book
Image processing	How to make an image clearer	Filtering, edge detection, morphology	Prerequisite knowledge
The camera model	How three dimensions become two	The pinhole model, intrinsics, distortion correction	Chapter 9
Feature engineering	How to find points that can be recognized repeatably	ORB, descriptors, matching	Chapter 20
Multiple view geometry	What geometric relation two images conceal	The epipolar constraint, E / F, PnP, triangulation	This chapter
Visual	How to estimate the	Solving frame by frame and	Chapters 20 and

Stage	The central question	Key techniques	Chapter of this book
odometry	camera trajectory continuously	accumulating	22
Visual SLAM	How to remove drift and build a consistent map	BA, loop closure, key frames	Chapters 22 and 23

Table 21-2 The Ladder of Knowledge from Image Processing to Visual SLAM

Three Key Turns of Thought

15. From processing an image to understanding geometry: image processing treats an image as a two-dimensional signal to be processed (filtering, enhancement); visual SLAM regards an image as a projection of the three-dimensional world and cares about the geometric structure behind it. This is the most fundamental change of perspective.
16. From a single image to the relation among several views: the epipolar geometry of this chapter is the core of this turn—a single image has lost depth, but the relation between two images suffices to recover motion and structure.
17. From exact assumptions to robust estimation: the algorithms of image processing mostly assume the input is clean; real SLAM must face false matches, noise, and moving objects, so RANSAC and robust optimization become indispensable tools. This also echoes the spirit of handling uncertainty by probability in Chapter 11.

Advice on Study

- Build geometric intuition first, then look at the formulas: the formulas of epipolar geometry are concise, but intuition matters more. Be sure to do the hands-on work of this chapter and draw the epipolar lines out to look at.
- Make good use of visualization: OpenCV can draw epipolar lines, inliers and outliers, and reprojection results. Once you see the picture, the abstract matrices acquire meaning.
- Compare with LiDAR SLAM: go back and compare the ICP of Chapter 17 (3D–3D) with this chapter's PnP (3D–2D) and epipolar geometry (2D–2D), and you will find them variants of one problem under different conditions of information.
- There is no need to derive every formula by hand: the details of the SVD factorization of the essential matrix, the derivation of EPnP, and the rest are already implemented in OpenCV. Understanding what it is solving, when it is used, and where it will fail matters more than memorizing a derivation. Those bent on research can go deeper into the classic textbook of Hartley and Zisserman.

At this point the mathematical foundation of visual SLAM is in place. In the next chapter we assemble these parts into a complete system—ORB-SLAM2. How do its three threads work together? How are key frames and map points managed? What is the covisibility graph? How are loop closures detected and corrected? A visual SLAM system that really runs is about to be laid out whole before the student's eyes.

21.8 After Epipolar Geometry: Three Turns in Two-View Estimation, 2024–2026

The mathematics of this chapter was already mature in the 1990s, and it remains to this

day the foundation of every visual SLAM system. But the way it is used has been changing in the past few years. This section sets out three turns worth knowing about.

One: Fewer Points, by Letting Monocular Depth Take Part in the Solution

The first turn makes use of a new fact: monocular depth estimation networks (mentioned in Chapter 9) are now quite usable, even though the depth they give carries an unknown scale and offset. A study of 2025 writes this imperfect depth straight into the solver: while estimating the relative pose it solves for the scale and the offset of the depth as unknowns at the same time.

The result is that the minimum number of correspondences required falls from five to three. By the iteration formula of Section 21.6, at an inlier ratio of 50 percent the three-point method needs only 34 iterations against 145 for the five-point method and 1,177 for the eight-point. What the paper reports in measurement is this: on a standard dataset the three-point solution has a median rotation error of 0.89 degrees and takes 22.4 milliseconds, while the five-point method paired with the best current dense matcher, though more accurate (0.56 degrees), needs 184.4 milliseconds. Accuracy, that is, is traded for an eightfold gain in speed—a decisive bargain on an embedded platform.

Two: Denser Matching, from a Few Hundred Features to the Whole Image

The second turn comes from the matching side. What Chapter 20 introduced was detecting feature points and then comparing descriptors, which usually yields a few hundred to a thousand or two correspondences per image. The dense matching methods of recent years instead find a correspondence for nearly every pixel in the image, producing tens of thousands to hundreds of thousands at a stroke, and are far

more robust than feature points under weak texture and large changes of viewpoint.

This creates a new problem for the mathematics of this chapter: RANSAC was designed for a few hundred correspondences, and faced with hundreds of thousands, merely testing every point once per iteration becomes unacceptably slow. Hence a line of research appeared in 2025 devoted to precisely this—first summarizing the dense matches into a small set of representative correspondences, then feeding those to the traditional two-view estimator. This is a good example: the new front end has not made the old geometry obsolete but has forced it to redesign its interface.

Three: Direct Regression, Skipping the Essential Matrix

The third turn is the most radical: do not solve for the essential matrix at all, but train a network that takes two images as input and outputs the relative pose between them directly.

A representative work of 2025 trained on about 8 million image pairs and reached a relative pose accuracy (AUC@20) of 75.56 on an indoor dataset, plainly above the 65.57 of comparable methods; on an indoor localization task its mean median errors were 0.04 meters and 1.02 degrees, against 0.36 meters and 18.38 degrees for the best comparable method before it. As for speed, it takes only 42 milliseconds per image pair (about 24 pairs a second), while a dense matching method needs 300 milliseconds and a reconstruction-based method 441.

Does this mean this chapter need no longer be studied? Quite the contrary, for three reasons. First, the training labels for networks of this kind are produced by the methods of this chapter. Second, their evaluation metrics (rotation error, translation direction error, AUC) rest entirely on this chapter's representation of pose. Third, and most

practical of all: when one of them estimates wrongly, you need this chapter's knowledge to judge where the fault lies—whether the parallax was insufficient, the scene degenerated into a plane, or there were simply too many outliers.

	Traditional epipolar geometry	Dense matching + geometry	Direct regression of pose
Input	A few hundred feature correspondences	Tens of thousands to hundreds of thousands of correspondences	Two images
Minimum number of points	5 to 8 (3 with depth)	The same	Not applicable
Interpretability	High; every step can be traced	Medium	Low
Handling degeneracy	Must be detected oneself (homography)	The same	Implicit in the training data
Typical time	A few milliseconds	A few hundred milliseconds	Tens of milliseconds
Present role	The foundation of every system	High-accuracy offline reconstruction	Fast initialization and relocalization

Table 21-3 A Comparison of Three Routes to Two-View Pose Estimation

The practical conclusion is that the three routes are dividing the labor rather than replacing one another. Traditional epipolar geometry, being fast, interpretable, and free of any need for training data, remains the default choice for a real-time SLAM front

end; dense matching is used for offline high-accuracy reconstruction; and direct regression suits occasions such as initialization and relocalization, where speed matters and a fair tolerance of error is acceptable. The ORB-SLAM family to be dissected in Chapters 22 and 23 takes the first route—and it is the one students should understand thoroughly first.

Formulas and Procedures

One: the Epipolar Constraint

Let p_1 and p_2 be a pair of corresponding points in two images (in homogeneous pixel coordinates) and K_1 and K_2 the camera intrinsics (Chapter 9); then the epipolar constraint is

$$p_2^{T} \cdot F \cdot p_1 = 0$$

Converting the pixel coordinates to normalized coordinates with the intrinsics, $x = K^{-1} \cdot p$, rewrites the constraint in essential matrix form:

$$x_2^{T} \cdot E \cdot x_1 = 0$$

The relation between the two matrices, and the composition of the essential matrix, are

$$E = K_2^{T} \cdot F \cdot K_1 \quad , E = [t]_x \cdot R$$

where $[t]_x$ is the skew-symmetric matrix of the translation vector (Chapter 10) and R the rotation matrix. This expression shows clearly that the essential matrix is a compact encoding of translation and rotation. Factoring R and t out of E gives the camera motion, but t has only a direction and no magnitude (the scale ambiguity).

Two: Computing the Epipolar Line

For a point p_1 in the first image, the epipolar line in the second image is

$$l_2 = F \cdot p_1 \qquad \textit{(the coefficients of the line equation } a \cdot u + b \cdot v + c = 0\textit{)}$$

Conversely, the epipolar line of p_2 in the first image is $l_1 = F^T \cdot p_2$. Drawing l_2 on the second image in an implementation lets one see with one's own eyes the constraint that the corresponding point must fall on this line—the most illuminating step of this chapter's hands-on work.

Three: the Reprojection Error (the Objective of PnP and BA)

For a three-dimensional point P, a camera pose (R, t), intrinsics K, and an observed pixel u_obs, the reprojection error is

$$e = u_obs - \pi(K, R, t, P)$$

Solving PnP means minimizing the sum of the squared reprojection errors over all points:

$$(R^*, t^*) = \operatorname{argmin} \Sigma_i \, \| u_obs_i - \pi(K, R, t, P_i) \|^2$$

Note: PnP optimizes only (R, t) and holds the three-dimensional points fixed, while a full bundle adjustment (Section 12.5) optimizes poses and three-dimensional points together. The two are different applications of one mathematical framework.

Four: the Number of RANSAC Iterations

Let the inlier ratio be w, let each sample require s points, and let the desired probability of drawing at least one all-inlier sample be p; then the number of iterations required is

$$N = \log(1 - p) \,/\, \log(1 - w^s)$$

For example: with an inlier ratio w = 0.5, s = 8 points for the essential matrix, and a desired success probability p = 0.99, N ≈ 1177. Change to a method needing only s = 5 points and N falls to about 145—which shows how precious a solver requiring few points is within RANSAC, and why the five-point algorithm is preferred to the eight-point.

Five: the Complete Solution Pipeline of the Visual Front End

Figure 21-5 The Complete Flow from Feature Correspondences to Camera Pose

① Feature matching (Chapter 20) → a set of correspondences that may contain false matches

↓

② Judge the situation:

• The system is initializing, with no map points → take 2D–2D (epipolar geometry)

• Map points already exist → take 3D–2D (PnP)

↓

③ RANSAC iteration: sample at random → solve the model → count inliers

↓

④ Take the model with the most inliers and reject the outliers

↓

⑤ Refine the pose by a nonlinear optimization over all the inliers (minimizing the reprojection error)

↓

⑥ Triangulation: for new correspondences of sufficient parallax angle, recover three-dimensional coordinates and add them to the map

↓

⑦ Output the camera pose + the new map points → hand them to the back end (Chapter 22)

Cases

Case 21-1 Why Monocular SLAM Dies When You Stand in One Place and Turn Around

This is the most common frustration students meet in implementing monocular visual SLAM. Phone or camera in hand, the system keeps displaying initializing and will not succeed however it is tried. The camera calibration, the number of features, and the lighting have all been checked and are all normal—so where is the trouble?

The answer is usually extremely simple: you have only been turning in place, without walking.

By the principle of Section 21.5: under pure rotation the positions of the optical center at the two exposures coincide exactly, the length of the baseline is zero, and the parallax angle is zero. The rays from the two optical centers through the corresponding pixels then overlap completely and there is no intersection to speak of—triangulation has no mathematical solution, and the system naturally cannot build an initial map.

More subtly still, epipolar geometry also degenerates under pure rotation: the translation vector t is zero, the essential matrix $E = [t]_x \cdot R$ becomes the zero matrix, and the solution naturally fails. What should be used then is the homography (Section 21.3)—which is exactly why ORB-SLAM computes both models at initialization and chooses between them automatically.

The remedy is simple: translate. Hold the camera and move it a few tens of centimeters from side to side or back and forth while keeping the features in view, and initialization usually succeeds within a few seconds. This case echoes IMU initialization requires excitation of motion in Section 7.7—the shared lesson being that both visual

and inertial systems need motion with content before certain quantities become observable.

Case 21-2 Visualizing the Epipolar Line: Seeing a Three-Dimensional Constraint at a Glance

The formulas of epipolar geometry look abstract, but the moment they are drawn out everything becomes clear. This is one of the most popular demonstrations in a university classroom.

The procedure: take two photographs of the same scene from slightly different viewpoints, find corresponding points with ORB, estimate the fundamental matrix F with RANSAC, and then, for every feature point in the first image, compute its epipolar line in the second image by Formula Two and draw it.

The result is a spectacular picture: many straight lines appear on the second image, and every corresponding feature point falls exactly on its own line. More remarkable still, if the camera moved forward, all these lines converge on one point in the frame—that is the epipole, the position at which the other camera's optical center projects, geometrically corresponding to the direction in which you advanced.

The teaching value of this experiment is very high: it lets students see with their own eyes that two ordinary photographs conceal a three-dimensional constraint of such rigor. The abstract matrix F suddenly becomes a visible line. Teachers are strongly encouraged to put this demonstration into the classroom and to have students photograph and implement it themselves—the hands-on activity of this chapter takes it as its core.

Case 21-3 The Universal Value of RANSAC: a Methodology Beyond Vision

RANSAC was proposed by Fischler and Bolles in 1981, and it was proposed for the pose estimation problem of computer vision in the first place (interestingly, what they treated in proposing it was precisely a PnP-type problem). But in the forty years since, its influence has reached far beyond vision.

It has been used for fitting planes and cylinders in point clouds (Chapter 17), for separating the ground in LiDAR data (LeGO-LOAM in Chapter 18), for calibrating the sensors of a robot, for sequence alignment in bioinformatics, for trend fitting in financial data—anywhere that outliers are mixed into the data and the mainstream model must be found.

RANSAC is so general because it embodies a profound methodology: when you cannot trust every datum, do not try to average over all the data but look for the explanation that the most data agree with. This is a change of thought from believing the data to believing the consensus.

The lesson for students is this: in studying an algorithm, besides remembering how it is done, understand the pattern of thought behind it. RANSAC's random sampling plus majority rule, the Kalman filter's weighting by reliability (Chapter 11), graph optimization's spreading of contradiction evenly (Chapter 12)—these frameworks of thought will serve you far longer than any particular formula.

Hands-On Activities

Activity 21-1 Epipolar Geometry and the Estimation of Camera Motion

Purpose: to compute the fundamental matrix with your own hands, draw the epipolar lines, carry out PnP and triangulation, and witness the power of RANSAC in rejecting false matches. This activity uses Python with OpenCV and needs no more than a phone camera, and it is among the experiments in this book that build geometric intuition best.

Part One: Visualizing the Epipolar Lines (everyone; the core of this chapter)

1. Take the material: photograph a classroom or a corridor twice, the second shot translated about 30 centimeters to the right of the first (note that this is a translation, not a turn in place). Make sure the frame has rich texture.
2. Extract and match features with ORB (the fruits of Chapter 20), applying the ratio test for a first rejection of false matches.
3. Estimate the fundamental matrix F with OpenCV's findFundamentalMat together with RANSAC, obtaining the inlier and outlier labels at the same time.
4. By Formula Two, compute the epipolar lines in the second image for the inliers of the first and draw them; draw the corresponding feature points as well.
5. Observe and verify: does every corresponding point fall on its own epipolar line? How large is the error (in pixels)? Do all the epipolar lines converge on one point (the epipole)?
6. A control experiment: take another set of photographs turning in place, repeat the procedure above, and observe whether the estimate becomes unstable—verifying the degeneracy of Case 21-1 at first hand.

Part Two: the Power of RANSAC (everyone)

7. Estimate F without RANSAC (by least squares instead), draw the epipolar lines, and observe whether they are plainly shifted.

8. Compare the inlier count: what percentage of all the matches does RANSAC judge to be inliers? What do the rejected outliers look like as matches in the image (draw them separately and look)?
9. Introduce false matches deliberately: add a number of random wrong correspondences to the match list, rerun RANSAC, and observe whether it still rejects them correctly. At what proportion does it fail?
10. Compute by Formula Four: at your inlier ratio, how many iterations are needed in theory for a 99 percent probability of success? Compare with OpenCV's default.

Part Three: PnP and Triangulation (university level)

11. A PnP implementation: with the camera calibrated in Chapter 9, photograph a checkerboard (or an ArUco marker); the three-dimensional coordinates of the grid points being known, solve for the camera pose with solvePnP and verify by projecting the coordinate axes back into the image.
12. A triangulation implementation: using the camera motion obtained in Part One, triangulate the matched inliers (OpenCV's triangulatePoints) and display the recovered three-dimensional points with matplotlib or Open3D.
13. A parallax angle experiment: triangulate with two sets of photographs, one translated 10 centimeters and the other 50, and compare the quality of the recovered point clouds (their scatter, the plausibility of the depths), verifying the effect of parallax angle on accuracy in Section 21.5.
14. A scale check: measure the true distance between two points in the scene and compare it with the triangulation result—you will find the proportions right but

the absolute values wrong, experiencing monocular scale ambiguity at first hand (Section 20.5).

Discussion Questions

1. In your visualization of the epipolar lines, by how many pixels do the corresponding points depart from their lines? Where does this error come from (error in feature localization? error in camera calibration?)
2. For the set of photographs taken while turning in place, why is the estimate of the fundamental matrix unstable? By Section 21.3, what model should be used instead?
3. What characteristics do the outliers RANSAC rejects display in the image (are they randomly distributed, or concentrated in certain regions such as repetitive texture)?
4. By how much do the triangulation results at 10 centimeters and at 50 centimeters of translation differ? What does this suggest about when a SLAM system ought to triangulate new map points?
5. (University level) The three-dimensional points recovered by triangulation have the right proportions but the wrong absolute values. To correct this, which three choices do you have by Section 20.5? What does each cost?

Extended Challenge (university level)

First: string the parts of this chapter into a complete two-frame visual odometry—feature matching → estimating the essential matrix with RANSAC → factoring out R and t → selecting the correct solution by the cheirality check → triangulating the initial map points. This is a simplified version of ORB-SLAM's initialization module, and on completing it you will have a very deep understanding of the system architecture of

Chapter 22. Second: implement ORB-SLAM's model selection strategy—compute the fundamental matrix and the homography at the same time, judge automatically from the fitting scores whether the scene is planar or three-dimensional, and initialize with the appropriate model. Test it on a white wall and on a three-dimensional scene in turn and verify that the choice is correct.

Further Reading

1. R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed., Cambridge University Press, 2004. (The authoritative bible of multiple view geometry and the source of the complete derivations of every topic in this chapter.)
2. M. A. Fischler and R. C. Bolles, "Random Sample Consensus," Communications of the ACM, 1981. (The original RANSAC paper, applied to the pose estimation problem at its proposal.)
3. D. Nistér, "An Efficient Solution to the Five-Point Relative Pose Problem," IEEE TPAMI, 2004. (The five-point method for the essential matrix, which greatly reduces the iterations RANSAC requires.)
4. V. Lepetit, F. Moreno-Noguer, and P. Fua, "EPnP: An Accurate O(n) Solution to the PnP Problem," IJCV, 2009. (The original EPnP paper, OpenCV's default PnP solver.)
5. Gao Xiang and Zhang Tao, Fourteen Lectures on Visual SLAM, Lecture 7 (Visual odometry: the feature point method). (The best Chinese-language teaching material on epipolar geometry, PnP, and triangulation, with complete source code.)

6. The Camera Calibration and 3D Reconstruction module of the official OpenCV documentation (docs.opencv.org). (Instructions for findFundamentalMat, solvePnP, triangulatePoints, and the rest.)
7. The Multiple View Geometry unit of UZH's Vision Algorithms for Mobile Robotics course (introduced in Chapter 2). (Freely available course videos and notes, to be studied alongside this chapter.)

Summary

1. Camera motion estimation falls into three situations according to what is known: 2D–2D uses epipolar geometry (system initialization), 3D–2D uses PnP (the mainstay while SLAM is running), and 3D–3D uses ICP (Chapter 17). Understand this classification and you can see when a SLAM system calls which kind of solver.
2. The core of epipolar geometry is the epipolar constraint: a point in the first image must fall on one epipolar line in the second, which reduces the search space from two dimensions to one. Its four elements are the baseline, the epipolar plane, the epipole, and the epipolar line, and every epipolar line in a given image passes through its epipole.
3. The matrix forms of the epipolar constraint are $p_2^{\mathrm{T}} \cdot F \cdot p_1 = 0$ and $x_2^{\mathrm{T}} \cdot E \cdot x_1 = 0$; the fundamental matrix uses pixel coordinates and needs no intrinsics, while the essential matrix uses normalized coordinates and is the purer of the two. Factoring $E = [t]_x \cdot R$ gives the motion, but the translation is known only in direction (the scale ambiguity), and the four solutions must be screened by the cheirality check. In a planar scene or under pure rotation a homography should be used instead.

4. PnP solves for camera pose from known three-dimensional points and their projections in the image; its merits are absolute scale, no degeneracy, few points required, and high efficiency. Common solutions include P3P, EPnP, and DLT, refined by a nonlinear optimization that minimizes the reprojection error—it is in essence a simplified bundle adjustment with the three-dimensional points held fixed.
5. Triangulation recovers three-dimensional coordinates from the intersection of two rays, and its accuracy depends on the parallax angle: when the angle is small (the camera having hardly moved, or the object being far away) the depth is extremely inaccurate; under pure rotation the angle is zero and triangulation is impossible—the most common cause of failed initialization in monocular SLAM.
6. RANSAC stands against false matches by sampling at random, solving a model, counting inliers, iterating, and taking the model with the most inliers; it does not correct outliers but excludes them, and can tolerate a very high proportion of them. The number of iterations required is set by the inlier ratio and the number of sample points, which is why the five-point method is preferred to the eight-point.
7. Three turns of thought lead from image processing to visual SLAM: from processing an image to understanding geometry, from a single image to the relation among several views, and from exact assumptions to robust estimation. Understanding what an algorithm is solving, when it is used, and where it will fail matters more than memorizing a derivation.

Carrying Forward

But all of these are still parts—features, epipolar geometry, PnP, triangulation, RANSAC. How are they assembled into a complete system that runs stably over long periods, builds a consistent map, and recovers when tracking is lost? In the next chapter we dissect ORB-SLAM2: how it works with three threads in concert, how key frames and map points are managed, how the covisibility graph organizes spatial relations, and how loop closures are detected and corrected. It is worth taking the three figures of this chapter along to every one of its design decisions: why a key frame waits for sufficient parallax before creating new points (Figure 21-3(b)), why RANSAC must be run at every step (Figure 21-4(a)), and why initialization computes both the fundamental matrix and the homography before choosing one. A visual SLAM system that really runs is about to be laid out whole.

This chapter has assembled the geometric tools of visual SLAM. But all of these are still parts—features, epipolar geometry, PnP, triangulation, RANSAC. How are they assembled into a complete system that runs stably over long periods, builds a consistent map, and recovers when tracking is lost? In the next chapter we dissect ORB-SLAM2: how it works with three threads in concert, how key frames and map points are managed, how the covisibility graph organizes spatial relations, and how loop closures are detected and corrected. A visual SLAM system that really runs is about to be laid out whole.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 22

ORB-SLAM2 in Theory and in Practice

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 20 gave us features and Chapter 21 gave us geometry. This chapter assembles them into a system that really runs.

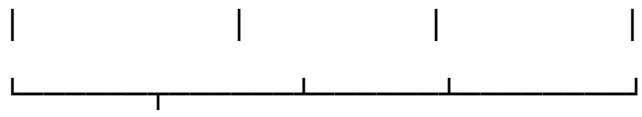
ORB-SLAM2 is worth a whole chapter's reading, not because any one of its algorithms is especially clever, but because it demonstrates how much engineering judgment it takes to put the right parts together: when a key frame should be inserted, how far a new map point should be trusted, what to do when the map grows too large, what happens if loop closure detection is wrong. The formulas of a paper hold no answer to these questions; only the design of a system can answer them.

The three figures of this chapter exist to quantify that engineering judgment. Figure 22-1 counts how many new map points are left after passing five gates and how many false matches were blocked; Figure 22-2 demonstrates something counterintuitive—the loop closed, and yet the map was not repaired; and Figure 22-3 draws the covisibility graph out to explain why it is not the same as the frames adjacent in time.

Figure 22-0 The Link Map of Chapter 22

The Parts Already Gathered

Chapter 20 ORB features Chapter 21 Epipolar geometry and PnP Chapter 12 BA Section 20.7 Bag of words



▼

【Chapter 22 ORB-SLAM2: assembling the parts into a system】

Three threads ─ key frames and map points ─ the covisibility graph ─ loop closure and global

BA

▼

Chapter 23 ORB-SLAM3 (visual-inertial + multiple maps)

▼

Chapter 24 The vision-navigated robot Part VIII ROS implementation

Learning Objectives

In Chapters 20 and 21 we gathered every part of visual SLAM: features, epipolar geometry, PnP, triangulation, RANSAC. But parts are not a system—how are they made to work in concert, to stay stable over long periods, to recover when tracking is lost, to correct themselves at a loop closure? This chapter dissects ORB-SLAM2, acknowledged as the most mature open-source visual SLAM system, and assembles the theory into an architecture that really runs. After studying this chapter, students should be able to:

1. Explain the role of the ORB feature within ORB-SLAM2 and the design merits of one set of features running through all three great tasks.
2. Explain how initialization differs among the monocular, stereo, and RGB-D modes.
3. Describe the workflow of the tracking thread and its three tracking strategies.
4. Explain how the local mapping thread adds map points, performs local BA, and culls redundancy.
5. Explain how the loop closing thread detects a loop, verifies the similarity transform, and corrects the global drift.
6. Describe the data structures and the uses of key frames, map points, the covisibility graph, and the spanning tree.

7. Plan and carry out an implementation and evaluation of ORB-SLAM2 on a public dataset.

Core Concepts

Chinese Term	English Term	Explanation
關鍵幀	KeyFrame	An image frame retained to represent a viewpoint, holding its pose and features.
地圖點	MapPoint	A three-dimensional feature point in the map, recording coordinates, descriptor, and observations.
共視圖	Covisibility Graph	The graph formed by linking key frames that observe map points in common.
生成樹	Spanning Tree	The minimum spanning tree linking all key frames; a lightweight structure.
本質圖	Essential Graph	Edges of high covisibility plus the spanning tree and the loop closure edges, used for pose graph optimization.
追蹤	Tracking	The front-end thread that estimates the camera pose for every frame.
局部建圖	Local Mapping	The thread that manages and optimizes the local map.
閉環	Loop Closing	The thread that detects a revisit and corrects the global drift.
僅運動 BA	Motion-only BA	A bundle adjustment that holds the map points fixed and optimizes only the camera pose.
局部 BA	Local BA	A bundle adjustment over the local key frames and the map points they see.
全域 BA	Full BA	A bundle adjustment over all key frames

Chinese Term	English Term	Explanation
		and map points.
地點辨識	Place Recognition	The module that judges by bag-of-words comparison whether a place has been visited before.
重定位	Relocalization	Finding one's position again within the map after tracking has failed.
相似變換	Similarity Transform	A transformation including scale, used in monocular loop closure correction.

22.1 ORB Feature Extraction

ORB-SLAM2 was developed by Mur-Artal and Tardós of the University of Zaragoza in Spain and is the most influential open-source system in the field of visual SLAM. Its name states its core outright: the whole system is built upon the ORB feature.

Why ORB: Three Reasons

Chapter 20 has already discussed ORB's advantage in speed, but within ORB-SLAM2 the choice of ORB rests on deeper considerations at the level of the system. These features are robust to rotation and to scale, and they also have good invariance to the camera's automatic gain, its automatic exposure, and changes of illumination; besides this they are fast to extract and to match, which supports real-time operation, and they show good precision and recall in bag-of-words place recognition.

- Speed: the hard requirement of a real-time system (Section 20.3).
- Invariance to exposure and changes of illumination: this point is often overlooked yet is absolutely crucial—a camera's automatic gain and automatic exposure continually alter the brightness of the image, and without this invariance the system loses tracking frequently. It is also precisely the soft spot of the direct method (Section 20.2).
- Suitability for a bag of words: a binary descriptor builds a vocabulary tree efficiently, making place recognition both fast and accurate.

A Key Design: One Set of Features, Three Uses

This is one of the most elegant decisions in the architecture of ORB-SLAM2. The system uses one and the same set of ORB features for all three tasks of tracking, mapping, and place recognition.

The benefits of this design are enormous:

1. Duplicate computation is saved: features need be extracted from an image only once and the three modules share them. Were tracking to use ORB and loop closing SIFT, two sets of features would have to be computed for every frame and the computational load would double.
2. The data structures are unified: a map point need store only one kind of descriptor, and a key frame only one bag-of-words representation.
3. The behavior is consistent: a feature that tracking can match, loop closing can match too, and no contradiction arises in which tracking sees something that loop closing cannot recognize.

Implementation Details: the Pyramid and Even Distribution

- The image pyramid: to gain scale invariance, ORB-SLAM2 scales the image into a pyramid of several levels and extracts FAST corners at every level. This lets the system match the same object at different distances.
- Even distribution: extracting FAST corners directly causes features to concentrate where texture is rich (crowding onto a single picture, for instance) while other regions are left empty. ORB-SLAM2 subdivides the image recursively with a quadtree to ensure that features are spread evenly over the whole frame—exactly the same intent as LOAM's selection of features by subregion in Section 18.2 (avoiding a degenerate geometric condition).

22.2 Initialization for Monocular, Stereo, and RGB-D

A great feature of ORB-SLAM2 is that one system supports three kinds of camera. But initialization differs greatly among the three, and this is one of the keys to understanding the system.

Monocular: the Most Difficult Initialization

A monocular camera has no depth information and must build its initial map from the parallax of two images (Chapter 21). And as Case 21-1 of Chapter 21 described, if the scene is planar or the camera rotates purely, epipolar geometry degenerates.

ORB-SLAM2's remedy is extremely clever: it computes two geometric models in parallel—a homography for a planar scene and a fundamental matrix for a non-planar one—and then chooses between them according to their relative scores.

- If the homography scores distinctly higher, the scene is judged planar and the motion is factored out of the homography.
- Otherwise the solution proceeds by the fundamental matrix (the essential matrix).
- If neither is good enough (insufficient parallax, for instance), this attempt at initialization is abandoned and the next frame is awaited for a retry.

This is the mechanical reason behind the initialization that will not succeed in students' implementations: the system is in fact protecting you, refusing to build a wrong map from an unreliable model. The remedy is as Chapter 21 described—translate, and create enough parallax.

Stereo and RGB-D: Able to Begin at Once

Initialization for stereo and RGB-D cameras is far simpler: there is depth information in the very first frame, so feature points with depth can be turned into map points directly, an initial map is built, and the system enters normal tracking immediately.

- Stereo: depth is computed from the disparity of the left and right images (Section 9.6).
- RGB-D: the depth image is read directly.
- Their common merits: absolute scale, no waiting for parallax to accumulate, and no failure under pure rotation.

Distinguishing Near Points from Far Points

ORB-SLAM2 has an important design in its stereo and RGB-D modes: feature points are divided by depth into near points and far points. The depth measurement of a near point (one whose depth is less than some multiple of the baseline) is reliable and can be triangulated into a map point directly; the depth of a far point is unreliable (the depth error growing with the square of the distance in Section 9.6), so it is triangulated only after observations from several viewpoints have accumulated enough parallax, just as in the monocular case.

This design displays engineering pragmatism: not all measurements are equally credible, and a system ought to handle them according to their credibility—of a piece with the spirit of weighting by reliability in the Kalman gain of Chapter 11.

22.3 The Tracking Thread

ORB-SLAM2 runs in three parallel threads. The system has three principal parallel threads: tracking, which localizes the camera for every frame by matching features against the local map and applying a motion-only BA to minimize the reprojection error; local mapping, which manages and optimizes the local map and performs local BA; and loop closing, which detects large loops and corrects the accumulated drift by pose graph optimization. This section looks at the first.

The Task of Tracking

The tracking thread is the front end, and its sole task is to compute, for every image that comes in, where the camera is now, as fast as possible. It must run in real time (in step with the camera's frame rate), so its computation must be light.

Three Tracking Strategies

ORB-SLAM2 adopts three strategies according to circumstance, attempted in order from the easiest to the hardest:

4. Tracking by the motion model: assume the camera moves at constant velocity, predict the current pose from the velocity of the previous frame, then match against the map points of the previous frame and refine. This is the fastest and the principal path under normal conditions.
5. Tracking by the reference key frame: if the motion model fails (the camera suddenly changing its motion, say), match features against the nearest reference key frame with the help of the bag of words and then solve for pose.
6. Relocalization: if both of the above fail (tracking is lost), the place recognition module is started to search the key frame database of the whole map for a similar view, and the pose is solved by PnP with RANSAC so that the system finds itself again.

Tracking the Local Map

Having obtained a preliminary pose, the tracking thread does one more crucial thing: it projects further map points from the local map into the current frame, looks for additional matches, and optimizes once more. This lets the pose estimate use more observations, and the accuracy improves appreciably.

The local map is obtained by way of the covisibility graph (Section 22.6): find the

key frames that share many observations with the current frame, together with the map points they see. These graph structures let the system take a local window of key frames, so that tracking and local mapping operate within a local range and can therefore work in a large environment.

The Selection of Key Frames

The last task of the tracking thread is to decide whether this frame ought to become a key frame. The criteria broadly include: enough frames have passed since the last key frame was inserted; the local mapping thread is idle; and the current frame tracks enough map points but its proportion of shared observations with the reference key frame has fallen (indicating an appreciable change of viewpoint).

ORB-SLAM2's strategy is to insert loosely and cull strictly: insert key frames as freely as possible to keep tracking robust, then let the local mapping thread cull the redundant ones (Section 22.4). This design makes the system hard to lose under fast motion while keeping the key frames from swelling without limit.

22.4 The Local Mapping Thread

The local mapping thread is the middle end; it need not run in real time, but it is responsible for maintaining the quality of the map. The local mapping thread inserts new points into the local map and optimizes the local map by performing local bundle adjustment.

Five Steps

7. Insert the key frame: the new key frame is inserted into the covisibility graph and into the spanning tree that links a key frame to the key frame with which it

shares the most points, and a bag-of-words representation of it is built (for the data association used in triangulating new points).

8. Cull poor map points: a newly created map point must pass a strict test before it is kept—it must be observed by enough key frames and its tracking success rate must be high enough. Those that do not qualify are deleted at once, lest they pollute the map.
9. Create new map points: new map points are produced by triangulating the ORB features of key frames connected in the covisibility graph; unmatched ORB features in one key frame are compared with unmatched ORB features in another, and a pairing is valid only if it satisfies the epipolar constraint; after a pair of ORB features has been triangulated, the depth in both frames must also be checked to be positive, and the parallax, the reprojection error, and the consistency of scale are all tested.
10. Local bundle adjustment: perform a local BA (Section 12.5) over the current key frame, the key frames connected to it in the covisibility graph, and all the map points those key frames see. This is the principal source of accuracy.
11. Cull redundant key frames: if nine tenths or more of the map points seen by a key frame are also seen by at least three other key frames, it is redundant and is deleted. This keeps the scale of the map from growing without limit over time.

Strict Quality Control

Students should notice that steps 2, 3, and 5 are all deleting things. This is precisely the secret of ORB-SLAM2's accuracy—it is extremely severe about the quality of the map, preferring fewer map points and key frames to unreliable data.

The multiple tests applied in triangulating a new point are especially rigorous: the

epipolar constraint, positive depth, the parallax angle (Section 21.5), the reprojection error, and the consistency of scale—five gates, not one of which may be omitted. This reflects an engineering principle: a wrong map point is more dangerous than no map point, because it will go on misleading every later estimate of pose.

A Consideration of the Baseline

One further detail of the implementation is worth mentioning: in triangulating new points the system takes neighboring key frames from the covisibility graph (more of them in monocular mode, fewer otherwise) and uses only those whose baseline is large enough. This is a direct application of the parallax angle decides the accuracy of triangulation of Section 21.5—a pair of key frames with too short a baseline triangulates to points whose depth cannot be trusted.

Measuring It: How Much Does Each of the Five Gates Block

Section 22.4 says that a new map point must pass five tests and that not one of the five gates may be omitted. That sounds reasonable, but how much does each of them actually block? This subsection counts it out with a reproducible simulation.

The setting is a laboratory 14×10 meters with 1,400 landmarks in all on the walls, the floor, and the furniture; the camera travels twice around an elliptical path (the second circuit of slightly smaller radius, since a real revisit does not follow the same line), and key frames are selected by the criteria of Section 22.3. Then, for every pair of covisible key frames the system would really attempt, the landmarks they see in common are taken as correct candidate matches, and 25 percent false matches are mixed in—half of them random mismatches and half deliberately made to fall on the epipolar line but at the wrong depth, because real false matches are often of exactly that kind (matching searches along the epipolar line to begin with).

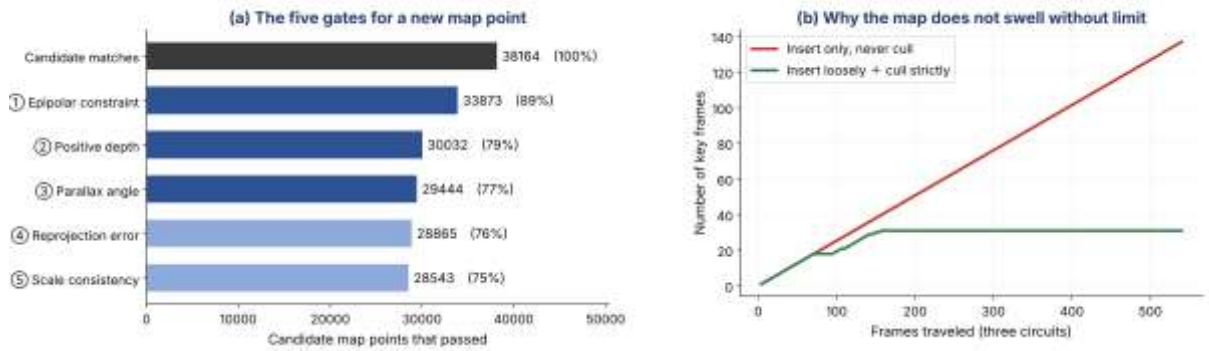


Figure 22-1 The Five Gates a New Map Point Must Pass, and Why Key Frames Do Not Grow Without Limit

Panel (a) is the funnel. Of 38,164 candidate matches, gate ① the epipolar constraint blocks 4,291 (89 percent remaining), gate ② positive depth blocks a further 3,841 (79 percent remaining), gate ③ the parallax angle blocks 588 (77 percent remaining), and gate ④ the reprojection error and gate ⑤ the consistency of scale block 579 and 322, leaving 28,543 at the end (75 percent).

What is most worth discussing in this figure is the gap between the first three gates and the last two. The epipolar constraint, positive depth, and the parallax angle together block more than nine tenths of all the candidates blocked, while the reprojection error and the consistency of scale between them block 2.4 percent of the candidates. Does this mean the last two could be dispensed with? It does not, for two reasons. First, the few hundred points they block are exactly those that passed the first three gates and are still wrong—and such points are the most dangerous of all, because they look entirely plausible. Second, this simulation does not model pyramid levels, whereas the scale consistency test of a real system compares the ratio of observation distances against the difference in pyramid level, and on real data the proportion it blocks is far higher.

More important than the number is the quality. Among the 28,543 surviving candidates,

97.1 percent are correct matches—a great improvement on the original 75 percent; of 9,541 false matches 8,706 were blocked (91 percent), while of 28,623 correct matches only 915 were killed by mistake (3.2 percent). This allocation, erring on the side of strictness, is the engineering realization of the sentence a wrong map point is more dangerous than no map point in Section 22.4.

Panel (b) answers another question: since key frames are inserted loosely, will they explode in number if the robot circles the same place three times? The red line is insertion with no culling—137 key frames accumulate over three circuits, and the growth is still linear. The green line adds the nine-tenths rule for culling redundancy (delete a key frame if nine tenths or more of the map points it sees are also seen by at least three other key frames), and the result settles at 31 by the end of the first circuit and stays at 31 for the remaining two.

This is precisely the key to ORB-SLAM2's ability to run for long periods. The scale of the map is decided by how large the environment is and not by how long one has been running—the same idea as the map maintenance of the inspection robot in Section 19.6 and the submap management of Cartographer in Chapter 15.

Insert Loosely, Cull Strictly: Why Not Simply Be Strict from the Start

A natural question: since seven tenths will be deleted in the end, why not insert fewer to begin with?

Because the amount of information available at insertion and at culling is entirely different. When the tracking thread decides whether this frame should be a key frame, it can see only the present; it does not know where the camera will go next or whether this viewpoint will later prove important. The local

mapping thread, when it culls, has already seen the later key frames and can judge whether a frame is truly redundant.

So this is not wasting first and tidying up after but deferring the decision until there is information enough. The same thinking appeared in the resampling of the particle filter in Chapter 14 and in the timing of submap freezing in Chapter 15, and it is a fairly general principle in system design.

22.5 The Loop Closing Thread

The loop closing thread is the back end, and it handles the most fundamental problem of SLAM: accumulated drift. The loop closing thread is responsible for detecting a revisited region and correcting the accumulated error by pose graph optimization in the g2o framework; after the pose graph optimization it launches a fourth thread to perform a full bundle adjustment, optimizing the whole map together with the trajectory estimate among the key frames.

Step One: Detecting Candidate Loop Closures

ORB-SLAM2 recognizes a revisited place by means of the DBoW2 library; it compares the features of the current image, in binary form, with the binary features stored in the hierarchical DBoW2 database, and if the similarity of the current frame to some representation in the database reaches a certain score, a loop closure has been detected.

The key here is efficiency: the database may hold thousands of key frames, and matching features against each in turn is out of the question. The bag-of-words model compresses each image into a statistical vector of visual words, so that comparison

requires only the similarity of two vectors and is extremely fast—precisely the value of DBoW2 introduced in Section 20.7.

Step Two: Geometric Verification

Similarity in the bag of words means only that it looks alike, which is not enough to confirm. The system next performs a strict geometric verification: it matches features between the candidate loop frame and the current frame and solves for the transformation between them. Only if the similarity is supported by enough inliers is the loop closure accepted.

This step is of the first importance. Chapter 12 and Section 15.4 both stressed it: one wrong loop closure drags the map bodily into a wrong position, with catastrophic consequences. Better, therefore, to miss a loop closure than to accept a wrong one.

Step Three: the Similarity Transform and the Correction of Scale

Here is a detail peculiar to monocular SLAM. Because monocular SLAM has scale drift (Section 20.5), the current scale at a loop closure may differ from the initial scale. What is solved for is therefore not an ordinary rigid transformation (rotation plus translation, Chapter 10) but a similarity transformation—with one further scale factor.

In correcting, the system spreads this difference of scale over the whole trajectory as well, correcting position and scale drift together. Stereo and RGB-D modes, having absolute scale, need only an ordinary rigid transformation.

Step Four: Pose Graph Optimization and Global BA

- Pose graph optimization: perform a pose graph optimization (Chapter 12) on the essential graph (Section 22.6) to pull the whole trajectory straight quickly.

Because only poses are optimized (without the map points) it is fast and can be completed in real time—exactly the strategy of Cartographer in Chapter 15.

- Global BA: after the pose graph optimization, the system launches a fourth thread to perform a full bundle adjustment, optimizing all key frame poses and all map points together for the best overall solution in structure and motion. This step is slower and is therefore run in the background, where it does not block the front-end tracking.

This two-stage design of pull straight quickly first and optimize finely afterward should feel familiar to students—it is the same pattern of system design embodied in the two-layer architecture of LOAM in Chapter 18 and the two subsystems of Cartographer in Chapter 15.

Measuring It: What Exactly Did the Loop Closure Repair

Section 22.5 says that loop closure corrects accumulated drift. That is true, but there is a point commonly misunderstood: joining the two ends of a loop does not mean the middle is now correct. This subsection measures the matter.

The experiment uses a simplified planar pose graph (the camera circling on a horizontal plane, so that a two-dimensional pose suffices, which lets the experiment be reproduced in full without losing the point). Noise is added to each step of the odometry, and after one circuit an appreciable drift has accumulated. Three settings of pose graph optimization are then compared: adding a single loop closure edge, adding the edges of the essential graph, and adding the edges of the full covisibility graph. Every residual is divided by the standard deviation of its measurement (the information weighting of Chapters 11 and 12).

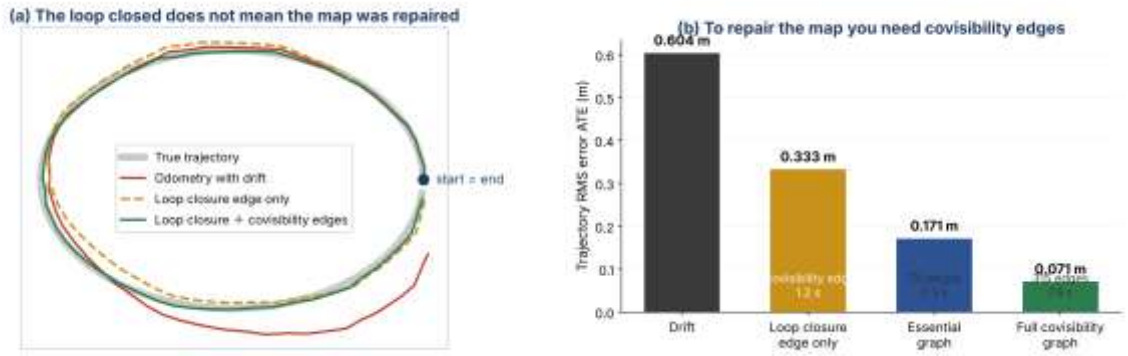


Figure 22-2 The Loop Closed Does Not Mean the Map Repaired

The thick gray line of panel (a) is the true trajectory and the red line the odometry estimate after drift, whose root mean square error over the whole trajectory (ATE) is 0.604 meters, and which on returning to the start is 1.48 meters away from where it should be.

The orange dashed line is the result of adding a single loop closure edge. It does indeed join the two ends—the endpoint error falls from 1.48 meters to 0.18, and the seam that had gaped open now closes. Judged by whether the trajectory closes, it succeeds completely.

But look at the whole trajectory: the orange dashed line still bulges plainly away from the gray one along the top and the lower left, at an ATE of 0.333 meters—only about half of what it was before optimization, which is a poor return for having removed almost all of the endpoint error. In other words, the loop closure has mostly moved the error away from the endpoint and spread it thinly over the middle; it has not removed it. The reason is intuitive: a single loop closure edge constrains only the relative relation between the first frame and the last, while the several dozen poses in between have no constraint among themselves beyond the odometry, and the optimizer has little reason to move them.

The green line is the result with covisibility edges added. The ATE falls to 0.071 meters, one eighth of the drift, and coincides almost exactly with the true trajectory. The key is that a covisibility edge binds together directly those poses that are not adjacent but see the same batch of landmarks, forming a net; the correction of the loop closure can therefore propagate along that net through the whole trajectory instead of stopping at the seam.

Panel (b) sets out the cost of the three settings as well. Loop closure edge only: 0 covisibility edges, 1.2 seconds, ATE 0.333 meters. The essential graph: 110 edges, 3.3 seconds, ATE 0.171 meters. The full covisibility graph: 315 edges, 7.6 seconds, ATE 0.071 meters. It must be said honestly that in this simulation the essential graph, while it halves the error again, still falls short of the full covisibility graph by a factor of more than two. What it buys is a third of the edges and under half the time.

That trade-off is exactly ORB-SLAM2's design: when a loop closure occurs, first perform pose graph optimization on the essential graph, because it is fast enough to finish in real time and will not block the front-end tracking; then launch a global BA in a background thread to refine the map with the full information. Pull the shape roughly straight by the cheap method, then make good the accuracy by the expensive one—the third appearance of the pattern already seen in the two-layer architecture of LOAM in Chapter 18 and the two subsystems of Cartographer in Chapter 15.

A Diagnosis Very Useful for a Project

If a student's SLAM system looks as though the loop closed but the map is still crooked, Figure 22-2 is describing your situation.

What to check is not loop closure detection but the covisibility relations: has

the system correctly created edges between non-adjacent key frames? Is the covisibility threshold set so high that the graph is too sparse? Does the pose graph optimization use the essential graph, or only odometry plus the loop closure?

Conversely, if there is a plain break at the seam of the loop closure, that is a problem of loop closure detection or geometric verification. The causes of the two symptoms are entirely different; tell them apart before setting to work.

22.6 The KeyFrame, the MapPoint, and the Covisibility Graph

The data structures of a system decide what it can do. This section describes the four core data structures of ORB-SLAM2.

The key frame (KeyFrame)

A key frame is a retained image frame representing a viewpoint of some significance. It stores: the camera pose (the transformation matrix of Chapter 10), all ORB feature points and their descriptors, the bag-of-words vector of that frame, and its connections to other key frames.

Why not keep every frame? Because the information in neighboring frames is highly redundant, and keeping all of them would make the scale of the BA explode. A key frame is the outcome of a trade-off: covering the most viewpoints with the fewest frames.

The map point (MapPoint)

A map point is a feature point in three-dimensional space. It stores: its three-dimensional coordinates, a representative descriptor (the most representative one

chosen from all its observations), the mean direction of observation, and a list of the key frames that have seen it.

That last item is of the first importance: it establishes a two-way link between map point and key frame, letting the system query quickly both which points this key frame sees and who has seen this point. The covisibility graph is derived from exactly this relation.

The covisibility graph

The system maintains a covisibility graph linking any two key frames that observe a map point in common, together with a minimum spanning tree linking all key frames.

- Nodes: every key frame.
- Edges: two key frames are joined by an edge if they observe enough map points in common; the weight of the edge is the number of map points observed in common.

The use of the covisibility graph is to define neighbors: a key frame that shares many observations with the current frame is one that is near in space and similar in viewpoint. Tracking and local mapping both operate only within this local range, which is how real-time performance is maintained in a large environment (the principle quoted in Section 22.3).

The Spanning Tree and the Essential Graph

- The spanning tree: a minimum spanning tree linking all key frames, each key frame joined to the key frame with which it shares the most observations. It is extremely light and guarantees that all key frames are connected.

- The essential graph: the structure on which pose graph optimization is performed at a loop closure. It contains all the edges of the spanning tree, the edges whose covisibility weight exceeds a threshold, and the loop closure edges. Having far fewer edges than the full covisibility graph, it lets the pose graph optimization finish quickly.

This layered design—the full covisibility graph for local operations, the pared-down essential graph for global optimization—is the key to ORB-SLAM2's balance of accuracy against efficiency. These graph structures also serve as the structure used for pose graph optimization at a loop closure.

Data structure	Contents	Principal use
KeyFrame	Pose, ORB features, bag-of-words vector, connections	The skeleton of the map; a variable of the BA
MapPoint	Three-dimensional coordinates, descriptor, list of observations	The reference for tracking; a variable of the BA
Covisibility graph	Key frame nodes + covisibility-weighted edges	Obtaining the local map; defining neighbors
Spanning tree	The edges of the minimum spanning tree	Guaranteeing connectivity; a lightweight structure
Essential graph	The spanning tree + strong covisibility edges + loop closure edges	Pose graph optimization at a loop closure

Table 22-1 The Core Data Structures of ORB-SLAM2

Drawing the Covisibility Graph Out

The covisibility graph is the most abstract idea in the chapter and the most easily mistaken for the key frames adjacent in time. This subsection draws it out. The

simulation is the same as that of Figure 22-1: the camera circles twice around a laboratory, the radius of the second circuit three tenths smaller than the first.



Figure 22-3 The Covisibility Graph, the Local Map, and the Essential Graph

Panel (a) is the full covisibility graph: 91 key frames and 561 edges (an edge is drawn where 15 or more map points are shared, and the thickness of a line represents the strength of the covisibility). Note the dense tangle of lines between the inner and the outer circuit—those are the cross-circuit covisibility edges, joining two frames dozens of key frames apart in time but looking at the same wall in space. The most important message of this figure is that the covisibility relation is decided by space and not by time.

Panel (b) is the local map. The red star is the current key frame and the orange its local map—the neighbors with which it shares most, plus the neighbors of those neighbors, 18 key frames in all. The tracking thread need only project into the current frame the map points seen by these 18 key frames, rather than trying the map points of all 91. Note that the orange points span both circuits at once: the robot can now use the map points built when it passed this way on the first circuit, which is the fundamental difference between SLAM and pure odometry.

More important still, the size of the local map does not grow with the map. In this figure it is 18 out of 91; if the robot were to circle ten more times and the key frames become

four hundred, the local map would still be only those twenty-odd near the current position. This is the concrete meaning of the covisibility graph letting the system work in a large environment in Section 22.6—the computation of tracking has nothing to do with the total size of the map, only with the local density.

Panel (c) is the essential graph: pared from 561 edges to 205, about a third. What is kept is the minimum spanning tree (ensuring every key frame is reachable) plus the edges of greatest covisibility strength. It preserves the overall connective structure of the graph and the principal cross-circuit connections while discarding a great many redundant short edges—this is the graph used for pose graph optimization at a loop closure, and the same kind of pruning as the 110 edges in Figure 22-2(b).

Three Panels for Three Threads

(a) The full covisibility graph: maintained by the local mapping thread, it is the connective structure of the whole map.

(b) The local map: used by the tracking thread, it decides which map points each frame is to be matched against.

(c) The essential graph: used by the loop closing thread, it is the object of pose graph optimization.

One and the same set of connections, from which three threads each take what they need—this is what Section 22.6 means by the data structures decide what a system can do. If the covisibility graph is built wrongly, all three threads go wrong at once, and with different symptoms.

22.7 Implementing ORB-SLAM2 and Testing It on Datasets

This section sets out the points of implementation, in preparation for the full implementation of Chapter 30 and the project of Chapter 31.

Compilation and Dependencies

ORB-SLAM2 is a C++ project, and several dependencies must be in place before compiling:

- OpenCV: image processing and feature extraction (Chapters 9 and 20).
- Eigen3: linear algebra.
- Pangolin: three-dimensional visualization, for displaying the map and the trajectory.
- g2o and DBoW2: already included in the project's Thirdparty folder, responsible respectively for graph optimization (Chapter 12) and place recognition (Section 20.7).

A note on implementation: ORB-SLAM2 was released several years ago, and on newer operating systems and versions of OpenCV a small amount of code may have to be modified before it will compile. Consulting a community-maintained fork, or using a containerized environment to avoid dependency trouble, is recommended—this is where students most often get stuck.

The Configuration File: Camera Parameters

A YAML configuration file must be prepared before running, containing:

- The camera intrinsics: fx, fy, cx, cy (the K matrix of Section 9.1).
- The distortion coefficients: k1, k2, p1, p2 (Section 9.2).
- The image size and the frame rate.

- The baseline times the focal length for stereo, and the depth scaling factor for RGB-D.
- ORB parameters: the number of features per frame (1000 to 2000 is common), the number of pyramid levels and the scale ratio, and the FAST threshold.

The importance of Chapter 9 must be stressed again here: if the camera calibration is inaccurate, there is no accuracy in the whole system to speak of. Be sure to perform a full calibration with your own camera, and do not simply apply the example parameters.

Public Datasets

Dataset	Camera type	Scene	Characteristics
TUM RGB-D	RGB-D	Indoor offices	Motion capture ground truth; suited to evaluating accuracy
KITTI	Stereo	Outdoor roads	Self-driving scenes; long trajectories over a large area
EuRoC MAV	Stereo + IMU	Indoor machine rooms and factory halls	A drone's point of view; includes fast-motion sequences

Table 22-2 Public Datasets Commonly Used for Visual SLAM

The recommended order of study is this: run an easy sequence of TUM RGB-D first (slow motion, indoors, with depth) to establish an experience of success, then take on the fast-motion sequences of EuRoC, and finally attempt the large outdoor scenes of KITTI.

Methods of Evaluation

- Absolute trajectory error (ATE): align the estimated trajectory with the ground truth and compute the root mean square of the position differences at corresponding times. This is the most commonly used measure of overall accuracy.
- Relative pose error (RPE): compute the error of the relative motion over a fixed interval of time, which reflects the degree of local drift.
- Tools: evo is at present the most convenient tool for evaluating trajectories, supporting a variety of dataset formats and able to align, compute metrics, and plot automatically.
- A special note for monocular: a monocular trajectory has no absolute scale (Section 20.5), and a scale alignment with the ground truth (a Sim3 alignment) must be performed before evaluation, or the error will be meaningless.

What to Watch For

While it runs, students are advised to watch several things in the Pangolin window:

12. The process of initialization: in monocular mode, observe how many frames and how much motion the system needs before it succeeds (against Section 22.2).
13. The distribution of key frames: observe how key frames are distributed along the trajectory—are they denser at turns and where the viewpoint changes greatly?
14. Covisibility relations: the lines between key frames in the visualization are the covisibility graph; observe its structure.

15. The moment of loop closure: on returning to the start, observe the instant at which the trajectory and the map are pulled straight (the striking picture of Chapters 12 and 15, reappearing here).
16. Loss of tracking and relocalization: cover the camera deliberately for a few seconds and then uncover it, and observe how the system loses itself and how it relocalizes—one of ORB-SLAM2's most impressive abilities.

At this point a complete visual SLAM system has been laid out whole. But ORB-SLAM2 still has two limitations: it is purely visual (with no IMU, so monocular has the scale problem and fast motion easily loses tracking), and once tracking is severely lost and relocalization fails, the system can go no further. The ORB-SLAM3 of the next chapter was born to solve exactly these two—a tightly coupled visual-inertial fusion, and a multiple-map system that can open a new map when tracking is lost and merge it later.

22.8 After ORB-SLAM2: Two Forks in Visual SLAM, 2024–2026

ORB-SLAM2 was published in 2017. Nearly a decade on, it is still the baseline most often used in comparing new methods—which in itself says something about its weight. But research in visual SLAM has indeed forked in two directions, and this section explains what each sets out to solve and how each relates to this chapter.

One: the First Fork, Replacing the Sparse Map with a Renderable Dense Map

ORB-SLAM2's map is sparse—tens of thousands of three-dimensional points, enough to localize with but not enough to see with. A principal line of recent years brings renderable representations of the 3D Gaussian splatting kind into SLAM, so that the

system builds, while localizing, a dense map that can be rendered back into an image.

What students should note is that systems of this kind have not discarded the content of this chapter. A representative work of late 2024 couples ORB features tightly with Gaussian splatting within a single optimization: tracking minimizes the photometric error and the reprojection error together, and the back end retains a lightweight bundle adjustment to suppress accumulated drift. The authors' reasoning is plain—a Gaussian splatting SLAM that relies on photometry alone is prone to artifacts and noise, chooses its viewpoints poorly, and lacks global optimization, and those three things are exactly what this chapter's key frame strategy, covisibility graph, and loop closure handle.

As to results, that work has a mean trajectory error of 0.91 centimeters on the TUM dataset against ORB-SLAM2's 1.00 centimeters—only a slight advantage in localization accuracy, the real difference lying in rendering quality (about 3.93 decibels better than the pure Gaussian splatting baseline). The authors also state the limitation honestly: training takes considerable time and the system cannot yet localize in real time. This is an important reminder—the price of a dense map is still high.

Two: the Second Fork, Replacing the Front End with a Learned Three-Dimensional Reconstruction Model

The second fork is more radical: hand even feature matching and epipolar geometry to a network. A class of methods appearing in 2025 takes a learned three-dimensional reconstruction model as the front end, predicting dense three-dimensional correspondences directly from two images and then attaching a SLAM back end.

Its most striking property is that it needs no camera calibration—the system uses a generic central camera model and can therefore handle video of unknown origin directly, which will come as something of a shock to students who spent a whole

section of Chapter 9 on calibrating intrinsics. As to speed, it runs at about 15 frames a second on a high-end graphics card and about 5 to 7 on a mid-range one; in accuracy it wins and loses against ORB-SLAM3 and other dense methods depending on the dataset.

Three: How Much of This Chapter Is Left

	ORB-SLAM2 (this chapter)	Gaussian splatting SLAM	SLAM with a learned front end
Form of the map	Sparse feature points	A renderable dense field	A dense point cloud
Front end	ORB matching + PnP	Photometric + features	A neural network predicts correspondences
Intrinsics required	Yes	Yes	Not necessarily
Key frame mechanism	Yes	Yes	Yes
Covisibility graph and loop closure	Yes	Mostly retained	Mostly retained
Real-time operation	Real time on a CPU	Mostly not yet real time	Needs a high-end GPU
Present position	The mature baseline	Rendering quality first	Calibration-free; generalization first

Table 22-3 A Comparison of the Three Routes

One thing can be seen from this table: what the two new routes replace is how the map is represented and how correspondences are found, while key frames, covisibility, loop closure detection, and pose graph optimization—that is, the whole of Sections 22.3

through 22.6 of this chapter—are kept almost untouched.

The reason is not hard to see. What these mechanisms solve is not how to compute a pose but how to manage information over a long run: which frames are worth remembering, which data are related to which, when one ought to admit that one has returned to an old place, and how the error should be redistributed. Whatever the front end is changed to, these questions remain. That is also the reason this chapter is worth reading carefully—what it teaches is system design, and system design outlives algorithms.

Formulas and Procedures

One: a Comparison of the Three Kinds of BA

ORB-SLAM2 uses three kinds of bundle adjustment (Section 12.5) at different stages, differing in which variables are optimized:

$$\textit{Motion-only BA: } \operatorname{argmin}_{R,t} \; \Sigma_i \, \rho\left(\| u_i - \pi(K, R, t, P_i) \|^2 \right)$$

The map points P_i are fixed and only the camera pose (R, t) is sought. Used on every frame in the tracking thread, it is the fastest.

$$\textit{Local BA: } \operatorname{argmin}_{R,t,P} \; \Sigma \text{ the reprojection errors of the local key frames and their map points}$$

Optimizes the poses of the local key frames together with the map points they see. Used in the local mapping thread, it is the principal source of accuracy.

$$\textit{Global BA: } \operatorname{argmin}_{\text{all } R,t,P} \; \Sigma \text{ the reprojection errors of all key frames and map points}$$

Optimizes the whole map; it is the heaviest and is run only in the fourth thread after a loop closure. In these expressions ρ is a robust kernel (Chapters 12 and 15), used to

reduce the influence of outlying observations.

Two: Pose Graph Optimization (Loop Closure Correction)

At a loop closure, pose graph optimization is performed on the essential graph, optimizing only key frame poses (without the map points):

$$argmin\ \Sigma_{(i,j)\in essential\ graph} \| log(\Delta T_{ij}^{-1} \cdot T_i^{-1} \cdot T_j) \|^2_\Sigma$$

where ΔT_{ij} is the relative pose measurement recorded on the edge and T_i and T_j are the key frame poses to be optimized. In monocular mode T is a similarity transform including scale (Sim3), while for stereo and RGB-D it is a rigid transform (SE3). This is the concrete realization of the pose graph optimization of Chapter 12.

Three: the Concerted Flow of the Three Threads

Figure 22-4 The Three-Thread Architecture of ORB-SLAM2

【Tracking thread】real time, run every frame

Extract ORB → solve for an initial pose by the motion model / reference frame / relocalization

→ track the local map → refine by motion-only BA → judge whether to insert a key frame

↓ (a new key frame)

【Local mapping thread】not real time, triggered by a key frame

Insert into the covisibility graph and spanning tree → cull poor map points → triangulate new map points

→ local BA → cull redundant key frames

↓ (a new key frame)

【Loop closing thread】not real time, run in the background

DBoW2 detects a candidate → geometric verification → solve the similarity transform (Sim3)

→ pose graph optimization on the essential graph (pulling the trajectory straight)

↓

【The fourth thread】global bundle adjustment, optimizing the whole map and trajectory

Four: the Five Tests for Triangulating a Map Point

Figure 22-5 The Tests a New Map Point Must Pass

① The epipolar constraint: the paired ORB features must satisfy the epipolar geometry (Chapter 21)

↓

② The positive depth test: after triangulation the depth must be positive in both frames

↓

③ The parallax angle test: the parallax must be large enough, or the depth is unreliable (Section 21.5)

↓

④ The reprojection error test: the error on projecting back into both images must be small enough

↓

⑤ The scale consistency test: the observed scale and distance must be plausible

↓

All five passed → it becomes a map point; any one failed → it is discarded

Cases

Case 22-1 Cover the Camera and Uncover It: the Magic Moment of Relocalization

This is the most dramatic demonstration of ORB-SLAM2's capability, and nearly every class is worth doing it once.

Let the system run normally until a stretch of map has been built, then cover the camera lens completely with your hand. The frame goes black, the features drop to zero instantly, the system immediately displays TRACK LOST, and the trajectory stops updating. Now take your hand away and let the camera see a region it has traversed before—in under a second the system jumps back to the correct position and the trajectory continues seamlessly.

What happened in that second? The system started the place recognition module: it turned the ORB features of the current image into a bag-of-words vector, searched the key frame database quickly for the most similar candidates, matched features against the candidates, solved for pose by PnP with RANSAC (Chapter 21), and resumed tracking once the verification passed.

A more interesting variation: after covering the camera, carry it to another corner of the room before uncovering it. The system can find itself just the same—the visual version of the kidnapped robot problem of Chapter 3. By contrast, pure visual odometry (with no map and no place recognition) could never come back after such an occlusion.

This demonstration lets students feel the difference between SLAM and VO at first hand: VO can only reckon forward, while SLAM possesses a map and a memory and can recover from failure. It is also the best gloss on the value of the loop closure detection module in the flow chart of Section 20.1.

Case 22-2 Insert Loosely, Cull Strictly: an Engineering Philosophy

ORB-SLAM2's treatment of key frames looks at first a little contradictory: the criteria for insertion are loose (key frames are inserted often), yet local mapping actively

culls the redundant ones. Why not simply be stricter from the start?

Because the two serve different ends. The aim at insertion is robust tracking: if key frames are too sparse, then under fast motion or a large turn the viewpoint of the current frame differs too much from that of the nearest key frame, matching fails, and tracking is lost. Loose insertion amounts to buying the system insurance.

The aim in culling is a compact map: key frames increasing without limit would swell the scale of the BA, exhaust memory, and gradually slow the system down. So once it is confirmed that the information of a key frame is adequately covered by others (nine tenths of its map points being observed by at least three other key frames), it is removed.

This strategy of stability first and refinement afterward is an engineering philosophy worth remembering: in an uncertain present, do more rather than less (keep your options), and pare down once there is enough information. Students will find it echoes the advantage of graph optimization in Chapter 12, that it can go back and correct history—it is precisely because the system can correct in hindsight that it dares to be loose in the moment.

Case 22-3 The Family That Grew Out of ORB-SLAM2

Once ORB-SLAM2 was open-sourced it became the basis of countless studies. Watching the branches of this technical family spread out shows the lay of the land in visual SLAM research.

- Adding other features: some work adds line segment features alongside point features. The system proposed by PL-SLAM is likewise built on three threads—visual odometry, local mapping, and loop closing; its map consists of key

frames, detected three-dimensional landmarks (both key points and line segments), a covisibility graph, and a spanning tree. Line features are especially valuable in man-made environments (corridors, buildings), where point features may be scarce but lines are abundant.

- Acceleration and lightening: some work proposes a descriptor-free variant. FastORB-SLAM is structured on ORB-SLAM2 and implements three threads, but establishes key point correspondences between adjacent frames by intensity matching rather than descriptor matching, handles the outlier problem with a two-stage, descriptor-independent key point matching method, and finally recovers the camera motion by minimizing the reprojection error.
- Extensions in application: some work applies ORB-SLAM2 to a wearable navigation device for the visually impaired—a sighted person wears the device and walks to build a virtual guide path, which the visually impaired then follow. That study explains that the tracking thread localizes the camera by matching features against the local map, the local mapping thread inserts new points and optimizes by local BA, and the loop closing thread detects large loops and optimizes the pose graph.
- Persistent maps: other work adds map saving to the visual localization of a self-driving car so that a map once built can be stored and reused, the system shutting down the local mapping and loop closing threads in localization mode and running tracking alone.

What these extensions have in common is that they all keep ORB-SLAM2's three-thread skeleton and only replace or add modules within it. This shows the value of a good architecture—it solves not only the problem of the moment but becomes a

platform for later innovation. For a student undertaking a visual SLAM project, improving upon ORB-SLAM2 is a practical path with predecessors to follow.

Hands-On Activities

Activity 22-1 Implementing and Evaluating the ORB-SLAM2 System

Purpose: to compile and run ORB-SLAM2 in full, to evaluate its accuracy on a public dataset, to observe the three threads working in concert, and to experiment with the limits of the system. This is the most important implementation of Part VI and the foundation of Chapters 30 and 31.

Part One: Setting Up the Environment and Running a Dataset (everyone)

1. Install the dependencies (OpenCV, Eigen3, Pangolin) and compile ORB-SLAM2. If compilation errors arise, record the problem and the remedy—an indispensable training in practical engineering.
2. Download an easy sequence of the TUM RGB-D dataset (freiburg1_xyz or freiburg2_desk are recommended to begin with), run it in RGB-D mode, and watch the map and the trajectory in the Pangolin window.
3. Run the same sequence in monocular mode and compare the two: the time taken to initialize, the completeness of the trajectory, the density of the map points.
4. Observe and record: how many key frames did the system create? How many map points? What pattern do the key frames follow along the trajectory?

Part Two: Observing the Behavior of the Three Threads (everyone)

5. Observing initialization (monocular): record the number of frames and the distance of camera movement from start-up to successful initialization, against the principles of Section 22.2.
6. Observing the covisibility graph: watch the lines between key frames in the visualization, judge which key frames share many observations, and compare with the camera trajectory.
7. Observing loop closure: choose a sequence containing a loop (KITTI sequence 00, say), watch the process by which the trajectory and the map are pulled straight at the instant the loop closure fires, and record it on video.
8. A relocalization experiment: interrupt the image input deliberately for a few seconds while running (or choose a sequence containing an occlusion) and observe the whole process of losing tracking and relocalizing, reproducing Case 22-1.

Part Three: Evaluating Accuracy (university level)

9. Install the evo tool, compare the trajectory ORB-SLAM2 outputs with the dataset's ground truth, and compute the absolute trajectory error (ATE) and the relative pose error (RPE).
10. Plot the trajectory comparison and the error against time, find the segment of greatest error, then go back and look at the images of that segment and infer the cause (fast motion? weak texture? moving objects?)
11. A note for monocular evaluation: remember to perform the scale alignment (Sim3) first, or the error is meaningless. Compare the error figures with and without the scale alignment to understand scale ambiguity at first hand.

12. A comparison of the three modes: if the dataset supports it, run the same scene in monocular, stereo, and RGB-D in turn and compare the ATE and the computational load of the three, filling in a comparison table of your own.

Part Four: Parameter Experiments and Testing the Limits (university level)

13. A feature count experiment: set the number of ORB features extracted per frame to 500, 1000, and 2000 in turn and compare the changes in accuracy, frame rate, and number of map points, finding the value suited to your scene.
14. A weak texture challenge: record a video of your own containing a large expanse of white wall and observe where the system loses tracking, verifying the curse of the white wall of Chapters 9 and 20.
15. A fast motion challenge: record a video with quick turns, observe whether tracking fails, and consider how an IMU might help (a trailer for Chapter 23).

Discussion Questions

1. How many frames does initialization in monocular mode require? What happens if the camera only rotates purely? Which statements of Section 22.2 and Section 21.5 does this bear out?
2. How many key frames did the system cull? What would happen over a long run if the culling of redundant key frames were switched off?
3. At the instant of loop closure correction, which segments of the trajectory move the most? Why those segments? (Hint: spreading contradiction evenly, Chapter 12.)
4. By how much do the ATEs of the three modes differ? Is RGB-D necessarily the most accurate? Under what conditions would stereo win?

5. (University level) Raising the feature count from 500 to 2000 improves accuracy by how much? And lowers the frame rate by how much? What does this trade-off mean for a real-time system?

Extended Challenge (university level)

First: read the source of ORB-SLAM2's Tracking.cc, find where the three tracking strategies (the motion model, the reference key frame, relocalization) are implemented, add output messages at the key places, and observe while running under what conditions the system switches strategy and how often. This is the best exercise there is for putting the theory of this chapter beside real code. Second: try running ORB-SLAM2 with your own camera (a phone or a webcam)—first complete the camera calibration of Chapter 9, write the YAML configuration file, then record video or feed it live, and build a map of a classroom or corridor in your own department building. This result can serve directly as material for the project of Chapter 31.

Further Reading

1. R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo and RGB-D Cameras," IEEE T-RO, 2017. (The original paper on which this chapter is chiefly based.)
2. R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM: A Versatile and Accurate Monocular SLAM System," IEEE T-RO, 2015. (Its predecessor, the complete design of the monocular version.)
3. The ORB-SLAM2 open-source project (GitHub: raulmur/ORB_SLAM2). (Source code, compilation instructions, and examples of running on datasets.)

4. D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," IEEE T-RO, 2012. (DBoW2, the technical basis of loop closure and relocalization.)
5. H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," RSS, 2010. (The source of the covisibility graph and of Sim3 loop closure correction.)
6. Gao Xiang and Zhang Tao, Fourteen Lectures on Visual SLAM, Lecture 9 (The back end) and the practical chapters. (The best Chinese-language supplement for dissecting the ORB-SLAM system.)
7. The evo trajectory evaluation tool (GitHub: MichaelGrupp/evo). (Computation of ATE and RPE and plotting of trajectories; indispensable for evaluation.)

Summary

1. ORB-SLAM2 uses one and the same set of ORB features for all three tasks of tracking, mapping, and place recognition; ORB has good invariance to rotation, to scale, to the camera's automatic gain and exposure, and to changes of illumination, is fast to extract and match, and shows good precision and recall in bag-of-words place recognition.
2. Monocular initialization computes a homography (for a planar scene) and a fundamental matrix (for a non-planar one) in parallel and chooses between them by their relative scores; stereo and RGB-D have depth in the first frame and can map at once, and divide features by depth into reliable near points and far points that must accumulate parallax.
3. The tracking thread localizes the camera by minimizing the reprojection error with a motion-only BA, attempting in turn the motion model, the reference key

frame, and relocalization, and obtains a local map through the covisibility graph for further refinement; key frames are inserted by a loose policy.

4. The local mapping thread inserts the key frame into the covisibility graph and the spanning tree and builds its bag of words, culls poor map points, triangulates new points from connected key frames (which must pass five tests: the epipolar constraint, positive depth, parallax, the reprojection error, and consistency of scale), performs local BA, and culls redundant key frames.
5. The loop closing thread detects a revisit by comparing binary features with DBoW2, accepts the loop closure once geometric verification yields enough inliers, solves for a similarity transform (including scale correction for monocular), performs pose graph optimization on the essential graph, and launches a fourth thread for a global BA.
6. The core data structures are the key frame, the map point, the covisibility graph (linking key frames that observe map points in common), the minimum spanning tree, and the essential graph; the covisibility graph serves local operations and so supports large environments, while the pared-down essential graph serves the pose graph optimization of loop closure.
7. Implementation requires OpenCV, Eigen3, and Pangolin, together with accurate camera intrinsics and distortion coefficients supplied in YAML; the datasets commonly used are TUM RGB-D, KITTI, and EuRoC, evaluation uses ATE and RPE as its measures, and monocular requires a scale alignment first.

Carrying Forward

This chapter has assembled the parts of Chapters 20 and 21 into a system that really runs, and has measured several key numbers along the way: the five gates for a new

map point screen the candidates from 38,164 down to 28,543, raising correctness from 75 percent to 97.1 percent while killing only 3.2 percent of the correct matches by mistake; the nine-tenths rule for culling redundancy holds the key frames of three circuits at 31 rather than 137; and Figure 22-2 demonstrated something counterintuitive—with only one loop closure edge the trajectory did close (the endpoint error falling from 1.48 meters to 0.18) while the error over the whole trajectory improved only by half (ATE 0.604 against 0.333 meters); to remove the error in earnest one needs covisibility edges to carry the correction along the whole trajectory (ATE falling to 0.071 meters).

ORB-SLAM2 is the benchmark of maturity in visual SLAM, but it still has two definite limitations: its purely visual architecture leaves monocular with the scale problem and loses tracking easily under fast motion; and when tracking is severely lost and relocalization fails, the system can go no further. The ORB-SLAM3 of the next chapter was born for exactly this—a tightly coupled visual-inertial fusion founded on maximum a posteriori estimation (where the IMU preintegration of Chapter 7 makes its entrance), and a multiple-map system that opens a new map when tracking is lost and merges it seamlessly later. Read Chapter 23 with a question in mind: the covisibility graph of Figure 22-3 is a single graph, so how must this machinery change when the system holds several unconnected maps at once? The summit of Part VI is about to appear.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 23

ORB-SLAM3 and Visual-Inertial SLAM

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

The ORB-SLAM2 of Chapter 22 is already a complete system, but it has two hard faults: monocular has no absolute scale, and once tracking is severely lost and relocalization fails, the whole system ends there. The ORB-SLAM3 of this chapter was born against exactly these two—the first solved by the IMU, the second by multiple maps.

This is also the summit of Part VI and the meeting point of Part II with Part VI. Chapter 7 spent a whole chapter on the IMU: why it drifts, what bias is, why excitation of motion is needed before it can be initialized—every one of those conclusions is put to use in this chapter, and quantified.

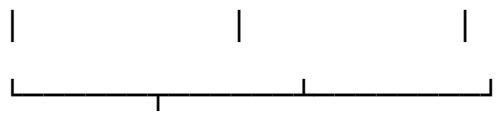
The three figures of this chapter answer three questions apiece: Figure 23-1 measures how long an IMU can hold out when vision fails, and the price of estimating the bias inaccurately; Figure 23-2 turns scale is unobservable under constant-velocity straight-line motion from a maxim into a condition number that can be computed; and Figure 23-4 compares the three fates after tracking is lost—stopping, opening a new map without merging, and Atlas's seamless merge.

Figure 23-0 The Link Map of Chapter 23

The Foundations Already Laid

Chapter 7 The IMU and preintegration Chapter 21 Epipolar geometry Chapter 22 The

ORB-SLAM2 architecture



▼

【Chapter 23 ORB-SLAM3: visual-inertial + multiple maps】

Tight coupling ─ IMU initialization ─ Atlas ─ relocalization and map merging

▼

Chapter 24 The vision-navigated robot (putting SLAM to work on a task)

▼

Part VII Drones and edge AI Part VIII ROS implementation

Learning Objectives

The ORB-SLAM2 of the last chapter is already the benchmark of maturity in visual SLAM, but it still has two limitations: its purely visual architecture leaves monocular with the scale problem and loses tracking easily under fast motion; and when tracking is severely lost and relocalization fails, the system can go no further. ORB-SLAM3 was born for exactly this. This chapter is the summit of Part VI and the culmination of the visual line of the whole book. After studying this chapter, students should be able to:

1. Explain the three core advances of ORB-SLAM3 over ORB-SLAM2.
2. Explain the meaning and the value of the four kinds of data association (short-term, mid-term, long-term, and multi-map).
3. Describe the architecture of monocular-inertial and stereo-inertial SLAM and the contributions the IMU makes.
4. Describe the three-stage procedure of IMU initialization founded on maximum a posteriori estimation.

5. Describe the composition of the Atlas multiple-map system and the two-stage strategy of map merging.
6. Describe the mechanism of relocalization and map merging after a loss of tracking.
7. Give examples of applications of visual-inertial SLAM in phones, drones, and wearable devices.

Core Concepts

Chinese Term	English Term	Explanation
視覺慣性 SLAM	Visual-Inertial SLAM	Simultaneous localization and mapping with camera and IMU tightly coupled.
多地圖系統	Atlas	An architecture maintaining several maps at once, one active and several inactive.
活躍地圖	Active Map	The map the tracking thread currently localizes in.
非活躍地圖	Inactive Map	A map in Atlas not in use for the time being but kept for a later merge.
短期資料關聯	Short-term	Matching map elements from the last few seconds.
中期資料關聯	Mid-term	Matching map elements near the camera that carry slight accumulated drift.
長期資料關聯	Long-term	Matching a previously visited region by place recognition, unaffected by accumulated drift.
多地圖資料關聯	Multi-map	Matching and using map elements from an earlier mapping session.
最大後驗估測	MAP Estimation	Seeking the most probable state estimate given the observations.

Chinese Term	English Term	Explanation
IMU 初始化	IMU Initialization	The start-up procedure that estimates scale, the direction of gravity, bias, and velocity.
地圖合併	Map Merging	The operation of fusing two maps seamlessly into one.
焊接視窗	Welding Window	The local range of key frames fused first during a map merge.
多階段作業	Multi-session	A mode of operation spanning several runs and reusing an earlier map.

23.1 The System Characteristics of ORB-SLAM3

ORB-SLAM3 was published in 2021 by Campos, Elvira, Rodríguez, Montiel, and Tardós, and it is the system already introduced in outline in Section 7.7. It is the culmination of the bloodline of feature-based SLAM, evolved by way of PTAM, ORB-SLAM, and ORB-SLAM2, and on its release it was the most complete and most accurate open-source SLAM library then available.

Three Core Advances

1. Tightly coupled visual-inertial SLAM: the first tightly coupled system founded on maximum a posteriori (MAP) estimation, with even the IMU initialization stage built upon MAP (Section 7.7). This solves the monocular scale problem and the question of robustness under fast motion.
2. A multiple-map system (Atlas): on losing tracking it no longer collapses but opens a new map and carries on, merging seamlessly when it later returns to the old region. This solves ORB-SLAM2's fatal weakness that losing tracking is the end.
3. Thoroughgoing use of four kinds of data association: this is the most fundamental architectural innovation. ORB-SLAM3 is the first system to use short-term, mid-term, long-term, and multi-map data association at every algorithmic stage.

The Sensor Configurations Supported

As Section 7.7 mentioned, ORB-SLAM3 supports an extremely wide range of configurations, an important source of its practical value:

- Camera types: monocular, stereo, RGB-D.

- Lens models: pinhole and fisheye (Chapter 9). Fisheye support is especially valuable for indoor wide-angle applications.
- With or without an IMU: every camera mode has a corresponding visual-inertial version.

Combining these, the system supports monocular, monocular-inertial, stereo, stereo-inertial, RGB-D, RGB-D-inertial, and other modes—one and the same codebase, with only a parameter changed. This makes it an excellent platform for research and teaching: students can compare the performance of different sensor configurations within exactly the same framework.

Accuracy

Section 7.7 has already quoted the key figures; here we add its performance in multi-session operation. In the EuRoC multi-session experiments, Atlas's map merging improves accuracy by more than a factor of two over CCM-SLAM and VINS-Mono; its advantage over VINS-Mono grows from 2.6 times in a single session to 3.2 times across sessions. On the whole, the paper's abstract claims an accuracy two to ten times better than previous methods; after its open-source release it became the standard basis of comparison for hundreds of later papers.

This phenomenon of winning already in a single session and winning by more across sessions shows the value of the multiple-map architecture plainly: a system that can reuse a map it built before is naturally more accurate. For robots actually deployed (running in the same site every day) this matters greatly.

23.2 Four Kinds of Data Association: the Spine of the Architecture

This section is the key to understanding the design philosophy of ORB-SLAM3. The authors divide data association—judging whether observations at different moments correspond to the same map element—into four levels, and argue that previous systems generally used only one or two of them.

Type	What is matched	Technical means	What it solves
Short-term	Map elements from the last few seconds	Projection matching	Basic frame-by-frame tracking
Mid-term	Elements near the camera carrying slight drift	Projection matching against the local map	Reduces the accumulation of drift and improves accuracy
Long-term	A previously visited region (however great the drift)	Place recognition (DBoW2)	Loop closure correction, relocalization
Multi-map	Map elements from an earlier mapping session	Place recognition + map merging	Reuse across sessions; recovery after a loss of tracking

Table 23-1 A Comparison of the Four Kinds of Data Association

Short-Term and Mid-Term: the Basis of Everyday Operation

Short-term and mid-term data association between a frame and the active map is found routinely by the tracking and mapping threads—projecting map points at the estimated camera pose and searching for a match within a window of only a few pixels in the image.

Mid-term association is an important source of ORB-SLAM3's advantage in

accuracy. Experiments show that in single-session monocular-inertial operation on the EuRoC dataset, ORB-SLAM3 is 2.6 times as accurate as VINS-Mono, which is credited to its ability to use mid-term data association.

Why does mid-term association matter so much? Because many visual odometry systems perform only short-term association (comparing only with the last few frames), so that once a map point leaves the field of view for a time and comes back, the system treats it as a new point—which wastes information and lets drift go on accumulating. ORB-SLAM3 can rematch these old points that are still nearby but somewhat drifted, which amounts to making small corrections continually.

Long-Term and Multi-Map: Recovery and Reuse

For the long-term data association required by relocalization and loop closure detection, ORB-SLAM uses the DBoW2 bag-of-words place recognition system, an approach adopted by most recent VO and SLAM systems as well.

Multi-map association is the fourth level, peculiar to ORB-SLAM3. It lets the system match map elements from different mapping sessions—whether a new map opened after a loss of tracking within one run, or a map built and saved yesterday. This opens the possibility of multi-session operation.

A Summary of the Idea

These four levels can be understood through an analogy with memory: short-term is what I just saw, mid-term what is familiar hereabouts, long-term where I have been before, and multi-map what I experienced on another occasion. A good SLAM system should make good use of every level of memory, as a person does—and that is the design philosophy of ORB-SLAM3.

23.3 Monocular / Stereo Visual-Inertial SLAM

Section 7.6 has already explained the four great contributions of the IMU in visual-inertial SLAM (absolute scale, the direction of gravity, filling the gaps where vision fails, and high-rate output); this section focuses on how ORB-SLAM3 realizes tight coupling.

Why Tight Coupling Is Needed

An IMU measures angular rate and linear acceleration but drifts appreciably, so that double integration of IMU data alone cannot be used for long-term pose estimation. That is the central conclusion of Section 7.3.

Conversely, pure vision fails under fast motion, weak texture, and violent changes of illumination. The two must therefore be fused—and as Section 7.6 described, tight coupling (putting the IMU preintegration constraint and the visual reprojection error into one and the same optimization problem) is appreciably more accurate than loose coupling.

How ORB-SLAM3 Realizes Tight Coupling

All three threads of the system are extended on account of the IMU:

- The tracking thread: besides minimizing the reprojection error, it adds the inertial error term of the IMU preintegration (Section 7.5) and estimates pose, velocity, and IMU bias together.
- The local mapping thread: the local mapping module adds key frames and map points to the current active map, removes redundant frames, and optimizes the map over the key frames neighboring the current one by a visual or a visual-inertial bundle adjustment.

- The loop closing and map merging thread: in inertial mode, loop closure correction must also handle the consistency of the direction of gravity and of scale.

The Extension of the State Vector

Against pure vision, visual-inertial SLAM has more to estimate for each key frame:

- Pure vision: pose (rotation R, translation t), 6 degrees of freedom in all.
- Visual-inertial: pose + velocity + accelerometer bias + gyroscope bias, 15 degrees of freedom in all.

More state means more computation, but what it buys is absolute scale (monocular no longer has a scale ambiguity), a known direction of gravity (the map is leveled automatically and roll and pitch become observable), and the ability to keep tracking for several seconds when vision briefly fails. In the great majority of applications this is an excellent bargain.

Stereo-Inertial: the Most Robust Configuration

If cost and bulk are no object, stereo-inertial is ORB-SLAM3's most robust configuration: the stereo pair supplies absolute scale and single-frame depth (with no waiting for parallax to accumulate) and the IMU supplies high-rate motion and robustness under fast motion. The EuRoC accuracy of 3.5 centimeters quoted in Section 7.7 is the result of the stereo-inertial mode. For a campus inspection robot or a drone project, this is the configuration of first choice where conditions permit.

Measuring It: How Long Can the IMU Actually Hold Out

Section 23.3 says that tight coupling can keep tracking for several seconds when vision briefly fails. How many is several? This subsection computes it, and answers a more

important question along the way: what does that ability depend on?

The experimental setting: let the body undergo a motion containing accelerations, decelerations, and rotations, with the IMU putting out specific force readings at 200 Hz (with bias and noise). From some moment on, vision is assumed to fail completely for τ seconds, during which the position is reckoned by double integration of the IMU alone, and at the end the reckoned position is compared with the true one. The key variable is how accurately the system has estimated the accelerometer bias, since the estimated bias must be subtracted before reckoning.

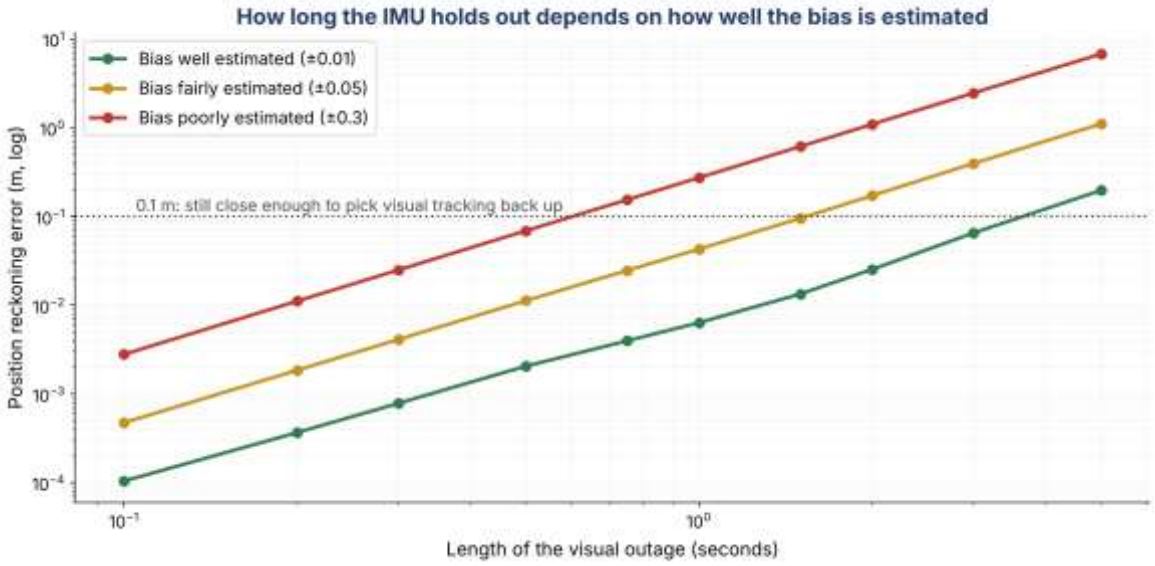


Figure 23-1 How Long the IMU Holds Out Depends on How Well the Bias Is Estimated

All three lines grow quadratically—the inevitable result of Section 7.3: an error in acceleration integrated twice becomes an error in position, so the error accumulates with the square of time. The real difference lies in the vertical distance between the three lines.

If the bias is estimated to the level of ±0.01 meters per second squared (the green line), the position error after 1 second of visual failure is 0.006 meters, after 2 seconds 0.025

meters, and even after 5 seconds only 0.20 meters. This accuracy is enough for the system to project the map points back into the frame and pick up again the moment vision returns—the technical basis of the user hardly noticing the interruption at all.

If the bias is estimated only to ±0.3 meters per second squared (the red line), the same 1-second error is 0.28 meters, the 2-second error 1.10 meters, and the 5-second error 6.82 meters. Against the green line, the difference at 2 seconds is fortyfold. An error of 1.10 meters means that reprojection will not line up with the original map points at all, and the system can only fall back on relocalization, or lose tracking outright.

These figures explain something: the IMU in visual-inertial SLAM is not simply a backup sensor. Whether it can save the situation depends on whether the bias has been estimated continually and correctly—and the estimate of the bias depends in turn on the positional constraint vision supplies. The two depend on each other, which is the fundamental reason Section 23.3 stresses tight and not loose coupling: attach the IMU as an independent module and the bias will never be estimated accurately.

Why Bias Matters Far More Than Noise

The random noise of an IMU is zero-mean, and integration cancels part of it, so the error grows roughly as $\sqrt{t}$. But bias is a constant offset, and after double integration the error grows as t^2—the two are not of the same order at all.

Take the numbers of this experiment: the accelerometer noise is 0.02 meters per second squared, while an uncalibrated bias may reach 0.1 meters per second squared or more. After five seconds the error contributed by the noise is under a few centimeters, while that contributed by the bias is on the order of meters.

So when students are tuning a visual-inertial system and find that the position drifts away slowly while standing still, what to check is the bias estimate and not the noise parameters.

23.4 IMU Initialization and Preintegration

Section 7.7 has introduced the difficulty of IMU initialization and the necessity of excitation of motion; this section goes deeper into its three-stage procedure—one of ORB-SLAM3's most important technical contributions.

What Initialization Must Solve For

At start-up a number of mutually coupled unknowns must be determined together:

- Scale: the true size of the map in monocular mode (known for stereo and RGB-D, which are therefore simpler).
- The direction of gravity: which way is down, so that gravity can be subtracted from the accelerometer readings (the idea of specific force in Section 7.1).
- IMU bias: the offsets of the accelerometer and the gyroscope (Section 7.3).
- Initial velocity: the velocity at each key frame.

The Three-Stage Procedure

ORB-SLAM3 splits initialization into three stages that converge by degrees:

4. Vision-only MAP estimation: first build a short stretch of map and trajectory from vision alone (of unknown scale in monocular mode). This provides the motion reference the later inertial estimate requires.

5. Inertial-only MAP estimation: hold the visual trajectory fixed and use the IMU measurements alone to estimate scale, the direction of gravity, bias, and velocity. With the visual trajectory already settled, this subproblem becomes relatively simple.
6. Joint visual-inertial BA: finally put all the variables (poses, velocities, biases, scale, gravity, map points) into one optimization problem for a joint refinement.

This strategy of solving separately first and refining jointly afterward greatly improves both the speed and the reliability of initialization.

The Performance of Initialization

IMU initialization reaches a 5 percent scale error within 2 seconds, and about 1 percent after refinement. Against the earlier systems mentioned in Section 7.7 (ORB-SLAM-VI requiring about 15 seconds) this is an enormous advance—2 seconds is all but imperceptible to a user, while 15 is plainly trying.

A Note on Implementation: Excitation of Motion

Let us stress again the key reminder of Section 7.7: IMU initialization requires sufficient excitation of motion. If the body moves only at constant velocity in a straight line, or not at all, the accelerometer reads almost nothing but gravity, and the system cannot separate scale from bias (they are mathematically unobservable).

In practice: after start-up, hold the device and move it for a few seconds with accelerations, decelerations, and rotations (small movements forward and back and side to side, with some turning), and initialization will complete smoothly. Together with pure rotation cannot triangulate of Chapter 21, this is one of the two great implementation traps of visual-inertial SLAM—students must remember it when they

come to implement Chapter 30.

Measuring It: Why You Really Must Shake It About

Section 23.4 says that if the body moves only at constant velocity in a straight line the system cannot separate scale from bias (they are mathematically unobservable). That statement matters, but unobservable sounds like an abstract word. This subsection turns it into a number that can be computed.

The method is to write the second stage of initialization (the visual trajectory held fixed, inertial estimation alone) as a system of linear equations. The unknowns are the velocity at each key frame, the gravity vector, the scale factor, and the accelerometer bias. The equations come from the preintegration relations between neighboring key frames—one set from position and one from velocity. Take the singular value decomposition of the coefficient matrix of this system, and the ratio of the largest to the smallest singular value (the condition number) tells us directly how well the problem can be solved.

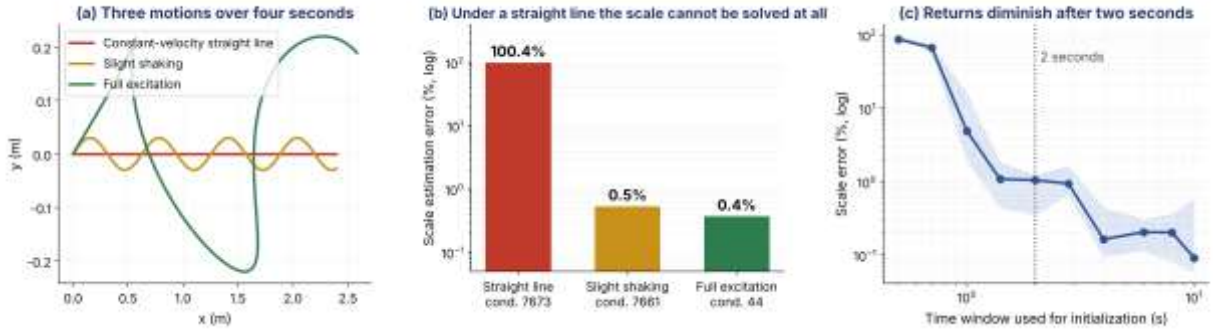


Figure 23-2 Excitation of Motion Decides Whether Initialization Can Be Solved at All

Panel (a) draws three motions over four seconds: constant-velocity straight line (red), slight shaking (amber, swaying about 3 centimeters sideways), and full excitation (green, with acceleration and deceleration fore and aft, lateral and vertical movement, and rotation in yaw and pitch). Note that all three trajectories are visibly moving—from

the trajectory alone it is hard to foresee that their initialization results will differ so greatly.

Panel (b) is the result, with the vertical axis logarithmic. The scale error of the constant-velocity straight line is 100.4 percent—not inaccurate but entirely insoluble; the scale actually solved for may even come out negative. The condition number reaches 7,673. Slight shaking brings the scale error down to 0.54 percent and full excitation to 0.37 percent, with the condition number falling to 44, a hundred and seventy times smaller than for the straight line.

Why does a constant-velocity straight line fail completely? The reason is quite plain. What an accelerometer measures is specific force, that is, the projection of true acceleration minus gravity into the body frame. Under constant-velocity straight-line motion the true acceleration is zero and the attitude does not change, so the accelerometer always reads the same constant vector. That constant can be explained as the projection of gravity, or equally as a bias, and the two explanations are mathematically identical—the system of equations simply has not enough information to separate them.

One further detail is worth noting here: slight shaking rescues the scale (0.54 percent), yet its condition number is still 7,661, as high as for the straight line, and its accelerometer bias error is in fact the largest of the three. In other words, a slight shake suffices to make scale observable but not to separate out the bias. This answers exactly to the conclusion of Figure 23-1—estimate the bias inaccurately and the IMU cannot carry you across a gap in vision.

Panel (c) answers another question: how long initialization takes. Holding the motion

at full excitation, the time window is swept from 0.5 seconds to 10. At 0.5 seconds the scale error is 87.9 percent—which is to say unsolved; at 1 second it falls at a stroke to 4.95 percent, at 1.4 seconds to 1.09 percent and at 2 seconds to 1.05 percent, after which the curve flattens markedly—only at 10 seconds does it reach 0.09 percent. A second and a half or so, that is, is the point of diminishing returns: waiting longer improves the accuracy little while consuming the user's patience linearly. This is why reaching a 5 percent scale error within 2 seconds, quoted in Section 23.4, is regarded as an important advance.

How This Experiment Differs from a Real System

To be fully reproducible, this experiment makes several simplifications: there is no gyroscope bias, the noise of the visual trajectory is independent and identically distributed, and the sensitivity of the preintegration to bias is taken only to first order. The absolute numbers here (0.7 percent at 2 seconds, say) are therefore optimistic against a real system.

A real system is at about 5 percent at 2 seconds and is refined to about 1 percent by the joint visual-inertial BA of the third stage. But the three qualitative conclusions agree entirely: a constant-velocity straight line is unobservable, a shake suffices to rescue the scale, and returns diminish after two seconds.

Should students redo this experiment on their own device, the most valuable product is not the absolute numbers but how much and how long your device must be shaken—which goes straight into your operating manual.

23.5 The Atlas Multiple-Map System

This is the second core innovation of ORB-SLAM3, and the design that best embodies engineering wisdom.

The Composition of Atlas

Atlas is a multi-submap system made up of a set of discrete maps. It maintains one active map, in which the tracking thread localizes the current frame and which the local mapping thread continually optimizes and updates with new key frames; the other maps in Atlas are inactive. The system also builds a database of key frame information founded on the bag-of-words model, used for relocalization, loop closure detection, and map merging.

- The active map: the one in use, on which tracking and local mapping both operate.
- An inactive map: one built earlier and not currently in use, but kept whole in memory awaiting a later merge.
- A shared bag-of-words database: the key frames of every map are registered in one and the same database—the key to being able to compare across maps.

What Happens When Tracking Is Lost

This is the very reason Atlas exists. Recall ORB-SLAM2's position: when tracking is severely lost and relocalization fails, the system simply stops there and all later data is wasted.

ORB-SLAM3's practice is this: if relocalization does not succeed shortly after tracking is lost, set the current map inactive, open a brand new map, and carry on mapping and tracking from nothing. The user hardly notices the interruption. And when the robot later walks back into a region covered by the old map, place recognition

detects a cross-map match and the system merges the two maps at once, restoring a single complete and consistent map.

How It Differs from Other Multiple-Map Schemes

It is worth noting that other systems have proposed multiple-map extensions of ORB-SLAM2, but they keep the submaps as separate entities; ORB-SLAM3 performs a seamless merge and builds a more accurate global map.

This difference is crucial. Merely storing maps separately without merging them loses no data, but you end up with several mutually unrelated maps, unable to plan a path across regions and unable to let their accuracies improve one another. Only a seamless merge realizes the true value of multiple maps.

The Value of Multi-Session Operation

Another important application of Atlas is reuse across runs: save the map built today, load it on starting up tomorrow, and the system can relocalize directly within the old map and go on extending it. This is extremely practical for service robots and inspection robots that run in the same site every day, and it explains the greater multi-session advantage in accuracy mentioned in Section 23.1.

23.6 Relocalization and Map Fusion After a Loss of Tracking

This section describes the technical details of how Atlas operates: how a cross-map association is detected, and how the merge is performed.

Place Recognition: a Design for High Recall

Whenever the mapping thread creates a new key frame, place recognition starts and attempts to detect a match with any existing key frame in Atlas. If the matching key

frame found belongs to the active map, a loop closure is performed; otherwise it is a multi-map data association, and the active map and the matching map are merged.

This is an elegant unification: loop closure and map merging are in fact two outcomes of one mechanism—the only difference being whether the matched key frame belongs to the same map or a different one.

The Trade-off Between Precision and Recall

Place recognition faces a classic dilemma:

- Recall must be high: every true match must be found if possible, or opportunities for loop closure and merging are missed.
- Precision must be a hundred percent: one wrong match drags the map bodily into a wrong position and causes a catastrophe (as Chapters 12, 15, and 22 have stressed repeatedly).

ORB-SLAM3's practice is this: for high recall, query the DBoW2 database for several similar key frames in Atlas for every new active key frame; and for a hundred percent precision, put every candidate through several steps of geometric verification.

Geometric verification is performed by a direct Sim(3) or SE(3) alignment (using RANSAC and Horn's method). Here again we see the shadow of RANSAC from Chapter 21, and the reason monocular requires a similarity transform (including scale) from Chapter 22.

The Two-Stage Strategy of Map Merging

Once a cross-map association is confirmed the merge is to begin. But there is an engineering difficulty here: the inactive map may be very large, and merging it all at once would block the system with computation.

ORB-SLAM3's remedy is to split the merge into two steps: first, perform the merge within a welding window defined by the matched key frames and their neighbors; and in the second stage, propagate the correction to the rest of the merged map by pose graph optimization.

Figure 23-3 The Two-Stage Strategy of Map Merging

A cross-map association is detected: key frame Ka of the active map Ma ↔ key frame Km of the inactive map Mm

↓ estimate the aligning transformation Tam

【Stage one: the welding window】

Perform the merge within the local window formed by the neighbors of Ka and Km

Search densely for mid-term data associations within this window to improve accuracy

→ locally usable at once; the tracking thread is not interrupted

↓

【Stage two: propagating the correction】

Propagate the correction to the rest of the merged map by pose graph optimization

→ global consistency is restored

This strategy of welding locally first and propagating globally afterward lets the merge proceed in real time without blocking tracking. Students should notice that it

agrees entirely with the architectural thinking of Cartographer's local in real time, global in the background in Chapter 15 and LOAM's high-rate odometry, low-rate mapping in Chapter 18—a universal pattern in the design of robotic systems.

A Detail: Strengthening Mid-Term Association

The authors' second innovation is this: once the relative pose between the new key frame and the matched map has been estimated, a local window is defined by the matched key frame and its neighbors in the covisibility graph, and mid-term data associations are searched for densely within this window to improve accuracy.

This echoes once more the spine of Section 23.2: ORB-SLAM3's advantage in accuracy comes in large measure from using mid-term data association wherever it can, at every point in the system.

Measuring It: Three Fates After Tracking Is Lost

Sections 23.5 and 23.6 described how Atlas operates. This subsection quantifies its value with a simulation: for one and the same trajectory and one and the same loss of tracking, what does each of three system designs end up with?

The setting is a robot circling indoors twice over 120 key frames, with vision failing completely between frames 66 and 78 (through a fast turn or facing a white wall, say). The three cases are: ORB-SLAM2's practice (stopping after relocalization fails), opening a new map without merging, and Atlas's seamless merge. Two measures compare them at the end: how much of the trajectory was covered, and the root mean square error of the final map.

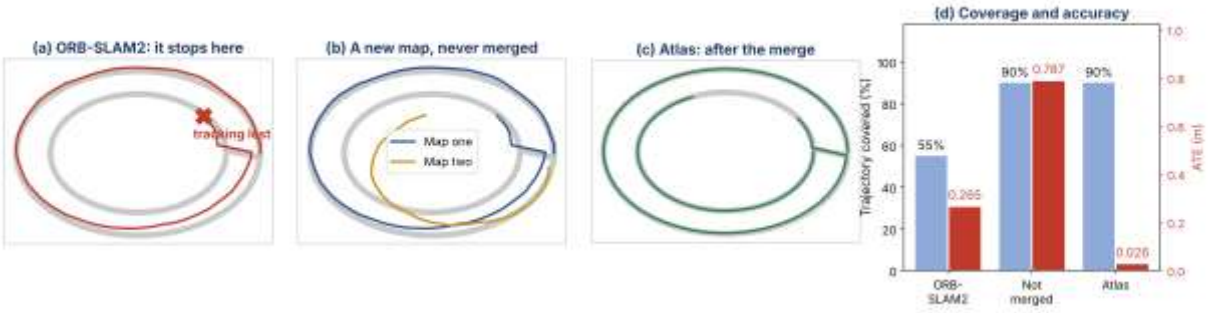


Figure 23-4 Three Fates After Tracking Is Lost

Panel (a) is ORB-SLAM2's position. The system loses tracking at frame 66, fails to relocalize, and stops there, covering only 55 percent of the trajectory while everything after is wasted. The ATE of the part already built is 0.265 meters—and note that this error is not small, because it never had the chance to be corrected by a loop closure on the second circuit.

Panel (b) is opening a new map without merging. Coverage rises to 90 percent and no data is indeed lost. But the two maps each have their own origin, and viewed in a common coordinate frame the overall ATE is 0.787 meters—three times worse than building only half. The blue and orange segments in the figure are plainly out of register, which is the visual form of unable to plan a path across regions in Section 23.5.

Panel (c) is Atlas. When the robot walks back into the region covered by the first map, place recognition detects a cross-map match and the system merges the two into one. Coverage is likewise 90 percent, but the ATE falls to 0.026 meters—one thirtieth of the unmerged case, and ten times better than ORB-SLAM2's half-built map. In the figure the merged green trajectory very nearly coincides with the true one, leaving a gap only over the stretch where tracking was lost.

Panel (d) sets the two measures side by side. The point this figure makes is that the value of Atlas is not only that no data is lost but that the data can correct one another.

The second map can be pulled back into the right position only by virtue of its cross-map covisibility with the first—exactly the conclusion of Figure 22-2 in Chapter 22: error can be redistributed only through the connectivity of the graph. Multiple maps that are never merged are merely data put into storage.

Why Merging Is So Much Harder Than Keeping

Saving the data after a loss of tracking as a new map is easy enough in engineering—just open a new container. What is really difficult is the merge: the two maps have different coordinate frames, their scales may differ (in monocular), and the merge must not block the tracking thread that is running.

That is the reason for the two-stage strategy of Section 23.6: fuse locally within the welding window first (a small range, completable in real time), then propagate the correction outward along the pose graph (a large range, done in the background). Again, pull straight quickly first and optimize finely afterward—the fourth appearance of this pattern in the book.

23.7 Visual-Inertial SLAM on the GoPro, the Phone, and the Drone

Visual-inertial SLAM is no longer merely a research topic; it is running in the everyday devices of you and me. This section describes several representative applications.

Phones and AR / VR

Section 20.7 mentioned AR's inside-out tracking, which technically is nothing other than visual-inertial SLAM. A phone's camera together with its built-in IMU (Chapter 7) makes a perfect visual-inertial combination: the camera supplies a reference to the

environment that does not drift, and the IMU supplies high-rate motion and robustness when the head turns quickly.

One particularly persuasive figure in the ORB-SLAM3 paper bears on applications of this kind. On the fisheye TUM-VI benchmark it performs better overall than VINS-Mono and Basalt; in the room sequences, which resemble AR / VR use, the mean absolute trajectory error is 0.009 meters in stereo-inertial mode and 0.011 meters in monocular-inertial.

Millimeter-level accuracy—precisely the level needed to pin a virtual object stably to a real tabletop. Students may consider: that AR ruler on your phone, able to measure the length of a table, rests on technology of this grade.

Wearable Cameras and Action Photography

An action camera (a GoPro, say) is worn on a helmet or on the chest and records while cycling, running, or climbing. Video of this kind is extremely challenging for SLAM: violent vibration, quick turns, motion blur, and strong changes of illumination (in and out of tree shade or a tunnel).

Pure visual methods are all but bound to lose tracking on such sequences, while visual-inertial SLAM, with the IMU holding out for several seconds when vision fails (the third contribution in Section 7.6), can maintain a continuous trajectory. Typical uses of such applications include the three-dimensional reconstruction of a sporting route, the analysis of trajectories in extreme sports, and the backpack spatial surveying mentioned in Part V.

Drones

The drone is one of the most important applications of visual-inertial SLAM and

the theme of Chapter 26. Its particular requirements include:

- Extremely constrained weight and power: a camera plus an IMU is the only workable combination (Section 9.5).
- Fast motion in six degrees of freedom: hard for pure vision to cope with, and the IMU is indispensable.
- GPS-denied environments: indoors, under a bridge, in a tunnel, visual-inertial is the only means of localization (Case 5-3 of Chapter 5).
- Stringent real-time requirements: a delay in localization bears directly on the stability and the safety of flight.

The EuRoC dataset mentioned in Case 7-3 of Chapter 7 was recorded with a micro air vehicle, and ORB-SLAM3's excellent performance on it (the 3.5-centimeter accuracy of Section 7.7) shows its practical value in drone applications.

Robots and the Campus Site

For readers of this book, several concrete project directions are worth considering:

- A vision-navigated robot: build a campus map with stereo-inertial ORB-SLAM3, then combine it with the visual navigation of Chapter 24 and the semantic understanding of Chapter 32.
- Multi-session mapping: use Atlas's multi-session capability to map a whole department building over several runs and verify the effect of map merging.
- A recovery-from-loss experiment: design an experimental procedure that loses tracking deliberately, and quantify Atlas's advantage over ORB-SLAM2 in the ability to recover—an excellent topic for a comparative study.

- A comparison with LiDAR SLAM: run ORB-SLAM3 (visual-inertial) and the LIO system of Chapter 18 (LiDAR-inertial) in the same site and compare their performance in different environments (a bright corridor, a dim machine room, a glass-curtained lobby). This comparison lets students feel deeply the complementarity of sensors stressed repeatedly in Chapters 8 and 9.

At this point the core systems of Part VI have been introduced in full. In the next chapter we widen our view from localization and mapping to navigation and application—how does a vision-navigated robot use these capabilities to carry out a task? Line following, visual servoing, AprilTag navigation, YOLO semantic navigation, and even VLM-assisted navigation will connect the technology of this book to real robotic tasks.

23.8 After Visual-Inertial: Two Directions in 2024–2026

ORB-SLAM3 was published in 2021 and is to this day the standard object of comparison in visual-inertial SLAM. Progress in the past few years has run mainly in two directions, and they are nearly opposite.

One: Downward, Squeezing onto a Milliwatt-Class Chip

The first direction shrinks visual-inertial odometry to the limit. A work of 2026 deploys a complete monocular visual-inertial pipeline (ORB features, the eight-point algorithm and EPnP with RANSAC, IMU preintegration, sliding-window pose graph optimization) onto a RISC-V ultra-low-power chip clocked at only 370 MHz.

The result is 20 frames a second, under 100 milliwatts of power, and less than 1 MB of memory (about 110 KB in the first level and about 850 KB in the second). Put these figures together and they are quite striking—a complete visual-inertial SLAM running

on a chip two orders of magnitude weaker than a phone.

But the price is stated honestly as well. That system does without loop closure detection, and on the standard EuRoC dataset its mean trajectory error is about 3.46 meters against ORB-SLAM3's 0.25 or so—an order of magnitude apart. The authors themselves point out that the reason is precisely the lack of global correction. This comparison belongs at the end of this chapter: it shows that loop closure and global optimization are not a flourish but the sole source of long-term consistency. The milliwatt-class platforms of Chapter 25 return to this theme.

Two: Upward, Letting Learned Modules Take Over the Front End

The second direction hands certain parts of the front end to neural networks: learned features and matching in place of ORB, a learned optical flow consistency check in place of purely geometric outlier rejection, or learned line features added in scenes of violently changing illumination.

These works have one thing in common worth the student's notice: what they replace is almost always the layer of how correspondences are found, while IMU preintegration, tightly coupled optimization, key frames, and map management—the whole of Sections 23.3 through 23.6 of this chapter—are mostly left untouched. This is exactly the phenomenon observed in Section 22.8 of Chapter 22.

	ORB-SLAM3 (this chapter)	Milliwatt-class VIO	VIO with a learned front end
Front end	ORB + RANSAC	ORB + RANSAC (pared down)	Learned features / optical flow
IMU fusion	Tightly coupled,	Tightly coupled	Tightly coupled

	with bias estimation	(sliding window)	
Loop closure and multiple maps	Yes	No	Mostly retained
Typical hardware	A desktop machine or a Jetson	RISC-V, 370 MHz	A GPU is required
Power	A few watts to tens of watts	Under 100 milliwatts	Tens of watts
Mean EuRoC error	About 0.25 meters	About 3.46 meters	Comparable to ORB-SLAM3
Where it applies	General robots and AR	Micro drones, wearables	Scenes of violently changing illumination and weak texture

Table 23-2 A Comparison of the Three Routes

From this table one may read a judgment quite useful in choosing a project: if your platform has a power budget of a few watts, ORB-SLAM3 remains the least troublesome and most reliable choice; if you are working on a gram-scale miniature platform, what you need is the technology of Chapter 25, and you must accept the consequences of having no loop closure; and if the lighting conditions of your site are poor, a learned front end is worth trying, though the system architecture remains the one in this chapter.

Formulas and Procedures

One: the State Vector of Visual-Inertial SLAM

In visual-inertial mode, the state to be estimated at each key frame i is

$$S_i = \{ R_i , p_i , v_i , b_g_i , b_a_i \}$$

where R_i is rotation and p_i position (6 degrees of freedom together), v_i velocity (3), and b_g and b_a the gyroscope and accelerometer biases (3 each), for 15 degrees of freedom in all—velocity and bias beyond the 6 of pure vision. Monocular-inertial mode must additionally estimate the scale s and the direction of gravity R_wg during initialization.

Two: the Joint Objective of Tight Coupling

ORB-SLAM3's visual-inertial bundle adjustment minimizes errors of both kinds, visual and inertial, at once:

$$min \Sigma \rho(|| e_visual ||^2_\Sigma) + \Sigma || e_inertial ||^2_\Sigma + || e_prior ||^2$$

- The visual error e_visual: the reprojection error (Chapters 21 and 22), requiring the projected position of a map point to agree with the actual observation.
- The inertial error e_inertial: the IMU preintegration residual (Section 7.5), requiring the changes of pose and velocity between neighboring key frames to agree with the IMU measurements.
- The prior error e_prior: retains the information accumulated by earlier optimizations.

ρ is a robust kernel. This expression is the concrete form of tight coupling in Section 7.6, and another application of the least squares framework of Chapter 12—by now students should be able to see that the mathematics of this whole book is highly unified.

Three: the Three Stages of IMU Initialization

Figure 23-5 The IMU Initialization Procedure

① Vision-only MAP estimation

Build a short stretch of map and trajectory with the camera alone (monocular scale unknown)

↓ supplies the motion reference

② Inertial-only MAP estimation (the visual trajectory held fixed)

Estimate: the scale s, the direction of gravity, the gyroscope bias, the accelerometer bias, and the velocity at each frame

↓ obtains initial values for the inertial parameters

③ Joint visual-inertial bundle adjustment

All variables (poses, velocities, biases, scale, gravity, map points) refined together

↓

Complete: a scale error of about 5 percent within 2 seconds, about 1 percent after refinement

【Precondition】there must be sufficient excitation of motion, or scale and bias are unobservable

Four: the Unified Flow of Loop Closure and Map Merging

Figure 23-6 The Two Outcomes Place Recognition May Trigger

The mapping thread creates a new key frame

↓

Place recognition: query the DBoW2 database for similar key frames in Atlas (high recall)

↓

Geometric verification: multi-step verification + Sim(3) / SE(3) alignment (RANSAC + Horn's method)

↓ (seeking 100 percent precision)

Which map does the matched frame belong to?

├─ The active map → 【loop closure】pose graph optimization + global BA (Chapter 22)

└─ Another map → 【map merging】fusion in the welding window + propagation by pose graph

Cases

Case 23-1 Losing Tracking Is No Longer the End: Atlas in Practice

This is the most dramatic experiment for contrasting ORB-SLAM2 with ORB-SLAM3, and demonstrating the two side by side in class is strongly recommended.

The design of the experiment is simple: run both systems on the same video, a video containing in the middle a severe visual failure—the camera completely covered for ten seconds, say, or swung quickly across a great expanse of white wall (the curse of the white wall of Chapter 9).

ORB-SLAM2's behavior: after losing tracking it attempts to relocalize, fails because it has strayed too far from the original region, and comes to a standstill. The display freezes and the rest of the video goes entirely unused—even if splendid scenes follow, all of it is wasted.

ORB-SLAM3's behavior: after losing tracking it tries briefly to relocalize, and on failing it opens a new map at once and goes on mapping and tracking normally, the user hardly noticing the interruption. And when the camera later circles back to the old region, place recognition detects the cross-map association and the two maps merge seamlessly—the screen showing the striking picture of two trajectories, once separate, suddenly joined.

This contrast lets students understand intuitively the value of robustness in a real system. On a clean dataset in the laboratory the difference in accuracy between the two may be only a number; but in challenging scenes in the real world, the difference is between usable and not. It also echoes the observation about Cartographer in Chapter 15: in a productized system, robustness often matters more than the last word in accuracy.

Case 23-2 The Value of Two Seconds: Why the Speed of Initialization Is Decisive

IMU initialization shortening from fifteen seconds to two sounds like being merely a little faster. But those thirteen seconds are decisive in practice.

Imagine a user opening an AR application on a phone. If the phone must be waved about for fifteen seconds before it can be used, the great majority of people will close the app by the fifth second—such a product could never be brought to market. A two-

second initialization, by contrast, finishes while the user is still adjusting their grip, all but imperceptibly.

The same holds for robots. A drone that must be shaken about on the ground for fifteen seconds before takeoff not only wastes time but is simply impracticable in some situations (an emergency rescue, say); a two-second initialization can be completed incidentally while taxiing for takeoff.

Looked at more deeply, fast initialization brings an indirect benefit as well: when the system loses tracking and must initialize afresh on a new map (Section 23.5), fast initialization means a very short interruption. Were initialization to take fifteen seconds, Atlas's seamless switch would not deserve the name. This shows how the various techniques in a system design support one another—the speed of initialization, the multiple-map architecture, and data association together constitute ORB-SLAM3's robustness.

The lesson for students is this: in assessing a technical improvement, do not look only at the change in a single number but ask what this change makes possible that was impossible before—that is the true measure of technical value.

Case 23-3 From PTAM to ORB-SLAM3: Twenty Years of One Bloodline

ORB-SLAM3 did not appear out of nowhere; it is the crystallization of twenty years of evolution in the technical bloodline of feature-based visual SLAM. It is the culmination of that bloodline, which began with PTAM and ORB-SLAM, and on its release it was the most complete and most accurate open-source SLAM library then available.

- PTAM (2007): first created the architecture of separating tracking and mapping into two threads, making real-time visual SLAM possible. That two-thread idea is the basis of every later system.
- ORB-SLAM (2015): unified the three tasks of tracking, mapping, and place recognition with the ORB feature (Section 22.1), added a loop closing thread, and introduced the covisibility graph and the essential graph (Section 22.6).
- ORB-SLAM2 (2017): extended to stereo and RGB-D and became the most widely used open-source visual SLAM (Chapter 22).
- ORB-SLAM3 (2021): added tightly coupled visual-inertial estimation and the Atlas multiple-map system, and used all four kinds of data association throughout.

Watching this bloodline, students can learn an important research methodology: no generation overturns the one before it; each keeps the architecture that works and makes its advance against a clearly identified weakness. PTAM's two threads, ORB-SLAM's covisibility graph and bag of words, ORB-SLAM2's support for several cameras—all of these are still present in ORB-SLAM3.

This also gives students bent on research a practical piece of advice: rather than pursuing the invention of something wholly new, understand one mature system deeply, identify its definite weakness, and propose a targeted improvement. ORB-SLAM3 could propose Atlas precisely because its authors knew intimately the pain of ORB-SLAM2's losing tracking is the end. Understand a limitation, and only then can you break through it.

Hands-On Activities

Activity 23-1 Implementing Visual-Inertial SLAM with ORB-SLAM3

Purpose: to run the several modes of ORB-SLAM3, to experience at first hand the difference the IMU makes and Atlas's ability to recover from a loss of tracking, and to complete a comparison with ORB-SLAM2. This is the concluding implementation of Part VI and an important foundation for Chapters 30 and 31.

Part One: Setting Up the Environment and Basic Operation (everyone)

1. Compile ORB-SLAM3 (the dependencies are similar to those of Chapter 22: OpenCV, Eigen3, Pangolin; mind the question of version compatibility).
2. Download a sequence of the EuRoC dataset (MH_01_easy is recommended to begin with), run it first in pure stereo mode, and record the trajectory and the map.
3. Run the same sequence in stereo-inertial mode and compare the two in trajectory, number of map points, and stability of the system.
4. Observe initialization: in monocular-inertial mode, record the time and the number of image frames from start-up to the completion of IMU initialization, against the two seconds of Section 23.4.

Part Two: the Value of the IMU (everyone, or university level)

5. A challenging sequence: choose a difficult sequence of fast motion (EuRoC's V1_03_difficult or MH_05_difficult, say) and run it in pure visual and in visual-inertial mode in turn, comparing whether and how often tracking is lost.
6. A scale check: run once in monocular and once in monocular-inertial mode; in evaluating with evo the former requires a scale alignment and the latter does not—verifying at first hand the value of the IMU's absolute scale (Section 20.5).

7. An excitation experiment: if using your own camera and IMU, try three ways of starting up—standing still, moving at constant velocity in a straight line, and moving with accelerations and rotations—and observe which succeeds in initializing, verifying the argument of Section 23.4.

Part Three: the Atlas Multiple-Map Experiment (the core; university level)

8. Cause a loss of tracking deliberately: while running a dataset, interrupt the image input by program for a few seconds (or choose a sequence with a severe occlusion) and observe whether the system opens a new map.
9. Observe the state of Atlas: from the terminal output or the visualization, confirm how many maps there are at present and which is active.
10. Map merging: let the camera return to the region covered by the old map, observe the instant the merge occurs, and record it on video—the picture most worth demonstrating in this chapter.
11. A comparative experiment: run ORB-SLAM2 and ORB-SLAM3 in turn on the same sequence containing a severe loss of tracking and compare the completeness of the final trajectory, reproducing Case 23-1.
12. A multi-session experiment: using several sequences of the same EuRoC scene (MH_01 through MH_05, say), run them in order and let the system merge them, observing how multi-session operation extends the map by degrees.

Discussion Questions

1. By how much do the accuracies (ATE) of pure stereo and stereo-inertial differ? On which kind of sequence is the gap greatest? Why?

2. On a difficult sequence, how many times did pure vision lose tracking? How much did adding the IMU improve matters? Against Section 7.6, which contribution of the IMU came into play here?
3. How long did IMU initialization take in your experiment? What happens if the body is held still during initialization? Which argument of Section 23.4 does this bear out?
4. What changes in the trajectory and the map when a merge occurs? How does this resemble and differ from the loop closure correction of Chapter 22? (Hint: the unified flow of Section 23.6.)
5. (University level) By the four kinds of data association of Section 23.2, which kind is the principal source of ORB-SLAM3's advantage in accuracy over VINS-Mono? Why does it matter so much?

Extended Challenge (university level)

First: a field test on campus—record data in the corridors and lobby of your department building with a stereo-inertial camera of your own (or a depth camera containing an IMU) and run ORB-SLAM3 to build a map. Design a route deliberately that passes through a dim machine room to cause a loss of tracking and then returns to a bright corridor, test Atlas's ability to merge, and assess the quality of the map by the method of Section 13.7. Second: a cross-modal comparison—record visual-inertial and LiDAR-inertial data in the same site at the same time, run ORB-SLAM3 and the LIO system of Chapter 18 respectively, and compare their performance in three environments (a bright corridor, a dim machine room, a glass-curtained lobby), verifying quantitatively the argument for the complementarity of sensors in Chapters 8 and 9. This topic has both teaching and research value and suits being the spine of a project.

Further Reading

1. C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM," IEEE T-RO, 2021. (The principal source of this chapter.)
2. R. Elvira, J. D. Tardós, and J. M. M. Montiel, "ORBSLAM-Atlas: A Robust and Accurate Multi-Map System," IROS, 2019. (The precursor work on the Atlas multiple-map architecture.)
3. C. Campos, J. M. M. Montiel, and J. D. Tardós, "Inertial-Only Optimization for Visual-Inertial Initialization," ICRA, 2020. (The theoretical source of the three-stage IMU initialization procedure.)
4. C. Forster et al., "On-Manifold Preintegration for Real-Time Visual-Inertial Odometry," IEEE T-RO, 2017. (IMU preintegration, the theoretical basis of Section 7.5.)
5. The ORB-SLAM3 open-source project (GitHub: UZ-SLAMLab/ORB_SLAM3). (Source code, compilation instructions, and examples of running each mode.)
6. T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," IEEE T-RO, 2018. (Another classic visual-inertial system, to be read alongside this chapter.)
7. The official websites of the EuRoC MAV and TUM-VI datasets. (The standard benchmarks used in this chapter's experiments and in the papers' evaluations.)

Summary

1. ORB-SLAM3 has three core advances: tightly coupled visual-inertial SLAM founded on maximum a posteriori estimation, the Atlas multiple-map system, and the thoroughgoing use of four kinds of data association at every

algorithmic stage; it supports monocular, stereo, and RGB-D cameras with pinhole and fisheye lenses in many configurations.

2. The four kinds of data association are short-term (the last few seconds), mid-term (elements nearby with slight drift), long-term (matching a previously visited region by place recognition), and multi-map (across mapping sessions); mid-term association is the key source of its advantage in accuracy, making it 2.6 times as accurate as VINS-Mono in single-session monocular-inertial operation on EuRoC.
3. In visual-inertial mode the state vector grows from the 6 degrees of freedom of pure vision to 15 (adding velocity and two biases); the IMU brings absolute scale, the direction of gravity, support when vision fails, and high-rate output, while vision repays it by correcting the IMU bias with observations that do not drift.
4. IMU initialization proceeds in three stages: vision-only MAP estimation → inertial-only MAP estimation with the visual trajectory held fixed → a joint visual-inertial BA. It reaches a 5 percent scale error within 2 seconds and about 1 percent after refinement, but there must be sufficient excitation of motion, or scale and bias are unobservable.
5. Atlas consists of one active map and several inactive maps sharing a bag-of-words key frame database; when tracking is lost it opens a new map and carries on, and performs a seamless merge when a cross-map association is later detected—unlike other schemes, which keep their submaps as separate entities.
6. Place recognition queries Atlas for similar frames for every new key frame to achieve high recall, then ensures a hundred percent precision with several steps

of geometric verification and a Sim(3) / SE(3) alignment; a match in the active map means a loop closure, otherwise a merge. The merge proceeds in two stages, fusion within a welding window and propagation by pose graph, so as not to block tracking.

7. Visual-inertial SLAM is widely applied in phone AR, wearable cameras, and drones; in the TUM-VI room sequences, which resemble AR / VR use, the mean ATE is 0.009 meters in stereo-inertial mode and 0.011 meters in monocular-inertial, which is millimeter-level accuracy.

Carrying Forward

This chapter has joined the IMU of Chapter 7 to the system architecture of Chapter 22, and measured three sets of numbers along the way: for the IMU to carry across a gap in vision the key is the bias—with the bias estimated to ±0.01 the reckoning error after two seconds is 0.025 meters, and with it estimated to ±0.3 it is 1.10 meters, forty times greater; under constant-velocity straight-line motion the condition number of initialization reaches 7,673 at a scale error of 100 percent, while full excitation brings the condition number down to 44 and the error to 0.37 percent; and after a loss of tracking, Atlas's merge presses the ATE from 0.787 meters unmerged down to 0.026 while covering 90 percent of the trajectory.

The core systems of Part VI have now been introduced in full. But SLAM is not an end in itself—once a robot has built a map and knows where it is, what is it to do with that? In the next chapter we widen our view from localization and mapping to navigation and tasks: how does visual navigation differ from SLAM? How do line following, visual servoing, and navigation by artificial markers work? How do object

detection and vision-language models let a robot understand an instruction such as please go to the registrar's office? And how does a robot dog or a campus inspection robot integrate all of this? Chapter 24 will connect the technology of this book to real robotic tasks.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 24

The Vision-Navigated Robot

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

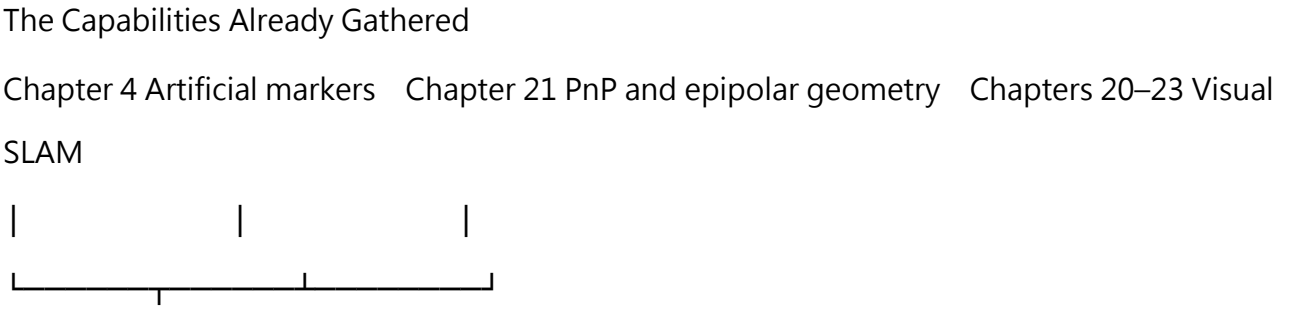
Where This Chapter Sits in the Book

Chapters 20 through 23 built visual SLAM all the way from features to a complete system. This chapter asks the next question: once a robot knows where it is, what is it to do with that?

The character of this chapter is quite unlike that of the four before it. Those four pursued accuracy and robustness; this one is about trade-offs—many navigation tasks do not need SLAM at all. Laying down a line, putting up a few markers, or simply doing visual servoing straight at the target is cheaper and more reliable than full SLAM in the right setting. The point of engineering judgment is not to use the strongest technology but to use just enough of it.

The three figures of this chapter all quantify these trade-offs: Figure 24-1 compares the convergence behavior of two kinds of visual servoing and explains why precise docking must choose the right one; Figure 24-2 measures how the pose accuracy of a marker varies with distance and viewing angle; and Figure 24-3 answers a very practical question—how far apart should markers be put up?

Figure 24-0 The Link Map of Chapter 24



navigation

▼

Part VII Miniature robots and drones Part VIII ROS implementation

▼

Chapter 32 Semantic SLAM Chapter 34 Language navigation and embodied intelligence

Learning Objectives

In the last four chapters we established a complete capability in visual SLAM: the robot can know where it is and what the environment looks like. But SLAM is not an end in itself—once a map has been built, what is it to be used for? This chapter, the last of Part VI, connects the technology to real robotic tasks, from the plainest line following all the way to VLM navigation that can understand human speech. After studying this chapter, students should be able to:

1. Explain how visual navigation differs from and complements visual SLAM.
2. Compare the three basic strategies of visual navigation: line following, landmark navigation, and visual servoing.
3. Describe four typical uses of AprilTag / ArUco in robot navigation.
4. Explain how YOLO object recognition supports semantic navigation, and understand its limitations from a robot's point of view.
5. Explain how a VLM vision-language model lets a robot understand a natural-language navigation instruction.
6. Describe the particular challenges of visual navigation on a robot dog and the measures taken against them.
7. Plan the system architecture of a campus inspection robot or a recycling-sorting robot.

Core Concepts

Chinese Term	English Term	Explanation
視覺導航	Visual Navigation	The technology of guiding a robot to a goal by visual information.
線追蹤	Line Following	The most basic form of visual navigation, traveling along a line on the floor.
視覺伺服	Visual Servoing	A method of controlling a robot's motion directly by image feedback.
影像式視覺伺服	IBVS	Takes the feature error in the image directly as the control signal.
位置式視覺伺服	PBVS	Solves for the three-dimensional pose error first and then controls.
地標導航	Landmark Navigation	Deciding the direction of travel from recognizable landmarks.
語意導航	Semantic Navigation	Navigating by the meaning of objects and scenes rather than geometric coordinates.
物件目標導航	Object-Goal Navigation	Accepting a task instruction of the form go find such-and-such an object.
視覺語言導航	VLN	Navigating an environment by natural-language instruction.
視覺語言模型	VLM	A multimodal AI model able to understand image and language at once.
視覺語言動作模型	VLA	A model that generates action commands directly from image and language.
邊界探索	Frontier Exploration	Advancing toward the boundary between the known and the unknown to explore an environment.
零樣本	Zero-Shot	The ability to perform a new task without task-specific training.

24.1 How Visual Navigation Differs from SLAM

This is the first idea that must be made clear in this chapter. Many students conflate visual SLAM with visual navigation, but the two answer different questions.

Two Different Questions

- Visual SLAM answers: where am I, and what does the environment look like? What it produces is a pose and a map—the theme of Chapters 20 through 23.
- Visual navigation answers: which way should I go? What it produces is a motion command—the theme of this chapter.

Set against the three questions of navigation in Chapter 3: SLAM addresses the first (where am I), while visual navigation covers the second and the third (where am I going, and how do I get there).

Visual Navigation Does Not Necessarily Need SLAM

This is an important fact that is often overlooked. Some methods of visual navigation build no map at all:

- A line-following robot looks only at the line on the floor; it does not know where it is and does not need to (Section 24.2).
- Visual servoing turns the offset of the target in the image directly into a motor command, passing through no map at all.
- A corridor robot following visual landmarks need only recognize that on seeing the fire extinguisher, turn right; it needs no global coordinates.

These methods belong to the reactive architecture introduced in Chapter 3—perception drives behavior directly, with no model and no planning. They are simple, fast, and robust, and very practical in a controlled environment.

SLAM Raises the Level of Navigation

But reactive navigation has a clear ceiling: it cannot plan a global path, cannot handle going to a place no line leads to, and cannot change its goal midway. With SLAM added, a robot gains:

1. Global path planning: computing the best route on a map (Chapters 3 and 29).
2. Navigation to any goal: able to go to any coordinate on the map, not only along a preset route.
3. Recovery from a break: able to relocalize after losing tracking and carry on with the task (Chapters 22 and 23).
4. Semantic tasks: combined with a semantic map, able to understand an instruction such as go to the registrar's office (Section 24.5).

Item of Comparison	Visual SLAM	Visual navigation
The central question	Where am I? What does the environment look like?	Which way should I go?
Output	Pose and map	Motion commands (speed, direction)
Is a map needed	It produces a map	Not necessarily
Type of architecture	Deliberative (a model is required)	May be reactive or deliberative
Which of the three questions of navigation	The first	The second and third
Chapters of	Chapters 19–22	This chapter and Chapter 29

Item of Comparison	Visual SLAM	Visual navigation
this book		

Table 24-1 Visual SLAM Compared with Visual Navigation

Working systems mostly use both: SLAM supplies global localization and planning while reactive visual navigation handles real-time obstacle avoidance and precise docking. This is exactly the hybrid architecture of Chapter 3.

24.2 Line Following, Landmark Navigation, and Visual Servoing

This section introduces three basic strategies of visual navigation that require no full SLAM, from the simple to the elaborate.

One: Line Following

This is the plainest robot, and the first that countless students ever build: lay a strip of black tape on the floor, have the robot detect the position of the line with a downward camera or an infrared sensor, and turn left if the line is to the left and right if it is to the right.

- The principle: convert the image to grayscale or binarize it (the basic skill of image processing), find the offset between the center of the line and the center of the frame, and turn that into a steering command.
- Advantages: extremely low cost, extremely simple code, extremely robust, and no need for a map or localization.
- Limitations: it can only follow a preset route, cannot go around an obstacle, and the environment must be modified (the line must be laid).

- In practice: guidance by magnetic strip or colored tape is still widely used for factory AGVs—in a controlled environment it is irreplaceably cheap and reliable (echoing the idea of the Amazon warehouse case in Chapter 4: the more controlled the environment, the simpler the choice of technology may be).

Two: Landmark Navigation

One step beyond line following: instead of laying a continuous line, place recognizable landmarks at key positions, and the robot turns right on seeing landmark A and stops on seeing landmark B.

A landmark may be artificial (the AprilTag, QR code, or color patch of Section 4.6) or natural (a door, a fire extinguisher, a fire hydrant). The former is reliable but requires modifying the environment, the latter requires no modification but is harder to recognize—which is where AI recognition technologies such as YOLO come into their own (Section 24.4).

Landmark navigation in essence establishes a topological map (Chapter 3): the nodes are the landmarks and the edges are the actions of traveling from one landmark to the next. It requires no precise coordinates yet can accomplish quite complicated route tasks.

Three: Visual Servoing

Visual servoing is the most precise of the three, and it puts the camera directly into the control loop: it continually compares the difference between the current image and a target image and generates motion commands to reduce that difference until the two agree.

It has two great schools:

- Image-based visual servoing (IBVS): takes the feature error in the image directly as the control signal (the target point should be at the center of the frame and is now 30 pixels to the left, say). Its advantages are that no three-dimensional reconstruction is required and that it is insensitive to errors in camera calibration; its drawback is that the robot's actual trajectory may be counterintuitive.
- Position-based visual servoing (PBVS): first solves from the image for the three-dimensional pose error of the camera relative to the target (using the PnP of Chapter 21) and plans its motion accordingly. Its advantage is that the trajectory is sensible in three-dimensional space; its drawback is that accurate camera calibration and a model of the target are required.

The typical application of visual servoing is precise docking: a robot arm grasping an object, a cleaning robot aligning itself to its dock, a drone landing precisely. It can reach millimeter-level relative positioning accuracy—far above the global localization accuracy of SLAM, because it cares only about position relative to the target in front of it.

Method	Requires modifying the environment	Accuracy	Flexibility	Typical applications
Line following	Yes (a line must be laid)	Medium	Low (a fixed route)	Factory AGVs, teaching robots
Landmark navigation	Depends on the landmarks	Medium	Medium (a topological route)	Corridor patrol, exhibition guiding
Visual servoing	No	Extremely high (millimeter	Low (only toward the target)	Precise docking, grasping, landing

Method	Requires modifying the environment	Accuracy	Flexibility	Typical applications
		level)		

Table 24-2 A Comparison of the Three Basic Strategies of Visual Navigation

Measuring It: Where the Two Kinds of Visual Servoing Differ

Section 24.2 says that in image-based visual servoing the robot's actual trajectory may be counterintuitive while in the position-based kind the trajectory is sensible in three-dimensional space. Those two statements sound like a difference of style; they are in fact a substantive difference that decides success or failure. This subsection computes it.

The experimental setting: the camera is to move from a displaced starting pose to a position facing the target squarely at a distance of 0.65 meters. The target is the four corners of a square 20 centimeters on a side. Both control laws take their standard form—the image-based one multiplies the feature error by the pseudoinverse of the image Jacobian, and the position-based one solves for the three-dimensional pose error first and commands a velocity directly. The key variable is how many degrees the camera is rotated about its optical axis at the start.

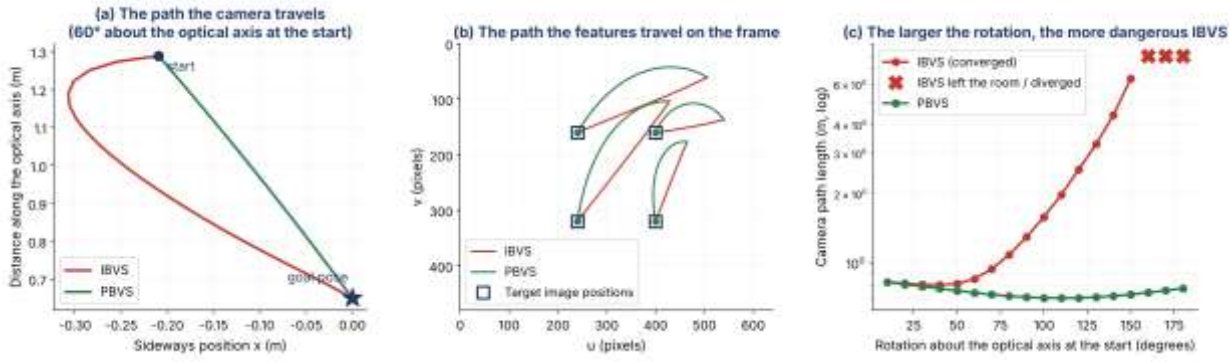


Figure 24-1 The Convergence Behavior of Image-Based and Position-Based Visual

Servoing

Panel (a) is the path the camera travels in space, starting rotated 60 degrees about the optical axis. The position-based control (green) runs almost straight from the starting pose to the goal, 0.74 meters in all; the image-based control (red) first swings further sideways, out to 0.31 meters off the axis of the goal pose when it started only 0.21 meters off it, and only then closes on the target, 0.85 meters in all. This swinging away before closing in is the same phenomenon as the famous camera retreat of image-based visual servoing: the control is shortening the error on the frame, not the error in space, so the camera is free to take a strange route through the room.

Panel (b) is the same process as it appears on the image plane, and here the verdict is entirely reversed. The four feature tracks of the image-based control are very nearly straight—which is exactly its design aim: it minimizes error in the image, so the features move along the shortest path on the frame. The feature tracks of the position-based control bend plainly, because it cares only about three-dimensional pose and not how the features travel across the frame.

This contrast shows the nature of the two: the image-based kind guarantees good behavior on the frame (the features are unlikely to leave the field of view) while the camera's actual motion may be strange; the position-based kind guarantees good motion of the camera while the features may leave the frame along the way. Which to choose depends on what your failure mode is—an arm working in a confined space that might collide should take the position-based kind; a target occupying only a small part of the frame that might be lost should take the image-based kind.

Panel (c) sweeps the starting rotation from 10 degrees to 180. The position-based path length stays between 0.70 and 0.82 meters throughout, very nearly the straight-line

distance from start to finish, and it converges every time. The image-based control is equally fine within 50 degrees (about 0.80 meters), beyond which the path swells rapidly—0.85 meters at 60 degrees, 1.09 at 80, 1.30 at 90, 2.55 at 120, 4.41 at 140 and 6.37 at 150—and beyond that it no longer converges at all: at 160, 170 and 180 degrees the camera retreats clean out of the room.

The Famous Counterexample at 180 Degrees

The best-known failure of image-based visual servoing is precisely the target being rotated exactly 180 degrees relative to the camera. The pseudoinverse of the image Jacobian then commands the camera to retreat straight back along the optical axis—because as far as the image is concerned, the farther back it goes the nearer the four corners come to the center of the frame, so the error appears to be decreasing. The result is that the camera retreats toward infinity while the error never comes down.

This experiment reproduces the phenomenon above 150 degrees: path lengths run to millions of meters, which is not the robot really traveling that far but the control law having lost control.

There are three remedies in practice: switched control (position-based at a distance, image-based close in), hybrid visual servoing (image-based for translation, position-based for rotation), or the simplest of all—align the attitude roughly by some other means first and let visual servoing finish the job.

24.3 Navigation with AprilTag / ArUco

Section 4.6 introduced the principle of visual fiducial markers—a complete 6-DoF

pose solved from four corners and the camera intrinsics (the PnP problem of Chapter 21). This section focuses on how they are actually used in navigation tasks.

Four Typical Uses

5. Absolute pose correction: put markers in corridors, at elevator doors, and at laboratory entrances. The moment the robot sees any marker it obtains an absolute pose that does not drift, eliminating SLAM's accumulated error at a stroke. This is the commonest and most practical use (the idea of fusion in Section 4.7).
6. Precise docking: put markers at the charging station, at pick-and-place points, and in front of elevator buttons, and complete the final alignment to the millimeter with visual servoing (Section 24.2). SLAM's localization accuracy is usually a few centimeters, not enough for an action requiring precise docking.
7. Identifying floors and zones: as Section 5.7 mentioned, LiDAR SLAM cannot tell apart different floors that look alike. Put markers of different ID at the elevator door of each floor and the problem is solved.
8. Evaluating SLAM accuracy: markers of known coordinates placed densely about a site can serve as an approximate ground truth trajectory for evaluating the accuracy of a SLAM system (the ATE evaluation of Section 22.7 requires a ground truth). This is extremely practical for a teaching laboratory with no motion capture system.

Points of Implementation

- Marker size and detection distance: the detection distance is roughly proportional to the side length of the marker. A corridor marker that is too small must be approached closely before it can be seen at all, which defeats the

purpose of correction. In practice markers of different sizes are made as required.

- Mounting height and angle: a marker should be put up at a sensible height within the robot camera's field of view and should face the passage as squarely as possible. At too oblique an angle the detection accuracy of the four corners falls and the error of the solved pose grows.
- Light and reflection: a marker should avoid direct reflection (a laminated marker may be entirely unrecognizable under a lamp). Printing on a matte material is recommended.
- Managing uniqueness: the ID of every marker and its world coordinates must be filed and managed. This marker map is itself a simple kind of semantic map (Chapter 3).

Levels of Fusion with SLAM

Section 4.7 mentioned three levels of fusion; here they are made concrete:

- Loose coupling: SLAM runs normally and the pose is simply pulled to the value the marker gives whenever a marker is seen. The simplest to implement and suitable for teaching.
- Tight coupling: the marker observation is added as a constraint in SLAM's back-end optimization (an edge, in the sense of Chapter 12). The most accurate, and a research-grade practice.
- Aiding initialization and relocalization: the most practical of all—SLAM runs on its own ordinarily and markers supply an absolute reference only at start-up or after a loss of tracking. This solves the most troublesome problem at a very low cost.

Measuring It: How Large, How Oblique, How Densely to Put Markers Up

The points of implementation of Section 24.3 gave three principles: a marker too small cannot be seen far, accuracy falls at too oblique an angle, and uniqueness must be managed. The first two can be quantified, and quantifying them yields something more useful than a principle—numbers that can be written straight into a planning document.

The method of the experiment: take a marker 16 centimeters on a side, observe it with a camera of 520 pixels focal length at a resolution of 640×480, add localization noise to the four corners, solve PnP, and compare with the true pose. The model of the corner noise is this: the smaller the marker projects on the frame (through distance or the foreshortening of an oblique view), the greater the noise—which is closer to the behavior of a real detector than a fixed noise would be.

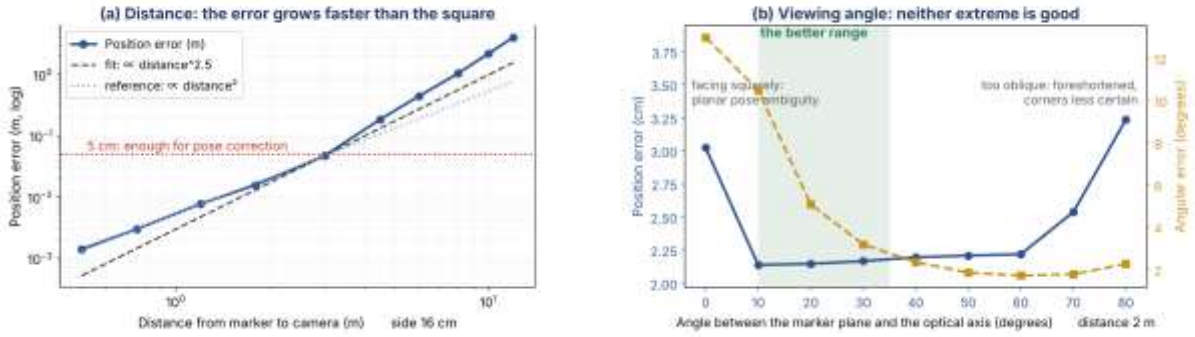


Figure 24-2 How the Pose Accuracy of a Marker Varies with Distance and Viewing Angle

Panel (a) is the effect of distance, with both axes logarithmic. At 0.5 meters the position error is only 1.4 millimeters, at 1.2 meters 7.7 millimeters, at 3 meters 4.8 centimeters, at 6 meters 0.44 meters, and at 12 meters 4.0 meters. The fitted exponent is 2.52—steeper than a simple inverse square. The extra half a power comes from corner detection itself: the smaller the marker on the frame, the harder the corner positions are to pin down, and the noise multiplies with the geometric magnification.

This curve yields a very practical design rule. Taking a position error of 5 centimeters as the threshold at which pose correction is worthwhile (SLAM's own localization error is usually a few centimeters too), the effective range of a 16-centimeter marker is about 3 meters. For a corridor where it must be usable at 6 meters, the side length must be doubled to about 32 centimeters. This is the concrete conversion behind the sentence the detection distance is roughly proportional to the side length of the marker in Section 24.3.

Panel (b) is the effect of viewing angle, and the result is more nuanced than the statement of Section 24.3. In principle the pose error grows at too oblique an angle is correct—at 80 degrees the position error is 3.2 centimeters, half as much again as the best value. But facing the marker squarely is no good either: at 0 degrees the angular error reaches 13.0 degrees, the worst point on the whole curve.

Facing squarely is poor because of the pose ambiguity of a planar target: when the marker faces the camera squarely, the four corners projected by tilting the marker a little to the left and a little to the right are nearly identical, and the direction of rotation is mathematically close to unobservable. Tilt it slightly and that symmetry is broken. Each end has its own trouble, and the optimum lies at about 10 to 35 degrees.

A Mounting Rule You Can Use Directly

Putting the two conclusions of Figure 24-2 together, the recommendation for mounting corridor markers is this: take the side length as the expected detection distance divided by 20 (16 centimeters for 3 meters, 32 centimeters for 6); and do not put it squarely on the wall straight ahead of the passage, but a little to one side, or let the robot pass obliquely, so that the viewing angle

falls between 10 and 35 degrees.

Neither of these costs anything, yet together they can make the same batch of markers several times more accurate. When planning a site, students would do well to run the experiment of Figure 24-2 once with their own camera—a different focal length and resolution will give different numbers.

Measuring It: How Far Apart Should Markers Be

What was measured above was the accuracy of a single marker; next comes the more practical question at the level of the system: how many markers does a patrol route need?

The experimental setting is a 90-meter U-shaped corridor patrol route of 360 key frames. Each step of the odometry carries noise, and an appreciable drift accumulates over the route. Markers are put up at a fixed interval, and the absolute pose measured from them carries noise (a standard deviation of 2 centimeters in position and 2 degrees in angle, inferred from the results of Figure 24-2). Three practices are compared: no markers at all, loose coupling (pull the pose over whenever a marker is seen), and tight coupling (take the marker observation as a constraint in the optimization and solve it together with the odometry).

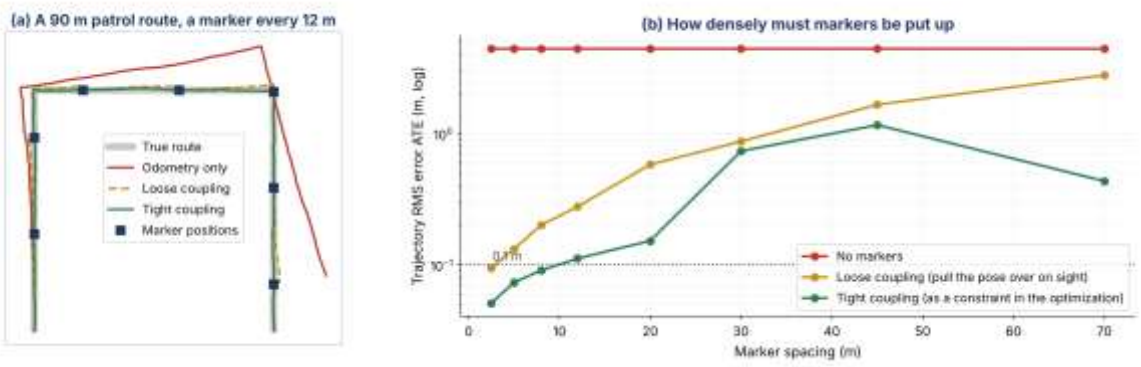


Figure 24-3 The Effect of Marker Spacing on Localization Accuracy

Panel (a) draws the actual trajectories with a marker every 12 meters. Pure odometry (red) has plainly departed from the route by the end of one circuit; loose coupling (the orange dashed line) is pulled back at every marker but drifts away again between them, forming a sawtooth trajectory; tight coupling (green) hugs the true route throughout.

Panel (b) sweeps the marker spacing. With no markers the ATE is fixed at 4.44 meters. Loose coupling gives 0.095 meters at a spacing of 2.5 meters, 0.132 at 5 meters, 0.278 at 12 meters, and 0.581 meters at 20. Tight coupling at the same spacings gives 0.051, 0.074, 0.112, and 0.153 meters—better at every point, and the greater the spacing the plainer the difference: at a spacing of 20 meters the error of tight coupling is only a quarter of that of loose coupling.

Why is the advantage of tight coupling greater when markers are sparse? Because loose coupling can correct only at the instant a marker is seen, and the correction cannot be carried back to the poses already traversed; tight coupling takes the marker as a constraint within the optimization, so the error is redistributed along the whole chain of poses. This is the same thing as the conclusion of Figure 22-2 in Chapter 22—a correction can spread only through the connectivity of the graph.

One detail must be pointed out honestly: at spacings of 45 and 70 meters only one

marker is left on the whole route, and the result is then governed by where that one marker happens to be rather than by the spacing itself. This is why the error at the 70-meter point in the figure is in fact lower than at 45. The figure can therefore be read directly only over the range in which the markers are numerous enough to form a chain.

Three Numbers for Planning a Site

Taking the setting of this experiment (a 90-meter route with odometry noise of a few centimeters):

To press the ATE within 10 centimeters, tight coupling requires a spacing of about 5 meters and loose coupling a spacing within 2.5—which is to say that adopting tight coupling halves the number of markers.

If 30 centimeters will do (enough for most inspection tasks), tight coupling may be relaxed to a spacing of about 20 meters, so that the whole route needs only four or five markers. This difference decides directly both the cost of deployment and the work of maintenance.

24.4 YOLO Object Recognition and Semantic Navigation

In the navigation of the three preceding sections the robot does not understand what it sees—a line is a line and a marker is a number. Only with object recognition added does a robot begin to understand the meaning of a scene.

From Geometry to Semantics

Chapters 1 and 3 mentioned the idea of a semantic map: upon the geometric map, labeling this is a door, that is a staircase, over there is a fire extinguisher. Real-time object detection models of the YOLO kind are exactly the tool for producing these

labels.

With semantics, the way a navigation task is expressed changes entirely:

- Geometric navigation: go to coordinates (12.5, 8.3)—the user must know the coordinates.
- Semantic navigation: go to the nearest water fountain—the user need only name the object.

The latter is the natural way for a human being to communicate, and the premise of a service robot that is genuinely usable.

Three Roles for YOLO in Navigation

9. Semantic labeling: a detected object is written into the map together with its three-dimensional position (depth supplied by RGB-D or stereo), forming a semantic map.
10. Handling moving objects: recognize movable things such as people and vehicles and remove the features of those regions within SLAM, keeping moving objects from disturbing the pose estimate (the weakness of visual SLAM mentioned in Section 9.5). This is an extremely practical technique.
11. Detecting task targets: an inspection robot recognizing instruments, cracks, and litter; a recycling robot recognizing kinds of bottle and can—this is the core function of the task itself.

The Challenge of the Robot's Point of View

Here is a problem easily overlooked yet absolutely crucial in practice: most vision models are trained on photographs from a human point of view, while a robot's point of view is often altogether different.

Research points out that autonomous navigation on a quadruped robot requires robust semantic perception, and yet, because such platforms operate at a height below 50 centimeters, vision models trained from a human-centric point of view often do not work well.

This problem holds for cleaning robots, robot dogs, and small wheeled robots alike: a chair seen looking up from 30 centimeters, or a door seen from floor level, differs greatly from the photographs in the training data, and the recognition rate falls accordingly.

- Remedy one: collect data from the robot's point of view and retrain or fine-tune the model—which is exactly the intent of stage four of the project in Chapter 31 (training YOLO to recognize inspection targets) and of the semantic labeling of Chapter 32.
- Remedy two: use a newer model with open-vocabulary ability, whose generalization is stronger.
- Remedy three: adjust the mounting height and pitch angle of the camera to approach the point of view of the training data as nearly as possible.

This case bears out once again an idea stressed repeatedly in this book: the performance of a technology depends greatly on the circumstances of its deployment, and one must not look only at the numbers in a paper.

24.5 Navigation Aided by a VLM Vision-Language Model

YOLO lets a robot recognize objects, but it can still answer only what is in the frame. A VLM (vision-language model) goes a step further—it can understand a scene, reason with common sense, and accept natural-language instructions. This is a paradigm shift

now under way in robot navigation.

What a VLM Brings

A VLM, pretrained on a vast corpus of images and text, displays a powerful ability to unify the visual and linguistic modalities, and is increasingly important for navigation tasks that require rich semantic perception. Early models aligned image and text features into a common embedding space, while CLIP introduced contrastive learning to achieve open-vocabulary visual recognition, greatly improving the generalization of multimodal representations; more recent VLMs go further and provide a unified interface for vision-language reasoning and decision support.

For navigation, the key value of a VLM lies in common-sense reasoning. If the instruction is go find a washbasin, for instance, the robot need not have labeled the position of a washbasin beforehand—a VLM knows that washbasins are usually in a bathroom or a kitchen and will explore regions of that kind first. This ability is called a semantic prior.

Three Modes of Application

12. Vision-language navigation (VLN): accepts a natural-language instruction such as walk to the end of the corridor, turn right, and stop at the second door. The system must understand the instruction, relate it to the current field of view, and carry it out step by step.
13. Object goal navigation: accepts a goal such as go find a trash can, and the system must explore an unknown environment and find it. A VLM's common-sense reasoning makes the exploration more efficient.

14. Socially aware navigation: understands the people and the situation in a scene and decides on fitting behavior (not forcing through a crowd, slowing down outside a classroom).

Zero-Shot Ability: No Task-Specific Training

The most pathbreaking property of VLM navigation is its zero-shot ability—no retraining is required for each new environment or new task.

Related research uses a VLM as a global planner, aggregating the submaps of several robots from different viewpoints into one unified global map that encodes the robots' states and the frontier regions; the VLM then allocates frontiers among the robots, bringing about coordinated and efficient exploration. Experiments show that methods of this kind surpass existing baselines in success rate and navigational efficiency without any task-specific training, and ablation experiments further confirm the importance of the VLM's semantic prior.

This stands in sharp contrast with traditional learned navigation, which generally requires collecting a great deal of data in the target environment for training and must start over in another site.

A Frontier Direction: the Vision-Language-Action Model (VLA)

A newer development is the VLA (Vision-Language-Action) model—which does not merely understand but generates action commands directly from image and language.

Recent research proposes a training framework achieving omni-modal goal conditioning: a high-capacity vision-language-action backbone trained simultaneously on three goal modalities—2D pose, first-person image, and natural

language—and on their combinations. This echoes a human capability: we interpret and combine different forms of goal specification flexibly, whether a verbal instruction, a spatial coordinate, or a visual reference.

Other research demonstrates one and the same agent core deployed on four heterogeneous real robots—a dual-arm wheeled platform, a quadruped, a humanoid, and a self-built quadrotor—with only the low-level controller changed. This generality across platforms is a concrete manifestation of the vision of a robotics foundation model.

Practical Limitations: Do Not Forget Engineering Reality

VLM navigation is exciting, but students must recognize its limitations:

- Computational burden: inference with a large VLM takes time and is hard to run in real time on a robot. Hence a variety of efficiency studies, such as attention-based assessment of token importance and query-guided spatiotemporal filtering, which retain the navigation-relevant information without retraining or modifying the pretrained model, and which have been verified for reliable, low-latency instruction-following navigation on a real quadruped robot.
- The risk of hallucination: a VLM may see things that are not there or reason wrongly. A robot that trusts it completely may run into a wall or take the wrong path. A working system must retain a geometric layer of safety checking (the obstacle avoidance of Chapter 29).
- Latency and safety: a VLM suits high-level decisions (where to go) and not real-time control (dodging an obstacle that appears suddenly). A layered

architecture is the inevitable choice—the modern version of the hybrid architecture of Chapter 3.

These limitations are exactly what Chapter 32 will discuss in depth. For now students need only establish one central understanding: a VLM lets a robot understand and reason, but walking steadily and not running into people still rests on the geometry and control of the preceding twenty-three chapters. Neither can be dispensed with.

24.6 Visual Navigation on a Robot Dog

The quadruped robot (the robot dog) has in recent years become a popular platform for research and application in visual navigation. This section explains its particular challenges and the measures taken against them.

Why Choose a Robot Dog

- Adaptability to terrain: it can go up and down stairs, cross thresholds and gravel, and walk over uneven ground—which a wheeled robot cannot do (Section 3.6).
- Suitability for campus and factory sites: on a campus with stairs, slopes, and outdoor passages, a robot dog's ability to get about is plainly better than a wheeled robot's.
- A mature platform: commercial quadruped robots are now quite widespread and provide open development interfaces, which suits them to research and teaching.

Four Particular Challenges

15. Gait vibration: in quadruped walking the body heaves continually up and down and sways from side to side, causing blurred images and violent changes of

pose. This is a severe test for visual SLAM—precisely why the tight IMU coupling of Chapter 23 is indispensable, and it echoes the argument of Chapter 18 that a robot dog needs tightly coupled LiDAR-inertial estimation.

16. The problem of the low viewpoint: as Section 24.4 described, a robot dog operates at a height below 50 centimeters, and vision models trained from a human-centric point of view often do not work well. The accuracy of semantic perception falls accordingly.

17. Understanding three-dimensional terrain: a wheeled robot need only know whether it can go forward, while a robot dog must also know how high this step is, whether it can be climbed, and whether the foothold is secure. Navigation rises from 2D to 2.5D or even full 3D (Section 3.6).

18. Endurance and computational limits: quadruped walking consumes far more energy than wheels, and the computing platform that can be carried is limited as well. This makes lightweight algorithms and edge computing especially important (the theme of Part VII).

Measures and System Design

- The sensor combination: a 3D LiDAR or depth camera (terrain) + an IMU (attitude and vibration compensation) + an RGB camera (semantics). Multi-sensor fusion is here not an option but a necessity.
- Tightly coupled estimation: with the LIO of Chapter 18 or the visual-inertial SLAM of Chapter 23 as the skeleton, the IMU is the key to stability.
- Layered control: the upper layer makes semantic decisions and plans paths (a VLM may be used), the middle layer assesses terrain and plans footholds, and the bottom layer controls the gait. The layers work on different time scales, a typical realization of the hybrid architecture of Chapter 3.

- Localizing the semantic model: collecting data from the robot dog's point of view to fine-tune the model is the most direct way to raise the recognition rate.

The Research Frontier

The robot dog has become the standard platform for verifying frontier navigation technologies. Recent research in vision-language navigation generally verifies the practicality of its methods on a real quadruped robot—because it has both the complexity of the real world and controllable experimental conditions. Work proposing a vision-language-action navigation model designed specifically for legged robots has already appeared as a representative piece, which shows how active this direction is.

24.7 The Campus Inspection Robot and the Recycling-Sorting Robot

This section integrates the technology of this chapter into two concrete project plans, as the conclusion of Part VI and in preparation for the project of Chapter 31.

Project One: the Campus Inspection Robot

This is the spine already mentioned in Sections 5.7 and 18.7; this section plans it fully from the standpoint of visual navigation.

Figure 24-4 The System Architecture of a Campus Inspection Robot

【Perception layer】an RGB-D or stereo camera + an IMU + (optionally) a LiDAR and GNSS

↓

【Localization and mapping】ORB-SLAM3 visual-inertial SLAM (Chapter 23) + AprilTag absolute pose correction (Section 24.3)

↓

【Semantic understanding】YOLO object detection → a semantic map (Section 24.4)

↓

【Task decision】a list of inspection points / a VLM natural-language task (Section 24.5)

↓

【Path planning】global planning (the map) + local obstacle avoidance (real-time perception)

↓

【Execution and recording】move to the inspection point → photograph / measure → label with three-dimensional coordinates and save

The Visual Essentials of an Inspection Task

- Reading instruments: detecting and reading the values of pressure gauges, electricity meters, and thermometers. This requires precise alignment (visual servoing, Section 24.2) together with digit recognition.

- Detecting anomalies: cracks, traces of leakage, rust, piled-up clutter—mostly things only vision can see (Chapter 8 mentioned that a LiDAR cannot supply semantics).
- Detecting change: comparing images of the same position at different times to find anomalies that were not there yesterday and are there today. This requires an accurate ability to relocalize repeatedly (the multi-session operation of Atlas in Chapter 23 applies exactly).
- Access control and safety: detecting whether a door is properly shut, whether a fire escape is blocked, whether a fire extinguisher is in place.

Project Two: the Recycling-Sorting Robot

This is a topic of both technical challenge and educational value, particularly suited to collaboration between high school and university students.

- The task: the robot patrols the campus, recognizes and records (or collects) recyclable items, and sorts them into PET bottles, aluminum cans, paper, and general waste.
- Visual challenges: the objects take many attitudes (lying, standing, crushed), the materials reflect (aluminum cans) or are transparent (PET bottles, a difficulty for depth cameras as well, echoing the limitations of Chapter 9), and the lighting varies (outdoors).
- Building the dataset: this is the core work of the project—photographing recyclables about the campus at various angles, in various lighting, against various backgrounds, building a labeled dataset, and training a YOLO model. This is exactly the method of stage four of the project in Chapter 31.

- System integration: SLAM supplies position (where it was found), YOLO supplies recognition, navigation lets the robot patrol autonomously, and a database records the results.

Planning by Level of Teaching

Stage	Content	Suited to
One	Line following and AprilTag navigation; understanding basic visual navigation	High school
Two	Training YOLO to recognize campus objects; building a dataset	Advanced high school / university
Three	SLAM mapping + semantic labeling; producing a semantic map	University
Four	Autonomous inspection: planning the route, carrying out the task, producing a report	A university project
Five	VLM natural-language tasks; detecting change across sessions	A project / graduate study

Table 24-3 Five Teaching Stages of a Visual Navigation Project

The merit of this plan by levels is that every stage has a result that can be demonstrated, so that students need not wait until the end to see something move; and the technologies of the stages build upon one another, so that the results of the earlier ones are never wasted.

With this, Part VI, Visual SLAM and the Vision-Navigated Robot, is complete. We set out from the fundamentals of features in Chapter 20, passed through the geometry of Chapter 21, the ORB-SLAM2 of Chapter 22, and the ORB-SLAM3 of Chapter 23, and in this chapter settled the technology into real tasks. In the next part we turn our view

to extremely constrained platforms—miniature robots and drones. When the computational resource is only a few milliwatts and the payload only tens of grams, how is SLAM to be realized? Part VII, Miniature Robots, Drones, and Edge AI SLAM, will bring thinking of an altogether different kind.

24.8 The State of Language Navigation: an Honest Assessment of 2025

Section 24.5 described the vision of navigation aided by a vision-language model. This section supplies something equally important: what, with today's technology, it can actually do.

One: a Benchmark for Embodied Navigation

A benchmark published at the end of 2025 gives a fairly clear answer. Its design is this: give the model a photograph of a real scene, a textual instruction, and a type of body (a person, a quadruped robot, a wheeled robot, a bicycle), and have the model draw on the image the trajectory it should take. The dataset has 1,000 situations and over 3,000 expert trajectories, and the scoring considers the shape of the trajectory, the accuracy of the endpoint, and a penalty for places this body ought not to go.

Method	Score
Human expert	75.40
Predicting only the path (told directly where the goal is)	56.55
A straight-line baseline to a known goal	51.89
Predicting the complete trajectory (the best model)	34.38

Predicting only the goal point	29.65

Table 24-4 Evaluation Results of the Embodied Navigation Benchmark (a higher score is better)

Two: Three Numbers Worth Remembering

First, the best model's 34.38 points fall 41.02 points short of the human expert's 75.40. This is not a gap that will be closed in another six months.

Second, and most worth pondering: a straight line to a known goal scores 51.89, far above the best model's 34.38 for a complete trajectory. A straight line that avoids no obstacle, takes no detour, and does not look at the environment at all beats the complete plan of the strongest contemporary model. This shows that the model loses its points chiefly not in the shape of the path but in not having worked out where the goal is.

The third point bears out that judgment: when the goal is given directly, the same model predicting only the path scores 56.55; conversely, predicting only the goal point drops it to 29.65. The model can walk but cannot recognize—mapping the goal in the language to the right place in the image is the real bottleneck at present.

Three: What This Means for the Layered Architecture of This Chapter

These three numbers support directly the position of Sections 24.1 through 24.3: since the model's weakest link is locating the goal, do not let it guess at a position that a marker could settle. A marker put up at the door of the registrar's office answers where is the registrar's office more reliably than any vision-language model.

Conversely, what the model is relatively good at should be handed to it: understanding spoken instructions, reading the content of a frame, generating a report, and

describing a route when the goal is known. The practical question is not whether a language model can do navigation but which parts of navigation it is fit to be given.

This also echoes the low-viewpoint problem of Case 24-2: the model's performance from a robot's point of view falls plainly short of its performance from a human's. Chapter 32 returns to the practice of collecting data and fine-tuning, and Chapter 34 unfolds this whole chain from language to action in full.

Formulas and Procedures

One: Proportional Control for Line Following

Let the width of the image be W, the detected center pixel of the line x_line, and the center of the frame x_center = W / 2; then the lateral error is

$$e = x_line - x_center$$

The most basic proportional controller turns the error directly into an angular rate command:

$$\omega = -K_p \cdot e$$

where K_p is the proportional gain. The minus sign means that when the line is to the right (e positive) the robot should turn right. In practice a derivative term is often added to damp oscillation (PD control), or the linear speed is adjusted by the size of the error—slowing when the error is large and speeding up on a straight run. This is the simplest visual feedback control and the best starting point for understanding visual servoing.

Two: the Control Law of Image-Based Visual Servoing

The aim of IBVS is to bring the current image features s to the target features s*.

Define the feature error

$$e = s - s^*$$

The control law is (driving the error down exponentially)

$$v = -\lambda \cdot L^{+} \cdot e$$

where v is the six-degree-of-freedom velocity command of the camera, λ the rate of convergence, L the image Jacobian (describing how far an image feature moves when the camera moves by one unit), and L^{+} its pseudoinverse. The spirit of this expression is of a piece with the optimization of Chapters 11 and 12: define an error, find the sensitivity of that error to the control variable, and move in the direction in which the error decreases.

Three: the Conversion of a Goal in Semantic Navigation

The core of semantic navigation is converting a semantic goal into a geometric one:

Figure 24-5 The Complete Flow from a Language Instruction to a Motion Command

The user's instruction: please go to the nearest water fountain

↓

① Language understanding (VLM / LLM): parse out the target object class = water fountain

↓

② Query the semantic map: has a water fountain been labeled in the map?

├─ Yes → obtain its three-dimensional coordinates and go straight to ④

└─ No → go to ③ and explore

↓

③ Semantic exploration: choose a promising direction by the VLM's common-sense reasoning

(a water fountain is usually near a corridor or a pantry) → frontier exploration + YOLO detection

↓

④ Geometric path planning: compute the path to the goal on the map (Chapters 3 and 29)

↓

⑤ Local obstacle avoidance and motion control: real-time perception, outputting velocity commands

↓

⑥ Precise docking on arrival (visual servoing, if required)

This flow displays clearly the overall architecture of this book: the upper layer understands semantics with AI, the middle layer supplies geometry with SLAM and planning, and the bottom layer executes the action with control. Not one of the three layers may be dispensed with—the central understanding stressed at the end of Section 24.5.

Four: Fusing an AprilTag Pose Correction

When the robot detects a marker of known world coordinates, it can reckon its own pose from that marker:

$$T_{world \leftarrow robot} = T_{world \leftarrow tag} \cdot T_{tag \leftarrow camera} \cdot T_{camera \leftarrow robot}$$

where T_world←tag is looked up in the marker map (known), T_tag←camera is solved by PnP (Chapter 21), and T_camera←robot is the extrinsic calibration (the camera's mounting position, Chapter 10). Multiplying the three transformations gives the robot's pose in world coordinates—a direct application of composing transformations is multiplying matrices in Chapter 10.

Cases

Case 24-1 From Laying a Line to Understanding Human Speech: Forty Years of Visual Navigation

Arrange the technologies of this chapter in time and a clear line of evolution appears, every step of which corresponds to a rise in the depth at which a robot understands the world.

- The 1980s—line following: the robot recognizes only differences of brightness, and the environment must be modified for it (a line laid down). It does not know where it is or where it is going; it only corrects an error continually.
- The 2000s—landmarks and visual servoing: the robot begins to recognize particular patterns and to localize itself precisely with respect to them. The need to modify the environment is reduced (only markers need be put up) but it has not gone away.
- The 2010s—visual SLAM: the robot can build a map and localize itself, and the environment need no longer be modified. But what it understands is geometry, not meaning.
- The 2020s—semantic and VLM navigation: the robot begins to understand that this is a door and that is a water fountain, and can accept natural-language instructions. At last it can communicate in a human way.

Watching this line, students will find an interesting regularity: what each generation reduces is the preparatory work a human being must do for the robot. From modifying the environment, to labeling landmarks, to merely giving an instruction—the robot adapts more and more to the human world instead of requiring the human to accommodate it.

But here is an important reminder: the new technologies have not made the old ones obsolete. Factory AGVs today still make heavy use of magnetic strip guidance, because in a controlled environment it is the most reliable and the cheapest; precise docking still requires visual servoing, because SLAM is not accurate enough. The engineer's task is not to use the newest technology always but to choose the fittest technology for each problem—the judgment this book has stressed repeatedly since

Chapter 8.

Case 24-2 The Predicament of the Low Viewpoint: When the Model Does Not Know the World as the Robot Sees It

This looks like a technical detail but is in fact of far-reaching effect, and it is the pit students most easily fall into when doing a project.

The situation is this: a student downloads an object detection model that performs excellently on a standard dataset, installs it eagerly on a robot dog or a cleaning robot, and finds the recognition rate dismal—a chair is plainly a chair, and the model cannot make it out; a door is plainly ahead, and the model labels it a wall.

The reason is as the research points out: a quadruped robot operates at a height below 50 centimeters, and vision models trained from a human-centric point of view often do not work well. The photographs of chairs in the training data were mostly taken by a standing person at a height of about 150 centimeters; seen looking up from 30 centimeters, a chair's shape, its perspective, and what occludes what are all entirely different.

This problem has several variants: a cleaning robot sees the lower half of furniture and the texture of the floor; a drone sees the top view of objects; an underwater robot faces images with a severe color cast. What they have in common is that the distribution of the deployment situation does not match that of the training situation.

There is no shortcut—collect data from the target point of view and retrain or fine-tune. That sounds troublesome, but it is exactly the core of stage four of the project in Chapter 31, and the most valuable part of such a project: a local dataset of campus objects from a robot's point of view is in itself a publishable and extensible research

result.

The deeper lesson of this case for students is that an AI model is not a universal black box. Understanding the difference between the distribution of its training data and the circumstances of deployment is a basic skill in using AI responsibly.

Case 24-3 Cooperative Exploration by Several Robots: a VLM as Global Planner

One study mentioned in Section 24.5 deserves unfolding on its own, because it displays the full potential of VLM navigation.

The framework that study proposes integrates a vision-language model as a global planner to achieve multi-robot visual semantic navigation with common-sense reasoning. It aggregates the submaps of several robots from different viewpoints into one unified global map encoding the robots' states and the frontier regions; the VLM then allocates frontiers among the robots, bringing about coordinated and efficient exploration.

What is splendid about this design is that the VLM is not used for low-level control but plays the part of a commander. It looks at the global map and uses common sense to judge which of several unexplored directions is most likely to hold the goal, and then assigns the task to one robot or another. Low-level movement and obstacle avoidance remain the business of traditional planning and control.

The experimental results show that this method surpasses existing baselines in both success rate and navigational efficiency, without any task-specific training; ablation experiments further confirm the importance of the VLM's semantic prior. The research team also verified the framework with a quadruped robot in a real setting.

The lesson for a campus site is that this is precisely the technical path of a project on campus inspection by several robot dogs (Chapters 27 and 31). Several robots explore separately, a VLM coordinates and allocates, each localizes and maps with SLAM, and at last the results are merged into a complete semantic map of the campus—a project topic rich in levels, divisible among a team, and in step with the international frontier.

Hands-On Activities

Activity 24-1 A Staged Implementation from Line Following to Semantic Navigation

Purpose: to build up a capability in visual navigation by degrees, following the five-stage plan of Section 24.7. This activity may be carried to whichever stage the level and resources of the class allow, and every stage has a result that can be demonstrated.

Stage One: Line Following (suitable for high school; no SLAM required)

1. Prepare a teaching cart with a camera (or work in a simulated environment) and lay out on the floor a route with straight stretches and turns.
2. Read the image with OpenCV, convert to grayscale, binarize, and find the center position of the line.
3. Implement the proportional controller of Formula One, turning the lateral error into a steering command so that the cart follows the line by itself.
4. A parameter experiment: adjust the proportional gain K_p and observe the phenomena of it being too small (sluggish response, running wide on a bend) and too large (oscillating from side to side), and find a suitable value.

5. A challenge: add a crossroads or a fork to the route and design rules by which the cart makes a choice.

Stage Two: AprilTag Navigation and Pose Correction (advanced high school / university level)

6. Put several AprilTags up around the classroom, and measure and record the world coordinates of each (building a marker map).
7. Detect the markers with the camera calibrated in Chapter 9 and solve by PnP for the pose of the camera relative to the marker (Chapter 21).
8. By Formula Four, multiply the three transformations to compute the robot's pose in world coordinates and display it in real time.
9. A comparison with SLAM: run visual SLAM at the same time and draw the two trajectories together—observe that the SLAM trajectory is smooth but drifts away by degrees while the marker trajectory jumps about but is always correct (reproducing Case 4-3 of Chapter 4).
10. A simple fusion: implement loose-coupled correction—use SLAM ordinarily and pull the pose back to the value the marker gives whenever one is seen, and observe whether the drift is suppressed.

Stage Three: YOLO Semantic Labeling (university level)

11. Collect data: with a camera at the robot's point of view (mind the mounting height), photograph target objects about the campus—doors, water fountains, fire extinguishers, trash cans, stairheads—at least 200 images of each class covering various angles and lighting.
12. Label them, train a YOLO model, and evaluate its accuracy on a validation set.

13. A viewpoint comparison experiment: photograph another set of test images from a human point of view (about 150 centimeters high) and compare the model's recognition rate at the two viewpoints, reproducing the phenomenon of Case 24-2 quantitatively.
14. Semantic labeling: combine the YOLO detections with RGB-D depth to compute the three-dimensional positions of the objects and label them onto the map built by SLAM, producing a semantic map.

Stage Four: a Semantic Navigation Task (a university-level project)

15. With the semantic map built, implement a simple semantic navigation: input the name of an object (water fountain, say), have the system look up that object's coordinates in the map, plan a path, and navigate to it.
16. Add the handling of moving objects: detect people with YOLO, remove the feature points on them within SLAM, and observe whether the stability of localization improves (the second role of Section 24.4).
17. Implement an inspection task: set several inspection points, have the robot visit them in order, photograph and record each, label them with three-dimensional coordinates, and produce an inspection report at the end.

Stage Five: VLM Assistance (a project / graduate level)

18. Feed the robot's current image to a VLM and ask questions in natural language: is the way ahead passable? Are there stairs in the frame? Where might a door be? Record the accuracy of the answers.
19. Compare the VLM with YOLO: on the same batch of images, compare the ability of the two to recognize open-vocabulary objects (classes not seen in training).

20. Measure the latency: record how long VLM inference takes and assess which layer of the system it belongs in (high-level decisions? real-time control?), against the discussion of limitations in Section 24.5.

Discussion Questions

1. In Stage One, what phenomena do too large and too small a proportional gain produce? Which ideas of control theory does this relate to?
2. In Stage Two, how does the difference between the SLAM trajectory and the AprilTag trajectory change with time? By how much does adding the correction improve matters?
3. In the viewpoint comparison of Stage Three, by how much do the recognition rates at the two viewpoints differ? What warning does this give about simply applying a pretrained model from the internet?
4. In Stage Four, did the stability of localization improve after removing the feature points on people? If there are many people in the scene (a corridor between classes, say), can the system still work?
5. (Project) What is the inference latency of the VLM? If the robot moves at 0.5 meters a second, how far does that latency mean the robot has traveled while thinking? What does this imply for safety design?

Extended Challenge (university level)

First: the recycling-sorting robot—following the plan of Project Two in Section 24.7, build a dataset of campus recyclables (paying particular attention to photographing transparent PET bottles and reflective aluminum cans), train a recognition model, and integrate SLAM so that the robot records what was found where. This topic covers AI, SLAM, and environmental education at once and demonstrates well. Second: a

simplified version of cooperative exploration by several robots—have two robots (or two simulated instances) explore the same site separately, merge their maps afterward (the idea of Atlas in Chapter 23 may be used), and compare the time cooperative and single-robot exploration take to complete coverage, echoing the research direction of Case 24-3.

Further Reading

1. F. Chaumette and S. Hutchinson, "Visual Servo Control, Part I: Basic Approaches" and "Part II: Advanced Approaches," IEEE Robotics & Automation Magazine, 2006–2007. (The authoritative tutorial on visual servoing, with complete accounts of IBVS and PBVS.)
2. E. Olson, "AprilTag: A Robust and Flexible Visual Fiducial System," ICRA, 2011. (The theoretical source of the fiducial markers common to Chapters 4 and 24.)
3. A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision (CLIP)," ICML, 2021. (A milestone in open-vocabulary visual recognition and the basis of VLM navigation.)
4. B. Yu et al., "Co-NavGPT: Multi-Robot Cooperative Visual Semantic Navigation Using Vision Language Models," arXiv, 2023 (revised 2025). (The source of Case 24-3, on cooperative multi-robot exploration with a VLM.)
5. N. Yokoyama et al., "VLFM: Vision-Language Frontier Maps for Zero-Shot Semantic Navigation," ICRA, 2024. (A representative work on zero-shot semantic navigation.)
6. The official Ultralytics YOLO documentation (docs.ultralytics.com). (Training and deployment of object detection models; the principal tool for stage four of the project in Chapter 31 and the semantic labeling of Chapter 32.)

7. The ArUco module and the image processing tutorials of the official OpenCV documentation (docs.opencv.org). (The implementation basis for line following and marker detection.)

Summary

1. Visual SLAM answers where am I and what does the environment look like, while visual navigation answers which way should I go; visual navigation does not necessarily need SLAM (line following and visual servoing belong to the reactive architecture), but only with SLAM added does it gain global planning, navigation to any goal, recovery from a break, and semantic tasks.
2. The three basic strategies: line following is extremely cheap but can only follow a preset route; landmark navigation builds a topological map and is more flexible; visual servoing controls directly by image feedback and can reach millimeter-level relative positioning accuracy, in the two schools of IBVS and PBVS.
3. AprilTag / ArUco has four uses in navigation: absolute pose correction, precise docking, identifying floors and zones, and serving as an approximate ground truth for evaluating SLAM accuracy; fusion with SLAM falls into the three levels of loose coupling, tight coupling, and aiding initialization.
4. YOLO plays three roles in navigation: semantic labeling, the removal of moving objects, and the detection of task targets; but because quadruped and small robots operate at a height below 50 centimeters, models trained from a human-centric point of view often do not work well, and data from the robot's point of view must be collected for fine-tuning.

5. A VLM, pretrained on a vast corpus of images and text, supplies semantic priors and common-sense reasoning, letting a robot accept natural-language instructions and perform new tasks zero-shot; but limited by computational burden, the risk of hallucination, and latency, it suits high-level decisions rather than real-time control.
6. Visual navigation on a robot dog faces four particular challenges: gait vibration, the semantic problem of the low viewpoint, the understanding of three-dimensional terrain, and limits of endurance and computation; the measures taken are tightly coupled multi-sensor estimation and a layered control architecture.
7. Campus inspection and recycling sorting are ideal projects for integrating the technology of this part, and may be planned in the five stages of line following, AprilTag, YOLO semantics, autonomous inspection, and VLM tasks, serving levels from high school to graduate study.

Carrying Forward

This chapter has carried the technology of the four preceding chapters into real tasks, and measured three sets of directly usable numbers along the way: in visual servoing, the position-based camera path stays between 0.70 and 0.82 meters throughout and converges every time, while the image-based path swells rapidly once the starting rotation exceeds 50 degrees and no longer converges at all beyond 150 degrees; for markers, a 16-centimeter marker has a position error of about 5 centimeters at 3 meters, the error growing with the 2.52nd power of distance, and the best viewing angle falls between 10 and 35 degrees (facing squarely meets the pose ambiguity of a plane); and for deployment, tight coupling at a 20-meter spacing has an ATE of 0.153 meters, only

a quarter of loose coupling's, which amounts to halving the number of markers needed.

Part VI, Visual SLAM and the Vision-Navigated Robot, is complete with this. We set out from features and geometry, built a complete visual SLAM system, and at last settled it into real robotic tasks. But all the discussion so far has presumed that the robot has computation and power enough. In the next part we go to the other extreme: when a robot weighs only 44 grams and its computation consumes only milliwatts, is SLAM still possible? Part VII, Miniature Robots, Drones, and Edge AI SLAM, will display the most astonishing engineering achievements of this field.